

딥러닝으로 꼬리손실(Tail Loss) 추정하기

Estimating Tail Loss with Deep Learning

VaR(Value at Risk)를 **LSTM 기반 DeepVaR**와 **트랜스포머·멀티-트랜스포머**로 추정하고, 그 결과를 **백테스팅**으로 검증하는 전 과정을 — 개념·수식·논문·코드까지 — 하나하나 풀어 쓴 노트입니다.

- ▶ Lesson 1-2 · **DeepVaR**: RNN→GRU→LSTM, DeepAR 확률예측, VaR 백테스팅
- ▶ Lesson 3-4 · **Transformer**: 어텐션, 멀티-헤드, 멀티-트랜스포머(MH)
- ▶ 논문 2편 완전 해설 · **Fatouros(2023)** & **Ramos-Pérez(2021)**
- ▶ 코드 3종 완전 해설 · `backtest.py` · DeepVaR · Multi-Transformer

작성
Seungeun Park

생성일
2026-07-04

0. 이 모듈이 하려는 일 — 큰 그림

- 전통적 방법과 이 모듈의 차별점
- 이 모듈의 학습 목표 6가지

LESSON 1 — DeepVaR: VaR가 LSTM을 만나다

- 1.1 왜 신경망인가 — RNN → GRU → LSTM 계보
- 1.2 VaR 백테스팅 — 만든 모델이 진짜 맞는지 검증
- Lesson 1 정리

LESSON 2 — DeepVaR: 파이썬 구현

- 2.1 확률예측(Probabilistic Forecasting)이란
- 2.2 DeepAR — DeepVaR의 중심 부품
- 2.3 DeepVaR의 포트폴리오 VaR
- Lesson 2 정리

LESSON 3 — 트랜스포머로 변동성 예측하기

- 3.1 트랜스포머 소개
- 3.2 인코더-디코더 모델
- 3.3 어텐션(Attention) 직관

LESSON 4 — 멀티-트랜스포머 실전

- 4.1 스케일드 닷-프로덕트 어텐션(Scaled Dot-Product Attention)
- 4.2 멀티-헤드 어텐션(Multi-Head Attention)
- 4.3 멀티-트랜스포머(Multi-Transformer)
- Lesson 4 & 모듈 정리

전체 한눈에 정리 (복습용 핵심 요약)

- ① 개념 계보 — RNN에서 트랜스포머까지
- ② DeepVaR 파이프라인
- ③ 백테스팅(Backtesting)
- ④ 어텐션 한 줄 요약

논문 ① 완전 해설 — DeepVaR (Fatouros et al., Digital Finance, 2023)

- 초록·문제의식
- 기존 VaR의 세 부류와 3대 약점
- DeepVaR의 3가지 핵심 혁신
- VaR의 수학적 정의
- DeepAR의 조건부 분포와 학습
- 백테스팅 수식(논문 정식)
- 포트폴리오 VaR(식 15)
- GARCH·RiskMetrics 벤치마크

- 실험과 결과

논문 ② 완전 해설 — Multi-Transformer (Ramos-Pérez et al., Mathematics, 2021)

- 초록·서론
- 데이터·입력·출력
- 모델 검증
- 벤치마크 GARCH 6종
- 트랜스포머 개조 — 위치 인코더
- 어텐션·멀티-헤드
- 멀티-트랜스포머 — 어텐션에 배경
- 제안 모델 4종·학습 설정
- 결과·결론

코드 완전 해설 ① — Bayer의 backtest.py

- 생성자와 기본 메서드
- 손실함수 4종
- lr_bt — Christoffersen 우도비 검정
- dq_bt — Engle-Manganelli 동적 분위수 검정
- 실행 예시

코드 완전 해설 ② — DeepVaR 노트북

- 데이터 준비(셀 0~7)
- DeepARModel 클래스(셀 8)
- var_p — 포트폴리오 VaR(셀 11)
- 벤치마크 VaRCalculation(셀 13)
- 롤링 윈도우 + 연속 학습(셀 14)

코드 완전 해설 ③ — Multi-Transformer 노트북(3_MT_GARCH.ipynb)

- MultiHeadSelfAttention 클래스 — 표준 어텐션 한 단위
- TransformerBlock 클래스 — ★멀티-트랜스포머 핵심(AMH/배경)★
- GARCH 보조 함수 요약
- 6개 노트북 ↔ 논문 모델 매핑

종합 마무리 — M3 전체를 한 줄로 꿰기

- 두 논문을 관통하는 다섯 가지 공통 교훈
- 다음 단계 예고 — M4: 극단값 이론(EVT)

0. 이 모듈이 하려는 일 — 큰 그림

이 모듈 전체를 관통하는 단 하나의 질문부터 붙잡고 시작하자.

"앞으로 며칠간 우리 포트폴리오가 얼마나 크게 손실을 볼 수 있는가?"

투자를 하는 사람이라면 누구나 밤에 잠들기 전 이 질문을 떠올린다. 그런데 "얼마나 크게"라는 말은 너무 막연하다. 은행에 제출하거나, 리스크 관리 회의에서 쓰거나, 규제 당국에 보고하려면 이걸 **숫자 하나로** 딱 떨어지게 만들어야 한다. 그 숫자를 만들어 주는 대표적인 지표가 바로 **VaR(Value at Risk, 밸류앳리스크)** 다.

VaR는 이렇게 읽으면 된다.

 **예시** — "우리 포트폴리오의 95% VaR가 100만 원이다"라는 말은 "평범한(정상적인) 하루 동안 손실이 100만 원을 넘어갈 확률은 5%뿐"이라는 뜻이다. 뒤집어 말하면 "100일 중 95일은 손실이 100만 원 안쪽에서 머문다"는 이야기다. 여기서 95%를 **신뢰수준(confidence level)**, 나머지 5%를 **꼬리 확률(tail probability)**이라고 부른다.

즉 VaR는 "손실 분포의 왼쪽 꼬리(나쁜 시나리오들)를 어디에서 자를 것인가"를 정하는 **분위수(quantile)** 다. 손실이 클수록 분포의 왼쪽 끝으로 가는데, VaR는 "여기까지가 정상 범위, 이 선을 넘으면 예외적인 사건"이라고 금을 긋는 경계선이라고 생각하면 된다.

 **쉬운 설명** — VaR는 "최악의 손실"이 아니라 "정상 범위의 경계선"이다. 95% VaR가 100만 원이라고 해서 손실이 절대 100만 원을 안 넘는다는 뜻이 결코 아니다. 오히려 100일 중 5일은 넘는 게 정상이고, 그 5일에 손실이 200만 원일지 500만 원일지는 VaR 혼자서는 말해 주지 않는다. VaR는 "얼마나 자주 선을 넘느냐"까지만 약속하는 지표다.

전통적 방법과 이 모듈의 차별점

전통적으로 VaR는 두 가지 방식으로 계산해 왔다.

- **과거분포(historical) 방법**: "과거 500일치 수익률을 늘어놓고 나쁜 쪽에서 5% 지점을 잘라 쓴다." 단순하지만 과거가 미래와 똑같다고 가정한다.
- **GARCH 계열 방법**: 변동성이 시간에 따라 변한다(오늘 시끄러우면 내일도 시끄럽다)는 성질을 통계 모형으로 잡아낸다.

이 모듈은 여기에 **딥러닝(deep learning)**을 접목한다. 접근은 크게 두 갈래다.

접근	핵심 도구	다루는 곳
DeepVaR	LSTM 신경망 + 몬테카를로 시뮬레이션으로 미래 수익률의 확률분포 를 예측하고, 그 분포에서 VaR를 뽑아냄	Lesson 1~2
Transformer / Multi-Transformer	변동성 예측에 어텐션(attention) 기반 신경망을 사용, 여러 트랜스포머를 결합	Lesson 3~4

그리고 이 두 갈래를 관통하는 공통 주제가 하나 있다.

🎯 **핵심** — 아무리 화려한 신경망으로 VaR를 만들어도, "그래서 이 VaR 모델이 진짜 맞느냐?"를 검증하지 못하면 아무 쓸모가 없다. 만든 VaR가 실제 손실을 제대로 예측했는지 사후에 채점하는 절차를 **백테스팅(Backtesting)**이라 부르며, 이것이 이 모듈 전체의 등뼈다. 모델 만드는 법(Lesson 1~4)과 채점하는 법(백테스팅)은 항상 짝으로 다닌다.

이 모듈의 학습 목표 6가지

이 모듈을 마치면 다음 여섯 가지를 할 수 있게 된다.

1. 여러 **유효성 검정(Validity tests)**으로 VaR를 백테스트한다.
2. **DeepAR**의 확률 예측을 VaR 추정에 적용하는 신경망을 설계한다.
3. 여러 트랜스포머를 결합한 **멀티-트랜스포머(Multi-Transformer)**를 구성한다.
4. 시계열의 **롤링 윈도우(rolling window)**에 VaR 추정을 적용한다.
5. **몬테카를로 시뮬레이션(Monte Carlo simulation)**으로 분위수를 추정한다.
6. 트랜스포머를 쓴 모델과 안 쓴 모델의 성능을 비교한다.

💡 **쉬운 설명** — 목표를 한 문장으로 묶으면 이렇다. "신경망으로 손실 분포를 그리고(→ 2, 3, 5번), 그 분포에서 VaR를 뽑아 시간 흐름에 따라 갱신하며(→ 4번), 잘 맞는지 채점하고(→ 1번), 어떤 방법이 더 나은지 비교한다(→ 6번)." 이번 Lesson 1은 이 중에서 5번(몬테카를로)과 1번(백테스팅)의 토대를 놓는 시간이다.

LESSON 1 — DeepVaR: VaR가 LSTM을 만나다

DeepVaR라는 이름은 "Deep learning + VaR"의 합성어다. 큰 그림을 먼저 그려 두면, DeepVaR는 이렇게 작동한다.

1. **LSTM 신경망**이 과거 수익률 시계열을 보고 "내일 수익률이 이런 모양의 확률분포를 따를 것이다"라고 예측한다.
2. 그 분포에서 **몬테카를로 시뮬레이션**으로 가상의 미래 수익률을 수천 번 뽑는다.
3. 뽑은 수천 개의 시나리오 중 나쁜 쪽 5% 지점을 잘라 **VaR**를 얻는다.
4. 만든 VaR가 실제와 맞는지 **백테스팅**으로 채점한다.

이번 레슨에서는 (1)의 재료가 되는 신경망 계보(RNN → GRU → LSTM)를 먼저 이해하고, (4)의 채점 도구인 백테스팅을 촘촘히 배운다.

1.1 왜 신경망인가 — RNN → GRU → LSTM 계보

주거나 수익률 같은 **시계열(time series)** 데이터는 "순서"가 생명이다. 어제 일어난 일이 오늘에 영향을 주고, 오늘이 내일에 영향을 준다. 이 "과거를 기억하며 다음을 처리하는" 능력을 신경망에 심어 준 것이 순환신경망 계열이다.

RNN — 되먹임으로 과거를 기억하다

RNN(Recurrent Neural Network, 순환신경망)의 핵심 아이디어는 단순하다. 보통의 신경망은 입력을 받아 출력을 뱉으면 끝이지만, RNN은 은닉층의 출력을 **다음 시점의 은닉층으로 되먹임(recurrent)** 한다. 그래서 "과거를 기억한 채 다음 입력을 처리"할 수 있다.

 **쉬운 설명** — 소설을 읽는 상황을 떠올려 보자. 지금 문장을 이해하려면 앞 문장에서 무슨 일이 있었는지 기억하고 있어야 한다. RNN에서 은닉층은 "지금까지 읽은 내용의 요약 메모"에 해당하고, 이 메모를 다음 문장을 읽을 때 계속 이어서 참고하는 것이 바로 되먹임이다.

단순 RNN을 시간축으로 쭉 **펼치면(unfolded)**, 마치 여러 개의 셀이 옆으로 늘어선 것처럼 보이지만 사실은 **같은 셀 하나가 시점마다 반복** 등장하는 것이다(가중치를 공유한다). 이 반복 구조에서 각 은닉 셀은 세 종류의 가중치를 학습한다.

1. **다음 시점의 자기 자신**으로 보내는 가중치
2. **다음 시점의 다른 은닉 셀**로 보내는 가중치
3. **출력 셀**로 보내는 가중치

그리고 각 은닉 셀은 **다섯 곳**에서 입력을 받는다.

- 현재 시점의 입력 셀 3개 (현재 들어온 데이터)
- 이전 시점의 자기 자신
- 이전 시점의 다른 은닉 셀

이 중에서 뒤의 두 개, 즉 "**이전 시점의 출력을 다시 받아들이는 것**"이 바로 RNN의 기억 메커니즘이다.

 **예시** — 오늘(t)의 수익률을 처리하는 은닉 셀을 상상하자. 이 셀은 오늘의 입력(3개)뿐 아니라 어제($t - 1$) 자신이 계산해 둔 상태와 어제 옆 셀의 상태까지 함께 받아 오늘의 상태를 만든다. 그렇게 만든 오늘의 상태는 다시 내일($t + 1$)의 계산으로 넘어간다. 이 사슬 덕분에 먼 과거의 정보가 조금씩이나마 현재까지 전달된다.

하지만 RNN에는 치명적인 약점이 있다.

 **주의** — 시퀀스가 길어지면 RNN은 **먼 과거를 잘 기억하지 못한다**. 되먹임을 여러 번 거치는 동안 옛 정보의 신호가 점점 희미해지기 때문인데, 이를 **기울기 소실(vanishing gradient)** 문제라고 부른다. 마치 여러 사람을 거쳐 귓속말을 전달하면 마지막엔 원래 말이 뭉개지는 것과 같다. 300일 전의 급락 신호가 오늘의 예측까지 살아남기 어렵다는 뜻이다.

GRU — 게이트로 정보 흐름을 조절하다

이 약점을 보완하려고 등장한 것이 **GRU(Gated Recurrent Unit)** 다. GRU는 단순 RNN에 **게이트(gate)** 2개를 추가한다.

 **쉬운 설명** — 게이트는 "정보를 얼마나 통과시킬지 0에서 1 사이 숫자로 조절하는 밸브(수도꼭지)"라고 생각하면 된다. 값이 0이면 완전히 잠가 정보를 막고, 1이면 활짝 열어 그대로 흘려보낸다. 0.7이면 70%만 통과시킨다.

GRU의 두 게이트는 다음과 같다.

- **리셋 게이트(reset gate)** — 주로 **단기 의존성**을 담당한다. "직전 기억을 얼마나 초기화(reset)하고 새 입력에 집중할까"를 조절한다.
- **업데이트 게이트(update gate)** — 주로 **장기 의존성**을 담당한다. "옛 기억을 얼마나 유지하고 새 정보를 얼마나 덮어쓸까"를 조절한다.

LSTM — 장·단기 기억의 완성형

LSTM(Long Short-Term Memory)은 게이트를 한 단계 더 정교하게 발전시킨 구조다. 이름 자체가 "긴(Long) 단기(Short-Term) 기억(Memory)", 즉 오래된 기억과 최근 기억을 함께 잘 다루겠다는 뜻이다. LSTM은 네 가지 구성요소로 이루어진다.

구성요소	하는 일 (비유)
입력 게이트(input gate)	무엇을 메모리 셀에 저장 할지 결정 (수첩에 뭘 적을지)
망각 게이트(forget gate)	이전 상태를 얼마나 포함/배제 할지 결정 (수첩에서 뭘 지울지)
출력 게이트(output gate)	새 입력과 업데이트된 메모리 셀을 결합해 다음 은닉층으로 내보냄 (지금 뭘 말할지)
메모리 셀(memory cell)	장기 기억 을 실어 나르는 컨베이어 벨트

 **쉬운 설명** — LSTM을 공장의 컨베이어 벨트로 그려 보자. 벨트(메모리 셀) 위에 장기 기억이 실려 흐른다. 망각 게이트는 벨트 위 낡은 물건을 골라 치우고, 입력 게이트는 새 물건을 벨트에 얹으며, 출력 게이트는 벨트 위 내용 중 지금 당장 필요한 것만 골라 밖으로 꺼낸다. 이 세 밸브 덕분에 중요한 옛 기억은 벨트에 오래 남고, 필요 없는 건 버려져서 기울기 소실 문제가 크게 완화된다.

여기서 반가운 실용적 소식이 하나 있다.

🎯 핵심 — 실제로 우리가 쓸 DeepAR 모듈이 LSTM 내부의 게이트 계산을 거의 다 알아서 처리해 준다. 그러니 게이트 하나하나의 수식을 통째로 외울 필요는 없다. "**LSTM은 게이트로 장기 기억과 단기 기억을 조절하는, 강력해진 RNN이다**"라는 한 문장만 확실히 붙잡고 가면 이번 모듈을 따라가는 데 전혀 지장이 없다.

1.2 VaR 백테스팅 — 만든 모델이 진짜 맞는지 검증

이제 이 모듈의 등뼈인 백테스팅으로 들어가자. 백테스팅은 "과거 데이터에 우리 VaR 모델을 적용해 보고, 실제 손실과 대조해 채점하는" 절차다.

부호 규약 — VaR는 음수다 (아주 중요)

시작 전에 반드시 약속해 둘 것이 있다. **이 논문에서 VaR는 음수다**. 손실을 "음의 수익률"로 표현하기 때문이다.

⚠ 주의 — 일상에서는 "VaR가 100만 원(손실 100만 원)"처럼 양수로 말하지만, 이 모듈의 수식과 코드에서는 손실을 음수 수익률로 다룬다. 예컨대 하루 3% 손실은 -0.03 , 3% 이익은 $+0.03$ 이다. 따라서 95% VaR도 -0.03 처럼 **음수**로 표기된다. 이 부호 규약을 놓치면 아래 부등식의 방향이 전부 헛갈리니 꼭 기억하자.

히트 변수 — 손실이 선을 넘었는가?

백테스팅의 가장 기본 재료는 **히트 변수(Hit Variable)** I_t 다. "오늘 실제 손실이 VaR 경계선을 넘었는가?"를 0 또는 1로 기록하는 스위치다.

$$I_t = \begin{cases} 1 & \text{if } \text{PnL}_t < \text{VaR}_t \quad (\text{실제 손실이 VaR 임계값 초과}) \\ 0 & \text{그 외} \end{cases}$$

기호를 하나하나 풀어 보자.

- I_t — t 시점의 히트 변수다. 값은 0 또는 1, 두 값만 갖는다.
- PnL_t — 시점 t 의 실제 손익. **PnL**은 **Profit and Loss(손익)**의 약자로, 그날 실제로 실현된 수익률이다.
- VaR_t — 시점 t 에 우리가 예측해 둔 VaR 경계선(음수).
- $<$ — "왼쪽이 오른쪽보다 더 작다(더 아래, 더 큰 손실이다)"는 부등호.

부호 규약 때문에 부등식 방향에 주의해야 한다. VaR가 음수이고 실제 수익률이 **그보다 더 아래로** 떨어졌다면(= 예측한 것보다 더 큰 손실), $I_t = 1$ 이 된다. 이때를 "**히트(Hit)**" 또는 "**예외(exceedance)**"가 발생했다고 말한다.

📝 예시 — 오늘 VaR를 $\text{VaR}_t = -0.03$ (즉 "−3%까지는 정상 범위")로 예측했다고 하자. 그런데 실제로 시장이 급락해 수익률이 $\text{PnL}_t = -0.05$ (−5%)를 기록했다. $-0.05 < -0.03$ 이므로 실제 손실이 경계선을 넘었고, 따라서 $I_t = 1$ (히트)이다. 반대로 오늘 수익률이 -0.01 이었다면 $-0.01 < -0.03$ 이 거짓이므로 $I_t = 0$ 이다.

"예상 위반" — 히트는 나야 정상이다

여기서 초보자가 가장 많이 오해하는 지점을 짚자. 95% VaR라면 대략 **5%의 날에 히트가 나야 정상**이다. 100일이면 5번, 250 거래일이면 약 12~13번. 99% VaR라면 100일 중 1번꼴이다. 논문은 이 히트들을 "**예상 위반(Expected Violations)**"이라고 부른다.

🎯 **핵심** — 이름이 "위반"이라 마치 모델이 뭔가를 어긴 것처럼 들리지만, 전혀 그렇지 않다. 신뢰수준상 **당연히 나와야 하는 초과 횟수**를 가리킬 뿐이다. 95% VaR인데 100일 동안 히트가 한 번도 없다면 오히려 VaR를 너무 크게 (보수적으로) 잡아 자본을 낭비하고 있다는 신호다. "손실이 절대 경계선을 안 넘는다"는 착각이야말로 더 위험하다.

측정치와 손실함수

이제 히트를 바탕으로 모델의 "부정확한 정도"를 수치화하는 여러 측정치를 배운다. 아래 코드는 모두 Bayer의 `backtest.py`에 있는 `backtest` 클래스의 메서드들이며, 이 클래스는 세 가지 입력을 받는다.

- `actual` — 실제 수익률 시계열 (위 수식의 PnL, actual)
- `forecast` — 예측한 VaR 시계열 (위 수식의 VaR, forecast)
- `alpha` — 유의수준 α (예: 95% VaR면 $\alpha = 0.05$)

① **히트 시리즈 (Hit Series)** — 앞서 정의한 I_t 를 시계열 전체에 대해 계산한다.

```
def hit_series(self):  
    return (self.actual < self.forecast) * 1 # 불리언(True/False)을 1/0으로 변환
```

`self.actual < self.forecast`는 각 날짜마다 `True` / `False`를 담은 배열을 만든다. 여기에 `* 1`을 곱하는 것은 불리언을 0/1 숫자로 바꾸는 간단한 방법이다(`True`는 1, `False`는 0으로 계산됨).

② **위반율 (violation rate)** — 히트가 전체에서 차지하는 비율이다.

$$\text{violation rate} = \frac{1}{T} \sum_{t=1}^T I_t$$

- T — 백테스트 기간의 전체 날짜 수.
- $\sum_{t=1}^T I_t$ — $t = 1$ 부터 T 까지 히트 변수를 모두 더한 값, 즉 **총 히트 횟수**.
- $\frac{1}{T}$ — 전체 날짜 수로 나눠 평균(비율)으로 만드는 것.

```
def hit_rate(self):  
    return self.hit_series().mean() # 0/1 배열의 평균 = 1(히트)의 비율
```

0과 1로 이루어진 배열의 평균은 곧 "1이 차지하는 비율"과 같다. 95% VaR라면 이 값이 이상적으로 0.05 근처여야 한다.

📝 **예시** — 250 거래일을 백테스트했더니 히트가 12번 났다고 하자. 그러면 위반율은 $\frac{12}{250} = 0.048$ 이다. 목표인 0.05에 아주 가까우니 이 VaR 모델은 "초과 빈도" 측면에서 합격점에 가깝다. 만약 히트가 30번이었다면 $\frac{30}{250} = 0.12$ 로, 위험을 심각하게 **과소평가**하고 있다는 경고다.

③ **이차 손실 (Quadratic Loss)** — 히트가 난 날, 얼마나 크게 빗나갔는지를 제곱으로 벌점 매긴다.

$$QL_t = I_t \cdot (1 + (\text{actual}_t - \text{forecast}_t)^2)$$

- QL_t — 시점 t 의 이차 손실.

- I_t — 히트 변수(0 또는 1). 앞에 곱해져 있어 **히트가 안 난 날**($I_t = 0$)은 **통째로 0**이 된다.
- $(\text{actual}_t - \text{forecast}_t)$ — 실제 수익률과 예측 VaR의 차이, 즉 경계선을 얼마나 넘어서는지(초과 폭).
- $(\dots)^2$ — 그 초과 폭을 제곱. 많이 빗나갈수록 벌점이 급격히 커진다.
- $1 +$ — 히트가 난 날엔 최소 기본 벌점 1을 깔고, 거기에 초과 폭의 제곱을 더한다.

```
def quadratic_loss(self, return_mean=True):
    loss = (self.hit_series() * (1 + (self.actual - self.forecast)**2))
```

 **예시** — 히트 난 날 $\text{actual} = -0.05$, $\text{forecast} = -0.03$ 이라고 하자. 그러면 $QL = 1 + (-0.05 - (-0.03))^2 = 1 + (-0.02)^2 = 1.0004$ 이다. 만약 더 크게 빗나가 $\text{actual} = -0.10$ 이었다면 $QL = 1 + (-0.07)^2 = 1.0049$ 로 벌점이 더 커진다. 히트가 안 난 날은 $I_t = 0$ 이라 $QL = 0$ 으로 기여하지 않는다. 즉 "초과한 날만, 초과 폭이 클수록 크게" 벌하는 손실이다.

④ **부드러운 손실 (Smooth Loss)** — 딱딱한 0/1 스위치를 매끄러운 곡선으로 바꾼 손실이다.

$$SL_t = \left(\alpha - \frac{1}{1 + e^{\delta(\text{actual}_t - \text{forecast}_t)}} \right) \cdot (\text{actual}_t - \text{forecast}_t)$$

- SL_t — 시점 t 의 부드러운 손실.
- α — 유의수준(예: 0.05).
- δ — 곡선의 가파른 정도를 정하는 값(논문의 d , 기본 25).
- e — 자연상수(약 2.718), 지수함수의 밑.
- $(\text{actual}_t - \text{forecast}_t)$ — 앞서와 같은 초과 폭.
- $\frac{1}{1 + e^{\delta(\dots)}}$ — **시그모이드(sigmoid)** 함수 꼴.

```
def smooth_loss(self, delta=25, return_mean=True):
    loss = ((self.alpha - (1 + np.exp(delta*(self.actual - self.forecast)))**(-1))
            * (self.actual - self.forecast))
```

여기서 핵심은 시그모이드다. 일반적으로 $\frac{1}{1+e^{-x}}$ 꼴을 **시그모이드**라 부르며, 어떤 실수든 0과 1 사이의 값으로 부드럽게 눌러 준다.

 **쉬운 설명** — 왜 굳이 부드럽게 만들까? 히트 변수 I_t 는 0에서 1로 **뚝 끊기는 계단**이라 미분이 안 된다. 그런데 신경망은 미분(기울기)으로 학습하기 때문에 계단 함수는 최적화에 아주 불리하다. 그래서 계단을 시그모이드라는 **매끄러운 S자 곡선**으로 근사해, 어디서든 기울기를 구할 수 있게 만든 것이다.

delta (논문의 $d = 25$)는 이 S자 곡선을 얼마나 **계단에 가깝게(가파르게)** 만들지를 조절한다. δ 가 클수록 곡선이 가팔라져 진짜 계단 함수에 근접한다.

 **예시** — $\alpha = 0.05, \delta = 25$, 히트 난 날 $\text{actual} = -0.05, \text{forecast} = -0.03$ 을 넣어 보자. 그러면 $x = \text{actual} - \text{forecast} = -0.02$ 이고, $\delta x = -0.5, e^{-0.5} \approx 0.607$ 이므로 $\frac{1}{1+0.607} \approx 0.622$ 가 된다. 따라서 $SL = (0.05 - 0.622) \times (-0.02) \approx 0.0114$ 로 양수 벌점이 나온다. 만약 δ 값을 250 으로 키우면 $\frac{1}{1+e^{-5}} \approx 0.993$ 이 되어 $SL \approx 0.019$ 로, 아래 틱 손실 값에 거의 딱 붙는다. 즉 δ 값을 키울수록 부드러운 근사가 딱딱한 계단에 수렴한다.

⑤ **틱 손실 (Tick Loss)** — 분위수 회귀에서 쓰는 비대칭 손실이다.

$$TL_t = (\alpha - I_t) \cdot (\text{actual}_t - \text{forecast}_t)$$

- TL_t — 시점 t 의 틱 손실.
- $(\alpha - I_t)$ — 히트 여부에 따라 달라지는 가중치다. 히트 안 난 날은 $I_t = 0$ 이므로 이 값이 α , 히트 난 날은 $I_t = 1$ 이므로 이 값이 $\alpha - 1$ 이 된다.
- $(\text{actual}_t - \text{forecast}_t)$ — 초과 폭.

```
def tick_loss(self, return_mean=True):
    loss = (self.alpha - self.hit_series()) * (self.actual - self.forecast)
```

초과한 날과 안 한 날에 서로 다른 가중치(α 대 $\alpha - 1$)를 매기는 것이 핵심이다. 이렇게 좌우를 다르게 벌하는 것을 **분위수 회귀(quantile regression)**의 비대칭 손실이라 부른다.

 **쉬운 설명** — 왜 비대칭일까? 우리는 "5% 분위수"라는 특정 지점을 맞히고 싶다. 이 점보다 아래로 벗어난 오차와 위로 벗어난 오차를 똑같이 벌하면 결국 중앙값(50%)을 맞히게 된다. 5% 지점을 정확히 맞히려면 한쪽 방향의 오차를 다른 쪽보다 더(혹은 덜) 아프게 벌해야 하고, 그 비율을 정하는 것이 바로 α 와 $\alpha - 1$ 이다.

⑥ **펌 손실 (Firm Loss)** — 이차 손실에 "너무 보수적인 것"에 대한 벌점을 더한 손실이다.

$$FL_t = I_t(1 + (\text{actual}_t - \text{forecast}_t)^2) - c(1 - I_t)\text{forecast}_t$$

- FL_t — 시점 t 의 펌 손실.
- $I_t(1 + (\text{actual}_t - \text{forecast}_t)^2)$ — 앞의 이차 손실과 똑같은 항. 히트 난 날 초과 폭을 벌한다.
- $(1 - I_t)$ — 히트가 **안** 난 날에만 1이 되는 스위치(히트 난 날은 0).
- c — 기회비용의 크기를 정하는 계수(논문의 $a = 1$, 코드의 **c=1**).
- forecast_t — 그날의 VaR 예측(음수).
- $-c(1 - I_t)\text{forecast}_t$ — 히트 안 난 날, VaR를 보수적으로 크게 잡아 놓은 것에 대한 기회비용 항.

```
def firm_loss(self, c=1, return_mean=True):
    loss = (self.hit_series() * (1 + (self.actual - self.forecast)**2)
            - c*(1-self.hit_series()) * self.forecast)
```

 **쉬운 설명** — 앞의 이차 손실은 "히트가 나면 벌한다"만 신경 쓴다. 그래서 VaR를 무조건 아주 크게(예: -0.5) 잡아 버리면 히트가 거의 안 나서 벌점이 0에 가까워진다. 하지만 현실에서 VaR를 그렇게 크게 잡으면 그만큼 많은 자본을 안전 마진으로 쌓아 놓느라 **놀리게 되어 손해**다. 펌 손실은 바로 이 낭비를 벌하려고 기회비용 항을 덧붙인 것이다.

부호 규약을 떠올리면 이 항의 방향이 이해된다. 히트 안 난 날 이 항은 $-c \cdot \text{forecast}_t$ 가 되는데, forecast_t (VaR)가 음수이므로 결과는 **양수 벌점**이 되고, VaR를 더 크게(더 음수로) 잡을수록 벌점이 커진다.

 **예시** — 히트 안 난 날 $c = 1$, $\text{forecast} = -0.03$ 이면 기회비용 항은 $-1 \times (-0.03) = +0.03$ 이다. 만약 VaR를 훨씬 보수적으로 $\text{forecast} = -0.10$ 으로 잡았다면 $-1 \times (-0.10) = +0.10$ 으로 벌점이 세 배 이상 커진다. 즉 "히트를 피하려고 VaR를 과하게 키우면 조용한 날마다 대가를 치른다"는 뜻이다. 논문의 계수 a 와 코드의 c 는 같은 역할을 한다.

 **주의** — 위 여섯 손실함수 코드에는 모두 `return_mean` 옵션이 있다. 이는 각 날짜별 손실을 그냥 **합할지, 아니면 평균 낼지**를 고르는 스위치일 뿐이다. 기본값 `True` 면 평균을 돌려준다. 손실의 정의 자체를 바꾸는 게 아니니 헛갈리지 말자.

유효성 검정 — 두 근본 가정을 통계로 따진다

위의 측정치들이 "얼마나 부정확한가(정도)"를 재는 자라면, **유효성 검정(Validity tests)**은 "이 VaR가 통계적으로 타당한가"를 예/아니오로 판정한다. 검정은 두 가지 **근본 가정**을 따진다.

(A) Christoffersen 우도비 검정 (Likelihood Ratio test) — 사실 두 개의 검정을 하나로 묶은 것이다.

- **무조건부 커버리지(unconditional coverage)** — "초과가 일어날 확률이 정말 $(1 - \text{신뢰수준})$ 인가?"를 따진다. 95% VaR인데 실제 초과율이 그보다 높으면 위험을 **과소평가**, 낮으면 **과대평가**하고 있다는 뜻이다.
- **독립성(independence)** — "초과들이 서로 독립인가?"를 따진다. 오늘 초과가 났다는 사실이 내일 초과 여부에 대해 어떤 정보도 주면 안 된다.

 **핵심** — 이 두 검정을 왜 함께 봐야 할까? 전체 초과 횟수(빈도)가 딱 맞더라도, 그 초과들이 **특정 위기 시기에 우르르 몰려서** 터지면 대단히 위험하다. 이렇게 초과가 한 시기에 뭉치는 현상을 **군집(clustering)**이라 부른다. 빈도가 맞는지는 무조건부 커버리지가, 몰려 있지 않은지는 독립성이 잡아낸다. 이 검정은 Bayer의 `lr_bt` 함수로 구현되어 있다.

 **예시** — 250일 중 초과가 정확히 12번(약 5%) 났다고 하자. 무조건부 커버리지만 보면 완벽하다. 그런데 그 12번이 모두 한 달 안에 몰려서 터졌다면? 그 한 달 동안 회사는 연속 손실로 파산할 수도 있다. 독립성 검정은 바로 이런 "빈도는 맞지만 시기가 몰린" 위험을 걸러낸다.

(B) Engle & Manganelli 동적 분위수 검정 (Dynamic Quantile test) — 초과들의 시간적 독립성을 더 정교하게 따지는 검정이다.

과거의 초과 기록들에 **선형회귀(linear regression)**를 적합한 뒤, "회귀계수가 전부 0인가?"를 **카이제곱(chi-square)** 검정으로 판정한다. 만약 기각되지 않으면(계수가 0이라고 봐도 무방하면), 최근의 초과가 과거의 초과와 **독립**이라는 뜻이다.

 **쉬운 설명** — 회귀계수가 0이 아니라는 것은 "과거 초과로 오늘 초과를 어느 정도 예측할 수 있다"는 뜻이고, 이는 초과들이 서로 얽혀 있다(독립이 아니다)는 나쁜 신호다. 반대로 계수가 전부 0이면 과거가 현재를 못 맞힌다는 것이고, 이는 초과가 무작위로 흩어져 있다는 좋은 신호다. 이 검정은 Bayer의 `dq_bt` 함수로 구현되어 있다.

Lesson 1 정리

이번 레슨의 요지를 한자리에 모으자.

🎯 **핵심** — DeepVaR는 LSTM으로 미래 수익률의 확률예측을 만들고, 거기에 몬테카를로 시뮬레이션을 걸어 수천 개의 가상 시나리오를 뽑은 뒤 그 나쁜 쪽 꼬리에서 VaR를 잘라낸다. 그리고 이렇게 만든 VaR가 진짜 맞는지는, 위반율·이차/부드러운/틱/편 손실 같은 **측정치**와 Christoffersen(무조건부 커버리지 + 독립성) · Engle-Manganelli 동적 분위수 같은 **유효성 검정**으로 백테스팅해서 채점한다.

- 신경망 계보 **RNN** → **GRU** → **LSTM**은 "과거를 얼마나 잘 기억하느냐"의 진화사이며, 우리는 그 완성형인 LSTM을 쓴다. 게이트 수식은 DeepAR 모듈이 처리하니 원리만 이해하면 된다.
- 백테스팅의 출발점은 **히트 변수** I_t 이고, 부호 규약상 **VaR**는 음수이며, 95% VaR라면 **약 5%의 히트는 나야 정상** ("예상 위반")이다.
- 측정치는 "얼마나 부정확한가(정도)"를, 유효성 검정은 "빈도가 맞는가 + 초과가 물리지 않는가(타당성)"를 본다. 둘은 항상 짝으로 확인해야 한다.

다음 Lesson 2에서는 이 백테스팅 잣대를 손에 든 채, DeepAR 신경망으로 실제 확률예측과 VaR를 만들어 내는 과정으로 들어간다.

LESSON 2 — DeepVaR: 파이썬 구현

레슨 1에서 우리는 VaR가 "평상시 시장에서 하루에 잃을 수 있는 최대 손실"을 특정 신뢰수준에서 재는 지표라는 것, 그리고 정규분포 가정 같은 전통적 방법이 금융 수익률의 두꺼운 꼬리를 자주 과소평가한다는 한계를 봤어요. 이번 레슨에서는 그 한계를 딥러닝으로 넘어서는 **DeepVaR**(딥러닝 기반 VaR 프레임워크)를 파이썬으로 직접 조립해 봅니다.

큰 그림은 딱 한 문장이예요. **"값 하나를 맞히려 하지 말고, 내일 수익률의 '분포 전체'를 예측한 다음, 그 분포의 꼬리에서 VaR를 읽어내자."** 이 문장을 세 조각으로 나눠서, (2.1) 왜 분포 전체가 필요한지, (2.2) 그 분포를 만들어 주는 부품 DeepAR가 무엇인지, (2.3) 거기서 포트폴리오 VaR를 어떻게 뽑는지 순서로 풀어 갈게요.

 **핵심** — DeepVaR의 출발점은 "예측 = 숫자 하나"라는 습관을 버리는 거예요. VaR는 분포의 꼬리 한 점을 묻기 때문에, 꼬리를 알려면 분포 전체를 먼저 예측해야 합니다.

2.1 확률예측(Probabilistic Forecasting)이란

우리가 "예측"이라고 하면 보통 값 하나를 떠올려요. "내일 이 종목 종가는?" → "72,000원." 이렇게 미래를 숫자 하나로 답하는 걸 **점 추정(point estimate)** 이라고 합니다. 평범한 회귀모형이 내놓는 답이 대개 이 형태죠.

그런데 VaR는 성격이 달라요. VaR는 "평균적으로 얼마?"가 아니라 "나쁜 쪽 극단에서 얼마?"를 묻습니다. 구체적으로는 내일 수익률 분포의 특정 **분위수(quantile)** 값이에요. 예를 들어 99% VaR는 "가장 나쁜 1%가 시작되는 경계"의 수익률을 봅니다. 실무에서 자주 쓰는 분위수가 바로 0.95, 0.99 (또는 손실 꼬리 쪽으로 뒤집어 보면 0.05, 0.01)예요. 분위수를 수식으로 적으면 이렇습니다.

$$\Pr(X \leq q_\alpha) = \alpha$$

기호를 하나씩 풀어 볼게요.

- X = 내일 수익률이라는 **확률변수(random variable)**. 아직 실현되지 않아서 여러 값이 가능한, "미래의 값"이에요.
- q_α = 우리가 찾는 α **분위수**. "이 값보다 작을 확률이 정확히 α 가 되는 경계선"이라는 뜻입니다.
- $\Pr(\cdot)$ = 괄호 안 사건이 일어날 확률.
- α = 꼬리 확률. 예컨대 $\alpha = 0.01$ 이면 $q_{0.01}$ 은 "1%의 최악 시나리오가 시작되는 수익률"이 됩니다.

여기서 핵심은, 분위수 q_α 를 알려면 **분포 전체의 모양을 알아야 한다**는 점이에요. 평균 하나만으로는 "왼쪽 1% 지점이 어디냐"에 답할 수 없거든요. 그래서 우리는 예측값 하나가 아니라 **예측 분포 전체**를 만들어야 합니다. 이렇게 미래를 분포째로 예측하는 걸 **확률예측(probabilistic forecast)** 이라고 부릅니다.

 **쉬운 설명** — 점 추정은 일기예보에서 "내일 기온 28도"라고 말하는 것이고, 확률예측은 "28도가 가장 유력하지만 24~32도에 퍼져 있고, 폭염(34도 이상)일 확률은 5%"라고 분포째 말하는 거예요. VaR는 바로 이 "5% 극단"을 묻기 때문에, 점 하나로는 절대 답이 안 나오고 분포 전체가 필요합니다.

 **예시** — 내일 수익률이 평균 0%, 표준편차 1.2% 근처에 퍼져 있다고 해봐요. 99% VaR는 $\alpha = 0.01$ 분위수, 즉 아주 왼쪽 꼬리라서 대략 -2.8% 같은 음수가 나옵니다. 만약 평균 0%만 예측했다면 이 -2.8%는 영영 알 수 없어요.

구현에는 **GluonTS**(`gluonts`), 확률예측에 특화된 시계열 딥러닝 툴킷)를 씁니다. 일반적인 회귀 라이브러리와 달리 처음부터 "분포를 내놓는" 걸 목표로 설계돼 있어서 VaR와 궁합이 좋아요. 예제 데이터는 여러 **외환(FX)** 시계열이고, 코드는 데이터를 가져오고(`get`) · 정리하고(`clean`) · 형식을 맞추고(`format`) · 변환하고(`transform`) · 보여주는(`display`) 전형적인 전처리 흐름을 따릅니다.

2.2 DeepAR — DeepVaR의 중심 부품

DeepVaR의 심장은 **DeepAR**(Deep AutoRegressive, 딥 자기회귀 모델)입니다. 이름 그대로 과거 값을 입력으로 받아 다음 값을 이어 예측하는 자기회귀 구조를, **RNN(순환신경망, Recurrent Neural Network)** 으로 구현한 확률예측 모델이에요. GluonTS에서는 `DeepAREstimator` 로 불러오는데, 주요 파라미터부터 표로 정리해 볼게요.

파라미터	의미	기본값
<code>freq</code>	데이터 빈도(일별·시간별 등)	(필수)
<code>prediction_length</code>	예측 구간의 길이(몇 스텝 앞까지 예측할지)	(필수)
<code>trainer</code>	학습을 담당하는 학습기(Trainer) 객체	—
<code>context_length</code>	예측 전에 RNN을 몇 스텝 펼쳐 과거를 볼지	<code>prediction_length</code>
<code>num_layers</code>	RNN 층 수	<code>2</code>
<code>num_cells</code>	층마다의 셀(뉴런) 수	<code>40</code>
<code>cell_type</code>	RNN 셀 종류(<code>'lstm'</code> 또는 <code>'gru'</code>)	<code>'lstm'</code>
<code>dropout_rate</code>	과적합 방지용 드롭아웃 비율	<code>0.1</code>
<code>distr_output</code>	관측 평가·예측 샘플링에 쓸 분포	<code>StudentTOutput()</code>

몇 개는 감이 잘 안 올 테니 풀어 볼게요. `context_length` 는 "예측 전에 과거 며칠 치를 훑어볼까?"이고, 기본값은 `prediction_length` 와 같게 맞춰져 있어요. `cell_type` 의 두 후보인 **LSTM(Long Short-Term Memory)** 과 **GRU(Gated Recurrent Unit)** 는 둘 다 "긴 과거를 잘 기억하도록" 설계된 RNN 셀입니다. `distr_output` 은 뒤에서 자세히 볼 "어떤 분포로 세상을 볼 것인가"를 정하는데, 기본이 **스튜던트 t 분포(Student-t distribution)** 예요.


쉬운 설명 — `context_length` 는 시험 직전 "과거 노트를 며칠 치 복습할까?"와 같아요. 너무 짧으면 흐름을 놓치고, 너무 길면 옛날 정보에 휘둘리며 학습이 무거워집니다. `dropout_rate=0.1` 은 학습 중 뉴런의 약 10%를 무작위로 꺼서 모델이 특정 신호에 과하게 의존(과적합)하지 못하게 하는 **드롭아웃(dropout)** 장치예요.

이제 DeepAR 학습의 진짜 핵심입니다. DeepAR는 값을 직접 맞히는 게 아니라 **분포의 파라미터를 학습**해요. 그러려면 먼저 "수익률이 어떤 분포에서 나온다"는 우도 가정을 세워야 하는데, 그 기본값이 스튜던트 t 분포입니다. 이유는 명확해요. 금융 수익률은 정규분포보다 극단값(폭락·폭등)이 훨씬 자주 나오는데, 스튜던트 t는 그 **두꺼운 꼬리**를 잘 잡아 주거든요.

💡 쉬운 설명 — 스튜던트 t 분포는 정규분포와 종 모양은 비슷하지만 꼬리가 더 두꺼워요. 자유도 ν 가 작을수록 꼬리가 두꺼워져서, 가끔 터지는 대폭락 같은 극단값을 정규분포보다 훨씬 자연스럽게 설명합니다. 금융 수익률이 딱 이런 성질이라, DeepAR가 이걸 기본 분포로 채택한 거예요.

그러면 "좋은 분포 파라미터"란 무엇일까요? 목표는 **우리가 실제로 관측한 학습 데이터가 가장 그럴듯해(most likely) 보이도록** 분포 파라미터를 맞추는 것입니다. 이걸 정식 용어로 **로그우도(log likelihood) 최대화**라고 해요. Salinas 등 (DeepAR 원논문)의 학습 목표식을 개념적으로 쓰면 다음과 같습니다.

$$\mathcal{L} = \sum_{i=1}^N \sum_{t=t_0}^T \log \ell(z_{i,t} | \theta(h_{i,t}, \Theta))$$

기호를 하나하나 풀어 볼게요.

- $\mathcal{L}$ = 우리가 **최대화**하려는 대상, 즉 전체 로그우도의 합(목적함수).
- $\sum_{i=1}^N$ = N 개의 시계열(종목/상품)에 대해 모두 더한다는 뜻.
- $\sum_{t=t_0}^T$ = 각 시계열에서 시점 t_0 부터 T 까지 모든 시점을 더한다는 뜻.
- $\log$ = 자연로그. 곱을 합으로 바꿔 계산을 안정시키려고 씁습니다.
- ℓ = 필기체로 쓴 **우도(likelihood)**. "이 파라미터라면 관측값 $z_{i,t}$ 가 나올 그럴듯함"의 크기예요.
- $z_{i,t}$ = 시계열 값. 아래첨자에서 i 는 종목/상품, t 는 시점을 가리킵니다.
- $\theta(h_{i,t}, \Theta)$ = 그 시점의 분포 파라미터. 여기서 θ 는 **신경망이 학습하는 파라미터 전체**로, 결정론적 함수의 출력 $h_{i,t}$ 와 신경망 가중치 Θ 를 받아 "그 시점에 쓸 분포 모양"을 만들어 냅니다.
- $h_{i,t}$ = **결정론적(deterministic) 함수** h 가 (i, t) 에서 내놓는 출력값(RNN의 은닉상태라고 보면 돼요).
- 세로줄 $|$ = "~가 주어졌을 때"라는 조건부 표시.

📝 예시 — 어제 수익률이 -2%로 관측됐다고 해요. 분포 A(평균 0%, 표준편차 1%)에서 -2%는 2 표준편차 밖이라 우도 ℓ 이 작고, 분포 B(평균 0%, 표준편차 2%)에서는 1 표준편차라 우도가 큼니다. -2% 같은 사건이 자주 나오면 학습은 $\log \ell$ 이 더 큰 B 쪽으로 파라미터를 밀어줘요. 이렇게 모든 시점의 $\log \ell$ 을 더한 것이 $\mathcal{L}$ 이고, 그걸 키우는 게 학습입니다.

이 식에서 특히 중요한 건 $h_{i,t}$ 가 **결정론적**이라는 사실이에요. 결정론적이라는 말은 이 함수가 완전히 관측된 변수들만의 함수라는 뜻입니다. 즉 숨어 있는 **잠재변수(latent variable)**가 없어요. 잠재변수가 없으니 "숨은 변수를 추정하는" 별도의 **추론(inference)** 단계가 필요 없고, 함수 h 를 **확률적 경사하강법(SGD, Stochastic Gradient Descent)**으로 곧장 최적화할 수 있습니다. 파라미터에 대해 기울기를 바로 계산할 수 있다는 뜻이죠.

💡 쉬운 설명 — 잠재변수가 있으면 "보이지 않는 원인부터 추정하고 → 그다음 파라미터를 고치는" 두 단계 씨름이 필요해요. DeepAR는 $h_{i,t}$ 가 관측값만으로 계산되니 그 씨름이 통째로 사라집니다. 그래서 목적함수 $\mathcal{L}$ 을 파라미터로 미분해 곧바로 경사하강법으로 밀어 올릴 수 있어요.

실제 논문에서 DeepVaR는 **Adam 옵티마이저(Adam optimizer)**를 씁니다. GluonTS의 `gluonts.mx.trainer` 소스를 열어 보면 `optimizer = mx.optimizer.Adam`으로 확인돼요. Adam은 각 파라미터마다 학습 속도를 알아서 조절해 주는, 실전에서 안정적이고 빠른 경사하강법 변형입니다.

🎯 **핵심** — Estimator가 `fit` 으로 적합되고 나면, 단순히 "모델 종류와 인자"만 가진 설계도가 아니라 모든 계수·하이퍼파라미터·가중치까지 담은 **완성된 모델**이 됩니다. 즉 이제 새로운 시점에 대해 분포를 뿜고, 거기서 샘플을 뽑아 VaR를 계산할 준비가 끝난 상태예요.

2.3 DeepVaR의 포트폴리오 VaR

이제 학습된 DeepAR가 각 종목마다 미래 수익률 분포(정확히는 그 분포에서 뽑은 샘플들)를 내놓습니다. 이것 포트폴리오 전체의 VaR로 합쳐 주는 게 리포지토리의 `var_p` 함수예요.

```
def var_p(predictions, returns, weights, days_ahead=0, alpha=95):
    V = np.zeros(len(weights))
    for i in range(len(weights)):
        if weights[i] < 0 :
            V[i] = weights[i] * np.percentile(predictions[i].samples[:,days_ahead], alpha)
        else:
            V[i] = weights[i] * np.percentile(predictions[i].samples[:,days_ahead], 100-alpha)
    R = returns.corr()
    return -np.sqrt(V @ R @ V.T)
```

코드를 한 줄씩 따라가 볼게요.

- `V = np.zeros(len(weights))` — 종목 수만큼 0으로 채운 벡터 **V**를 준비합니다. 여기에 종목별 "가중 VaR"를 채워 넣을 거예요.
- `for i in range(len(weights))` — 종목을 하나씩 돌면서,
- `predictions[i].samples[:, days_ahead]` — *i*번째 종목의 예측 샘플 중 `days_ahead` (기본 0, 즉 하루 앞) 시점의 샘플들만 뽑습니다. DeepAR가 분포에서 만들어 낸 수많은 시나리오 값이에요.
- `np.percentile(..., alpha)` 또는 `np.percentile(..., 100-alpha)` — 그 샘플들에서 특정 퍼센타일 (분위수)을 읽습니다. 롱이나 숏이냐에 따라 어느 쪽 꼬리를 볼지 갈립니다(바로 아래 ① 참고).
- `weights[i] * ...` — 그 분위수 손실에 종목 비중을 곱해 "가중 VaR"로 만들고 `V[i]` 에 저장.
- `R = returns.corr()` — 수익률들의 **상관행렬(correlation matrix)** *R*을 계산.
- `return -np.sqrt(V @ R @ V.T)` — 상관을 반영해 종목별 VaR를 합쳐 포트폴리오 VaR를 돌려줍니다.

마지막 줄을 수식으로 옮기면 다음이 최종 공식이에요.

$$\text{VaR}_p = -\sqrt{\mathbf{V}^T \mathbf{R} \mathbf{V}}$$

- VaR_p = 포트폴리오 전체의 VaR.
- **V** = 종목별 **가중 VaR 추정치 벡터**(각 성분이 `weights[i] × 그 종목의 분위수 손실`).
- $\mathbf{V}^T$ = **V**의 전치(transpose), 즉 세로 벡터를 가로로 눕힌 것.
- **R** = 상관행렬. 종목들이 함께 움직이는 정도를 담습니다.
- $\mathbf{V}^T \mathbf{R} \mathbf{V}$ = 이차형식(quadratic form). 상관을 끼워 개별 VaR들을 합산하는 표준적인 위험 합성 방식이에요.
- $\sqrt{\cdot}$ = 제곱근. 분산 스케일을 다시 표준편차 스케일로 되돌립니다.
- 앞의 음수 부호 $-$ = 분위수가 음의 수익률(손실)로 나오므로, VaR를 "양의 손실 크기"로 보고하기 위해 부호를 뒤집어 줍니다.

 **예시** — 두 종목의 가중 VaR가 v_1, v_2 이고 상관인 ρ 면 $\mathbf{V}^T R \mathbf{V} = v_1^2 + v_2^2 + 2\rho v_1 v_2$ 가 돼요. 완전 동조($\rho = 1$)면 $(v_1 + v_2)^2$ 이라 둘이 그대로 더해져 분산효과가 없고, 무상관($\rho = 0$)이면 $\sqrt{v_1^2 + v_2^2}$ 라 단순 합보다 작아져 **분산투자 효과**가 생깁니다. 상관행렬 R 이 바로 이 "함께 무너질 위험"을 조율하는 손잡이예요.

이 함수에서 주목할 두 가지가 있어요.

① **롱/숏 처리**. 가중치가 양수(롱)면 `np.percentile(..., 100-alpha)` 를 씁니다. `alpha=99` 면 $100 - 99 = 1$, 즉 샘플의 아주 낮은 쪽(1퍼센타일)을 봐요. 롱은 큰 손실이 낮은(음의) 수익률에서 오니까 왼쪽 꼬리를 보는 게 맞습니다. 반대로 가중치가 음수(숏)면 그냥 `alpha` (99퍼센타일, 높은 쪽)를 봅니다. 숏은 가격이 크게 오르는 것(높은 수익률)이 곧 손실이기 때문이에요.

 **예시** — `alpha=99` 로 99% VaR를 구한다고 해봐요. 롱(가중치 양수)이면 코드가 `100-99` =1퍼센타일을 뽑아 샘플의 아주 낮은 음의 수익률을 봅니다 — 롱은 값이 떨어질 때 손해니까요. 숏(가중치 음수)이면 그냥 99퍼센타일, 즉 아주 높은 수익률을 봅니다 — 숏은 값이 오를 때 손해거든요.

② **상관관계 처리의 한계(비판)**. `returns.corr()` 는 전체 시계열에 대해 **단 한 번** 계산한 평균 상관이에요. 그런데 논문은 "직전 125일 윈도우 상관"을 지시하니, 코드와 논문이 어긋납니다. 문제는 상관이 시장 격변기에 급변한다는 점이에요. 이런 성질을 **비정상성(non-stationarity)** 이라고 하는데, 전체 평균 하나로는 이 급변을 담아낼 수 없습니다. 특히 VaR는 손실 꼬리에 집중하는 지표라, 위기 때 급등하는 상관(turmoil correlation, 위기 시 상관)에 특별히 주의해야 해요.

 **주의** — 평상시 상관이 0.2였던 두 종목이 위기 때 0.9까지 치솟는 일은 흔합니다. 그런데 `returns.corr()` 가 준 평균값은 여전히 0.2 근처라, 정작 위기 상황의 포트폴리오 VaR를 과소평가하게 돼요. VaR가 하필 그 꼬리 구간을 채는 지표이니, 이 어긋남은 그냥 넘길 수 없는 약점입니다.

참고로, 논문이 이 자리에 상관행렬 대신 **공분산(covariance)** 이라고 쓴 부분은 오타로 보여요. 종목별 분산(변동성 크기)은 이미 $\mathbf{V}$ 에 반영돼 있으니, 여기서는 "함께 움직이는 정도"만 담는 상관행렬 R 을 쓰는 게 더 적절합니다.

과제(학습자용). 이 한계를 직접 손봐 보는 세 가지 연습이에요.

1. 최근 상관을 **공변량(covariate)** 입력으로 DeepAR에 넣도록 코드를 수정할 수 있을까요? (원래 DeepAR는 공변량을 허용합니다. 표기 차이도 알아 두세요 — Salinas는 조건부 분포를 p 로, Fatouros는 우도를 ℓ 로 씁니다.)
2. `var_p` 를 전체 평균 상관 대신 **125일 롤링 상관**으로 재작성할 수 있을까요?
3. 위 변경이 예측과 VaR를 실제로 개선하는지 검증할 방법을, 코드/의사코드/실험과 함께 짧은 에세이로 정리해 보세요.

Lesson 2 정리

레슨 1과 2를 관통하는 한 줄은 이거예요. "**신경망으로 VaR를 더 잘 예측하는 법.**" 값 하나가 아니라 분포 전체를 예측하고(확률예측), 그 분포를 두꺼운 꼬리에 강한 스튜던트 t 기반 DeepAR로 학습해, 마지막에 상관행렬로 종목별 VaR를 합성했습니다. 동시에 "전체 평균 상관"이라는 구현상의 약점까지 짚어, 무엇을 더 손봐야 하는지도 봤고요.

다음 두 레슨에서는 같은 목적(더 나은 VaR 예측)에 이번과 다른 무기인 **트랜스포머(Transformer)** 를 적용해 봅니다. RNN 계열인 DeepAR가 과거를 순서대로 훑었다면, 트랜스포머는 과거 전체를 한눈에 보는 방식이라 무엇이 어떻게

달라지는지 비교하며 이어 가겠습니다.

LESSON 3 — 트랜스포머로 변동성 예측하기

3.1 트랜스포머 소개

이제 우리는 최근 딥러닝 역사에서 **가장 큰 발견 중 하나로** 손꼽히는 **트랜스포머(Transformer)**를 만날 차례예요. 챗봇, 번역기, 그림 생성 모델 같은 요즘 인공지능의 대부분이 이 구조 위에 서 있다고 봐도 됩니다. 우리 과제인 변동성·VaR 추정에서도 이 트랜스포머가 아주 강력한 도구가 되어 줍니다.

이번 레슨은 수식을 파고드는 시간이 아니라, **다음 레슨(LESSON 4)의 코드를 이해하기 위한 "큰 그림"을 그리는 시간**이에요. 그러니 세부 계산에 겁먹지 말고, "트랜스포머가 대체 무슨 일을 하는 녀석인지" 감을 잡는 데 집중하면 됩니다.

 **쉬운 설명** — 앞 모듈에서 다룬 LSTM이 "기억을 한 칸씩 옆으로 넘겨 가며" 시퀀스를 읽었다면, 트랜스포머는 "한눈에 전체를 훑어보고 어디에 주목할지" 정하는 방식이에요. 이 "주목한다"는 아이디어가 바로 다음에 나올 어텐션입니다.

3.2 인코더-디코더 모델

트랜스포머는 사실 **인코더-디코더(encoder-decoder)** 구조의 한 종류예요. 원래 **시퀀스→시퀀스(seq2seq)** 과제 — 예를 들어 "영어 문장을 넣으면 한국어 문장이 나온다" 같은 과제 — 에서는 보통 **RNN(순환신경망)** 계열, 즉 GRU나 LSTM을 사용했습니다. 트랜스포머의 핵심 아이디어는 이 순환 구조를 통째로 **어텐션(attention)**으로 갈아 끼운 것이에요.

인코더(encoder)는 하나의 신경망으로, **특징 추출(feature extraction)**을 담당합니다. 들어온 입력을 **특징공간(feature space)**으로 사영(projection)해서, 각 입력을 하나의 **특징 벡터(feature vector, = 잠재 벡터 latent vector)**로 바꿔 줍니다. 여기서 어텐션의 마법이 발휘되는데, 이 특징 벡터에는 그 입력 하나의 정보뿐 아니라 **다른 모든 입력과의 관계**까지 담기게 돼요. 그리고 이 특징공간이 어떻게 생겼는지는 사람이 정해 주는 게 아니라 **학습으로 찾아냅니다**.

 **쉬운 설명** — "특징공간으로 사영한다"는 말이 어렵죠? 쉽게 말하면, 원본 데이터를 모델이 다루기 좋은 새 좌표계의 숫자 묶음(벡터)으로 번역하는 거예요. 주가 수익률 하나를 넣으면, 그 값이 "어제·그제 값들과 어떤 관계인지"까지 머무린 새 벡터가 튀어나온다고 생각하면 됩니다.

디코더(decoder) 역시 신경망인데, 이쪽은 최종 **출력(예측)**을 만들어 냅니다. 디코더는 인코더와 **같은 개수의 층**을 갖습니다. 다만 각 층의 구성이 조금 달라요. 인코더 층이 가진 두 개의 서브층(뒤에 나올 **MHSA**와 **FFN**)에 더해, 디코더에는 **세 번째 서브층인 "인코더-디코더 어텐션"**이 붙습니다. 이 서브층이 인코더 스택과 디코더 스택을 이어 주는 다리 역할을 하죠. 또한 디코더는 **이전 디코더 층의 출력을 자기 입력으로** 받아서 한 단계씩 예측을 다듬어 갑니다.

여기서 꼭 짚을 점 하나. 트랜스포머에서는 **셀(cell)이 더 이상 순환형(GRU·LSTM)이 아닙니다**. 인코더는 여러 개의 **트랜스포머 층(transformer layer)**을 쌓아 만드는데, 각 층은 두 개의 서브층으로 이루어져요.

- **멀티-헤드 셀프-어텐션(MHSA, multi-head self-attention)** — 입력들끼리 서로를 얼마나 참고할지 정하는 부분.

- **완전연결 순방향 신경망(FFN, feed-forward network)** — 그렇게 모은 정보를 한 번 더 비선형 변환으로 다듬는 부분.

🎯 **핵심** — 그리고 우리 과제에서 정말 중요한 실용 포인트가 있어요. **시계열 예측에는 디코더가 필요 없습니다.** 인코더가 뽑아낸 출력을 "다음 시점의 실제 관측값"과 직접 비교하고, 그 오차를 줄이는 방향으로 학습하면 되거든요. 그래서 우리는 인코더 스택만 쓰면 됩니다.

트랜스포머는 특히 **고차원 데이터셋(변수가 많은 데이터)**에서 성능이 아주 좋다는 장점도 있어요. 여러 종목·여러 지표가 얹힌 금융 데이터에 잘 어울리는 이유죠.

3.3 어텐션(Attention) 직관

어텐션을 처음 이해할 때 가장 잘 통하는 것이 **데이터베이스 비유**예요. 데이터베이스에는 여러 개의 **값(value)**이 저장되어 있고, 각 값은 **키(key)**로 참조합니다. 그리고 우리가 정보를 찾으려고 던지는 질문이 **쿼리(query)**예요. 도서관으로 치면, 찾고 싶은 주제가 쿼리, 책마다 붙은 청구기호가 키, 책 내용 자체가 값입니다.

그런데 여기서 어텐션만의 결정적 특징이 나옵니다. 현실의 질문은 답이 딱 하나로 떨어지지 않아요. **여러 가능한 답이 있고, 각각 "그럴듯함"의 정도가 다릅니다.** 그래서 "이 쿼리는 반드시 이 키/값 하나"라는 식의 깔끔한 선형 매핑이 존재하지 않아요. 대신 어텐션은 **각 쿼리에 대해 여러 키들에 확률을 나눠 부여**합니다. 그리고 이 확률을 어떻게 매길지는 **방대한 학습을 통해 스스로 배웁니다.**

📝 **예시** — "오늘 변동성은?"이라는 쿼리를 던졌다고 해 봐요. 어텐션은 "어제 값에 0.7, 그저께 값에 0.2, 일주일 전 값에 0.1만큼 주목하자"처럼 확률을 나눠 줍니다. 합은 항상 1이 되고요. 어제에 가장 크게 주목했지만 나머지도 조금씩은 반영하는 거예요.

원 논문인 Vaswani et al.의 **"Attention is All You Need"**는 어텐션을 이렇게 정의합니다. **어텐션 함수는 하나의 쿼리와 여러 키-값 쌍을 받아 하나의 출력으로 매핑하는 함수이며, 쿼리·키·값·출력이 모두 벡터다.** 그리고 그 출력은 **값들의 가중합(weighted sum)**인데, 각 값에 붙는 가중치는 **쿼리와 그 값의 키 사이의 호환성(compatibility, 즉 유사도)** 함수로 계산됩니다.

🎯 **핵심** — 한 문장으로 정리하면 이래요. 쿼리는 여러 키와 **유사도(= 확률분포)**로 연결되고, 각 키는 하나의 값에 **결정론적으로** 대응하며, 그 확률로 가중된 값들의 조합이 곧 **출력(예측)**이 됩니다. "얼마나 주목할지(확률)"는 배우고, "무엇을 가져올지(값)"는 고정 — 이 둘의 곱이 어텐션입니다.

LESSON 4 — 멀티-트랜스포머 실전

4.1 스케일드 닷-프로덕트 어텐션(Scaled Dot-Product Attention)

이제 앞에서 그린 큰 그림을 실제 수식과 코드로 확인해 볼게요. 트랜스포머가 쓰는 어텐션의 정식 이름은 **스케일드 닷-프로덕트 어텐션(Scaled Dot-Product Attention)**이고, 딱 이 한 줄로 표현됩니다.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

기호를 하나하나 풀어 볼게요.

- Q — **쿼리(query) 행렬**. "무엇을 찾고 싶은지"를 담은 벡터들을 행으로 쌓은 것.
- K — **키(key) 행렬**. 각 값에 붙은 "색인표"에 해당하는 벡터들.
- V — **값(value) 행렬**. 실제로 가져올 내용물 벡터들.
- K^T — 키 행렬의 **전치(transpose)**. Q 와 곱하기 위해 행과 열을 뒤집은 것.
- d_k — **키의 차원(dimension)**, 즉 키 벡터 하나에 들어 있는 특징의 개수.
- $\sqrt{d_k}$ — 그 차원의 제곱근. 뒤에서 설명할 **스케일링(scaling)**에 쓰입니다.
- softmax — 여러 점수를 **합이 1인 확률**로 바꿔 주는 함수.

실제 구현은 다음과 같아요.

```
def attention(self, query, key, value):
    score = tf.matmul(query, key, transpose_b=True)      # Q · KT
    dim_key = tf.cast(tf.shape(key)[-1], tf.float32)    # d_k
    scaled_score = score / tf.math.sqrt(dim_key)         # ÷ √d_k
    weights = tf.nn.softmax(scaled_score, axis=-1)      # softmax → 확률
    output = tf.matmul(weights, value)                  # 확률 · V
    return output, weights
```

단계별로 따라가 볼게요.

1. **점수 계산 QK^T** — 쿼리와 키의 **내적(dot product)**입니다. 두 벡터가 얼마나 닮았는지를 재는 척도, 즉 앞에서 말한 **호환성(유사도)**이에요. 방향이 비슷할수록 값이 커집니다.
2. **스케일링 $\div \sqrt{d_k}$** — d_k 가 크면 내적 값이 너무 커집니다. 그러면 softmax가 한쪽으로 극단적으로 쏠려서(거의 1 하나, 나머지 전부 0) **기울기가 사라지고(gradient vanishing)** 학습이 불안정해져요. 그래서 키 차원의 제곱근으로 나눠 크기를 정규화합니다.
3. **softmax** — 스케일된 점수들을 합이 1인 가중치(확률)로 바꿉니다.
4. **출력** — 그 확률을 V 에 곱하면($\text{weights} \times V$), 결국 **값들의 가중합**이 나옵니다. 이것이 어텐션의 최종 출력이에요.

 **예시** — 왜 $\sqrt{d_k}$ 로 나누는지 숫자로 볼게요. $d_k = 64$ 면 $\sqrt{d_k} = 8$ 입니다. 만약 스케일 전 점수가 $[40, 8, 0]$ 이라면 softmax가 거의 $[1, 0, 0]$ 처럼 한쪽에 몰려요. 하지만 8로 나눈 $[5, 1, 0]$ 은 훨씬 부드러운 분포가 되어 학습이 안정됩니다.

⚠ 주의 — "닷 프로젝트(내적)라면서 코드에선 왜 **행렬 곱**을 쓰나요?"라는 의문이 들 수 있어요. **3차원 행렬**이 들어오면 내적이 **배치(batch)** 단위로 한 번에 계산되기 때문입니다. 배치 개수는 넘어온 행렬들의 **첫 번째 차원 크기 (batch_size)**로 정해지고, $Q \cdot K \cdot V$ 세 행렬 모두 이 배치 크기가 동일합니다.

실제로 인코더에 입력이 들어오면, 하나의 입력에서 쿼리·키·값을 각각 서로 다른 밀집층(dense layer)으로 만들어 냅니다. 세 텐서의 모양이 모두 같다는 점을 코드로 확인해 보세요.

```
query = self.query_dense(inputs) # (batch_size, seq_len, embed_dim)
key   = self.key_dense(inputs)   # (batch_size, seq_len, embed_dim)
value = self.value_dense(inputs) # (batch_size, seq_len, embed_dim)
```

여기서 `seq_len` 은 시퀀스 길이(몇 시점을 보는지), `embed_dim` 은 각 시점을 표현하는 임베딩 차원이에요. 첫 차원 `batch_size` 가 바로 위에서 말한 "배치 개수"입니다.

4.2 멀티-헤드 어텐션(Multi-Head Attention)

실제 모델은 어텐션을 딱 한 번만 하지 않고 **여러 번 병렬로** 수행합니다. 이것이 **멀티-헤드 어텐션(Multi-Head Attention)**이에요. 우리 구현에서는 $h = 3$ 개의 어텐션(=헤드)을 계산해 **이어붙입니다(concatenate)**.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

기호를 풀어 볼게요.

- head_i — i 번째 **헤드(head)**. 각 헤드는 하나의 독립적인 어텐션 계산이에요.
- W_i^Q, W_i^K, W_i^V — i 번째 헤드가 **자기만의 방식으로** $Q \cdot K \cdot V$ 를 사영할 때 쓰는 학습 가중치들. 헤드마다 값이 다릅니다.
- $\text{Concat}(\cdot)$ — 여러 헤드의 결과를 옆으로 **이어붙이는** 연산.
- W^O — 이어붙인 결과를 한 번 더 사영해 최종 모양으로 맞추는 **출력 가중치(output weight)**.

즉, 각 헤드 i 는 $Q \cdot K \cdot V$ 를 각자 다른 학습 가중치(W_i^Q, W_i^K, W_i^V)로 사영한 뒤 어텐션을 계산하고, 여러 헤드의 결과를 이어붙인 다음 W^O 로 한 번 더 사영해 마무리합니다. 코드로는 이렇게 이어붙이고 합치죠.

```
concat_attention = tf.reshape(attention, (batch_size, -1, self.embed_dim))
output = self.combine_heads(concat_attention) # (batch_size, seq_len, embed_dim)
```

💡 쉬운 설명 — 그럼 왜 굳이 h 번이나 반복할까요? 특징공간을 학습하는 과정(가중치 최적화)에는 **무작위성**이 있어요. 그래서 여러 번 사영하면 매번 조금씩 **다른 관점의 특징**을 뽑게 됩니다. 여러 각도로 봐 두면 그중 의미 있는 특징을 잡아낼 확률이 올라가죠. 한마디로 "여러 번 시도해서 좋은 것을 얻자"는 전략입니다.

📝 예시 — 한 헤드는 "바로 어제와의 관계"에 주목하고, 다른 헤드는 "일주일 주기 패턴"에, 또 다른 헤드는 "급락 직후 반등"에 주목할 수 있어요. 서로 다른 시각 셋을 합치니, 하나만 볼 때보다 훨씬 풍부한 표현이 됩니다.

4.3 멀티-트랜스포머(Multi-Transformer)

Ramos-Perez 등의 연구는 방금 그 "여러 번 시도하기" 아이디어를 한 단계 더 크게 밀어붙였어요. 논리는 이렇습니다. 멀티-헤드가 단일 어텐션보다 낫다면, 여러 트랜스포머를 결합한 멀티-트랜스포머도 단일 트랜스포머보다 낫지 않겠느냐는 거죠. 실제로 실험에서 이 멀티-트랜스포머(Multi-Transformer)가 더 정확했습니다.

그런데 문제가 하나 있어요. 우리가 최종적으로 필요한 건 보통 VaR 예측값 하나입니다. 그렇다면 여러 트랜스포머의 출력을 어떻게 하나로 모을까요? 답은 배깅(bagging, bootstrap aggregating)입니다.

- 다섯 개의 트랜스포머를 준비합니다.
- 각 트랜스포머는 가용 관측치의 90%를 무작위로 표본 추출해서 학습합니다(그래서 서로 조금씩 다른 데이터로 배웁니다).
- 다섯 트랜스포머의 출력을 평균 내어 최종 예측으로 삼습니다.

 **예시** — 다섯 트랜스포머가 각각 VaR을 -3.2%, -3.5%, -3.1%, -3.4%, -3.3%로 예측했다면, 최종 예측은 이들의 평균인 -3.3%가 됩니다. 어느 하나가 튀어도 평균이 완충해 주죠.

 **핵심** — 배깅의 이점은 명확해요. 서로 조금씩 다른 여러 모델의 평균을 쓰면 과적합(overfitting)에 덜 취약하고 예측의 분산(variance)이 작아집니다. 이것이 앙상블 학습(ensemble learning)의 핵심 원리 — "제각각인 여러 모델의 평균이 개별 모델보다 안정적이다" — 입니다.

Lesson 4 & 모듈 정리

이번 모듈에서 우리는 정교한 신경망으로 VaR 같은 위험지표를 더 잘 추정하는 법을 배웠어요. 앞 모듈의 LSTM 기반 DeepVaR, 그리고 이번 레슨의 트랜스포머 / 멀티-트랜스포머가 그 주인공이었죠. 순환 구조(LSTM)든 어텐션 구조(트랜스포머)든, 목표는 같습니다. 데이터에서 위험의 패턴을 잘 뽑아내 더 정확한 VaR을 내놓는 것.

 **핵심** — 다음 모듈 M4에서는 접근을 완전히 바꿔, 극단값 이론(Extreme Value Theory)으로 분포의 꼬리(tail)와 극단 손실을 직접 다룹니다. 신경망이 "패턴 전반"을 봤다면, 극단값 이론은 "드물지만 치명적인 사건"에 현미경을 들이대는 방법이에요.

전체 한눈에 정리 (복습용 핵심 요약)

지금까지 온 길을 표와 콜아웃으로 한눈에 정리해 볼게요. 시험 직전에 이 부분만 훑어도 흐름이 잡히도록 구성했습니다.

① 개념 계보 — RNN에서 트랜스포머까지

세대	모델	한 줄 특징	내부 구성
1세대	RNN(순환신경망)	순환·기억	은닉상태를 다음 시점으로 넘김
2세대	GRU	게이트 2개	리셋 게이트, 업데이트 게이트
2세대	LSTM	4구성	입력·망각·출력 게이트 + 메모리셀
3세대	트랜스포머	어텐션	순환 구조를 셀프-어텐션으로 대체

💡 **쉬운 설명** — 흐름만 기억하세요. RNN → GRU → LSTM은 "기억을 옆으로 넘기는" 순환 계열의 발전사예요. LSTM은 앞 모듈 **DeepVaR**의 부품이었고, 트랜스포머는 그 순환 구조를 어텐션으로 갈아 끼운 **신세대**입니다.

② DeepVaR 파이프라인

단계	무엇을 하나	도구 / 수식
1	분포 파라미터 학습	GluonTS DeepAR + 스튜던트 t 분포, 로그우도 최대화(Adam)
2	예측 분포 생성	몬테카를로 샘플링
3	종목별 VaR	예측 분포의 분위수(quantile) → 벡터 $\mathbf{V}$
4	포트폴리오 VaR	상관행렬 R 로 결합: $-\sqrt{\mathbf{V}^\top R \mathbf{V}}$

포트폴리오 VaR 식의 기호를 풀면, $\mathbf{V}$ 는 종목별 VaR을 모은 벡터, $\mathbf{V}^\top$ 는 그 전치, R 은 종목 간 **상관행렬(correlation matrix)**이에요. 상관관계를 반영해 전체 위험을 합산한 뒤 제곱근을 씌우고, 손실을 음수 부호로 표현합니다.

⚠️ **주의** — 종목별 VaR을 그냥 다 더하면 안 됩니다. 종목들이 완벽히 같이 움직이지 않으므로(분산 효과), 반드시 상관행렬 R 을 끼운 $-\sqrt{\mathbf{V}^\top R \mathbf{V}}$ 로 결합해야 실제 위험에 가까워져요.

③ 백테스팅(Backtesting)

먼저 예측이 맞았는지 틀렸는지를 표시하는 **히트(hit)** 지표를 정의합니다.

$$I_t = 1[\text{actual} < \text{forecast}]$$

여기서 I_t 는 t 시점의 히트, $1[\cdot]$ 는 조건이 참이면 1·거짓이면 0인 **지시함수(indicator function)**, actual은 실제 수익률, forecast는 예측한 VaR 경계예요. 실제 손실이 예측 경계를 넘어서면(위반하면) 1이 됩니다.

구분	항목	요점
위반율	violation rate	$\approx 1 - \text{신뢰수준}$ 이어야 정상
손실함수	5종	대표적으로 이차·부드러운·틱·펌
유효성 검정	Christoffersen 우도비	무조건부 커버리지 + 독립성
유효성 검정	Engle-Manganelli 동적분위수	회귀 + 카이제곱 독립성

 **예시** — 99% 신뢰수준 VaR이라면 이론상 위반율은 $1 - 0.99 = 1\%$ 근처여야 해요. 100일 중 대략 1번 위반이 정상. 만약 8번씩 위반한다면 그 VaR 모델은 위험을 너무 낙관한 것이고, 0번이면 지나치게 보수적인 셈입니다.

 **쉬운 설명** — 두 유효성 검정을 한 줄로 구분하면, **Christoffersen**은 "위반이 적정 빈도인가(무조건부 커버리지) + 위반이 몰려 터지지 않고 독립적인가"를 우도비로 보고, **Engle-Manganelli**는 같은 독립성 문제를 회귀와 카이제곱으로 검정합니다.

④ 어텐션 한 줄 요약

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- 쿼리·키의 **내적** QK^T 으로 유사도를 재고,
- $\sqrt{d_k}$ 로 **스케일**해 softmax 쓸림을 막고,
- **softmax**로 합이 1인 확률로 바꾼 뒤,
- 그 확률로 V 의 **가중합**을 낸다.

 **핵심** — 그리고 이 어텐션을 h 개 **헤드**로 병렬로 돌리면 **멀티-헤드**, 그렇게 만든 **트랜스포머 5개를 배경으로** 앙상블하면 **멀티-트랜스포머**입니다. "여러 번 시도해 좋은 것을 평균 낸다"는 하나의 철학이 헤드 단위에서 모델 단위까지 그대로 반복되는 셈이에요.

논문 ① 완전 해설 — DeepVaR (Fatouros et al., Digital Finance, 2023)

이 논문의 부제는 "확률적 심층신경망을 활용한 포트폴리오 위험평가 프레임워크" 입니다. 한 문장으로 요약하면, "값 하나만 콕 찍어 예측하던 전통적 위험지표를, 미래 수익률의 분포 전체를 예측하는 딥러닝으로 바뀌서, 위기 때에도 잘 맞는 위험지표를 만들자"는 이야기예요. 이번 절에서는 배경 문제의식부터 수식 하나하나, 그리고 실험 결과까지 차근차근 풀어가겠습니다.

초록·문제의식

금융회사가 "우리가 하루에 잃을 수 있는 최대 손실이 얼마인가?"를 물을 때 가장 널리 쓰는 답이 바로 **위험가치(Value at Risk, VaR)** 입니다. 예를 들어 "99% VaR가 -3%"라면, "100일 중 99일은 손실이 3%를 넘지 않는다(즉 100일 중 1일 정도만 3%보다 더 크게 잃는다)"는 뜻이에요. 은행 자본규제(바젤)에서도 핵심 지표로 쓰일 만큼 표준적인 도구입니다.

문제는 이 VaR가 **평온한 시기에는 잘 맞지만, 정작 중요한 격변기에는 형편없이 빗나간다**는 점이에요. 2008년 글로벌 금융위기, 2020년 코로나 폭락처럼 시장이 요동칠 때 VaR는 손실을 크게 과소평가했고, 이는 곧 잘못된 위험 결정으로 이어졌습니다. 이 논문은 유명한 비유를 인용해요.

쉬운 설명 — Einhorn & Brown은 VaR를 "평소엔 잘 작동하지만 정작 사고가 나면 안 터지는 에어백"에 비유했습니다. 필요 없을 때는 멀쩡히 작동하다가, 정작 목숨이 걸린 충돌 순간에 안 터지는 에어백만큼 위험한 장치도 없겠죠. VaR도 위기 때 안 맞으면 오히려 헛된 안전감만 주는 셈입니다.

논문이 내놓는 해법은 **확률적 딥러닝(probabilistic deep learning)** 입니다. 특히 외환(FX) 포트폴리오의 99% VaR에서 매우 유망한 결과를 보였다고 초록에서 밝혀요. 핵심 키워드 세 가지를 미리 정리하면 아래와 같습니다.

키워드	의미	왜 중요한가
확률적 (probabilistic)	값 하나가 아니라 분포 를 예측	VaR 자체가 분포의 한 지점(백분위수)이므로 분포를 알면 VaR를 정확히 뽑을 수 있음
딥러닝(deep learning)	LSTM 기반 DeepAR 모델 사용	비선형 패턴·희귀 사건까지 학습해 위기 대응력 확보
99% VaR / FX	신뢰수준 99%, 외환 4쌍 대상	가장 까다로운 극단 위험 구간에서 검증

기존 VaR의 세 부류와 3대 약점

기존의 VaR 계산법은 크게 세 부류로 나뉩니다. 하나씩 살펴볼게요.

부류	핵심 아이디어	대표 기법	장점	한계
비모수(non-parametric)	분포를 가정하지 않고 과거 데이터의 분위수 를 그대로 사용	역사적 시뮬레이션 (Historical Simulation, HS)	계산복잡도가 낮고 단순	과거 관측창(window)에 없던 새로운 변동은 못 잡음
모수(parametric)	수익률이 특정 분포(정규 또는 스튜던트 t)를 따른다고 가정	분산-공분산(Variance-Covariance, VC), GARCH	수식이 단순하고 빠름	실제 데이터가 그 가정을 잘 안 따름
준모수(semi-parametric)	위 두 방식을 혼합	몬테카를로(Monte Carlo, MC)	유연함	여전히 분포·시뮬레이션 설정에 의존

가장 고전적인 **분산-공분산 VaR**(식 1)는 다음과 같습니다.

$$\text{VaR}^\alpha = z_{1-\alpha} \sqrt{W^\top \Sigma W}$$

기호를 하나하나 풀어볼게요.

- $z_{1-\alpha}$: **z-점수(z-score)**, 즉 표준정규분포의 분위수입니다. 신뢰수준 α 에 대응하는 값이에요. 예를 들어 99% VaR면 $z \approx 2.33$ 입니다.
- W : **포트폴리오 가중치(weights)** 벡터. 각 자산에 돈을 얼마씩 배분했는지를 나타냅니다.
- Σ : 자산 수익률들의 **공분산행렬(covariance matrix)**. 각 자산의 변동성과 자산 간 상관관계를 담고 있어요.
- $W^\top$: W 의 전치(transpose). 행벡터·열벡터를 맞춰 곱하기 위한 표기입니다.

 **쉬운 설명** — $\sqrt{W^\top \Sigma W}$ 부분은 그냥 **포트폴리오 전체의 표준편차(변동성)**입니다. 여러 자산을 섞었을 때 전체가 얼마나 출렁이는지를 하나의 숫자로 압축한 거예요. 거기에 신뢰수준에 해당하는 z-점수 $z_{1-\alpha}$ 를 곱하면, "이 정도 변동성이면 극단적으로는 얼마까지 잃을 수 있다"는 VaR가 나옵니다.

 **예시** — 포트폴리오 하루 표준편차가 2%이고 99% 신뢰수준($z \approx 2.33$)이라면, $\text{VaR} \approx 2.33 \times 2\% = 4.66\%$ 입니다. "웬만하면 하루 손실이 4.66%를 안 넘는다"는 뜻이죠.

이런 전통적 VaR에는 **3대 약점**이 있습니다.

1. **위기 때 위반이 특정 시기에 뭉침(clustered)** — 위기 국면에서 실제 손실이 VaR를 넘어서는 "위반(violation)"이 여기저기 흩어지지 않고 한 시기에 몰려서 터집니다. 특히 99% 같은 높은 신뢰수준에서 이 문제가 심각해요.
2. **두꺼운 꼬리와 레버리지 효과 미고려** — 실제 수익률 분포는 정규분포보다 극단값이 훨씬 자주 나오는 **두꺼운 꼬리(fat tails)**를 가지고, 하락 시 변동성이 커지는 **레버리지 효과(leverage effect)**도 있습니다. 그래서 초과손실이 실제로 크게 나는데 전통 모델은 이를 거의 반영하지 못합니다.
3. **단일 자산 편중** — 대부분의 기존 연구가 S&P 지수 같은 **단일 자산** 하나만 다룹니다. 여러 자산을 섞은 실제 포트폴리오 수준의 위험 평가는 연구가 부족했어요.

DeepVaR의 3가지 핵심 혁신

DeepVaR는 위 약점들을 정면으로 겨냥합니다. 혁신 포인트가 세 가지예요.

혁신	무엇을 하는가	어떤 약점을 해결하나
① 연속 학습(continuous learning)	새 시장 데이터가 들어올 때마다 모델을 재학습	뒤틀린 위반과 모델 편향/드리프트 방지
② DeepAR 기반 확률예측	자기회귀 순환신경망으로 미래 분포 예측	짧은 학습으로도 희귀 사건 포착
③ 포트폴리오 수준 what-if 분석	재학습 없이 사전거래 시나리오 분석	단일 자산 한계 극복, 실무 적용성

첫 번째, **연속 학습**은 새로운 데이터가 도착할 때마다 모델을 다시 훈련시켜서, 모델이 과거에 갇혀 편향되거나 시간이 지나며 성능이 뒤쳐지는 **드리프트(drift)**를 막습니다. 이게 가능한 이유는 학습이 매우 빠르기 때문이에요.

🔗 **핵심** — DeepVaR의 한 번 학습 시간은 **13초 미만**입니다. 그래서 하루가 끝나길 기다릴 필요 없이, 장중(intra-day)에도 VaR를 새로 계산할 수 있어요. 시장이 급변하는 순간 바로바로 위험을 다시 재는 것이 실전에서 결정적입니다.

두 번째, **자기회귀 순환신경망(DeepAR)** 기반 확률예측은 값 하나가 아니라 분포를 뽑아내기 때문에, 학습 데이터가 짧아도 극단적인 희귀 사건까지 확률적으로 포착할 수 있습니다. 세 번째, 학습된 개별 자산 예측을 조합하면 **재학습 없이** 포트폴리오 구성을 이리저리 바꿔보는 **사전거래/what-if 분석**을 거의 실시간으로 할 수 있습니다.

VaR의 수학적 정의

이제 VaR를 수식으로 엄밀히 정의해 봅시다. 먼저 수익률을 어떻게 재는지부터요. 이 논문은 **로그수익률(log-return)**을 손익(PnL)으로 사용합니다.

$$\text{PnL}_t = r_t = \log \frac{P_t}{P_{t-1}}$$

- P_t : t 일의 종가(closing price)
- P_{t-1} : 전날($t - 1$ 일)의 종가
- $\log$: 자연로그. 종가 비율에 로그를 씌우면 로그수익률이 됩니다.

💡 **쉬운 설명** — $\frac{P_t}{P_{t-1}}$ 는 "어제 대비 오늘 가격이 몇 배가 됐다"이고, 여기에 로그를 씌운 것이 로그수익률입니다. 변화가 작을 때는 단순 수익률(예: +1%)과 거의 같지만, 로그수익률은 여러 날 수익을 **더하기만 하면 누적**이 되는 편리한 성질이 있어 금융에서 즐겨 씁니다.

그럼 1일 VaR의 정식 정의(식 2)는 다음과 같습니다.

$$P(r_t \leq \text{VaR}_t^\alpha) = 1 - \alpha$$

이 식은 "수익률 r_t 가 VaR_t^α 이하로 떨어질 확률이 정확히 $1 - \alpha$ "라는 뜻입니다. 여기서 α 는 신뢰수준(예: 0.99)이고, $1 - \alpha$ 는 그 반대편 꼬리 확률(예: 0.01)이에요.

이제 q_α 를 수익률 분포의 α -분위수(quantile)라고 하면, VaR는 곧 그 분위수 자체입니다(식 3).

$$\text{VaR}^\alpha = q_{1-\alpha}$$

🎯 **핵심** — 결국 **VaR = 수익률 분포의 특정 백분위수**입니다. 99% VaR는 수익률 분포에서 아래쪽 1% 지점의 값이에요. 그러니 VaR를 잘 맞히려면 그 지점, 나아가 **분포 전체를 잘 예측**하는 것이 필요합니다. 바로 이 지점에서 "값 하나"가 아니라 "분포"를 뽑는 확률예측이 등장하는 겁니다.

DeepAR의 조건부 분포와 학습

DeepVaR의 심장은 **DeepAR**라는 확률적 시계열 예측 모델입니다. DeepAR가 실제로 학습하는 대상은 하나의 값이 아니라 **조건부 분포(conditional distribution)** 예요.

$$P(z_{i, t_0:T} \mid z_{i, 1:t_0-1})$$

기호를 풀어볼게요.

- $z_{i, 1:t_0-1}$: 종목 i 의 과거 관측값들(1일부터 $t_0 - 1$ 일까지)
- $z_{i, t_0:T}$: 종목 i 의 미래 값들(t_0 일부터 T 일까지)
- $\mid$: "주어졌을 때(조건부)"를 뜻하는 기호

즉 "종목 i 의 과거가 이랬을 때, 미래는 어떤 분포를 그릴까?"를 학습하는 거예요.

모델이 표현하는 분포 $Q_\theta(\theta$ 는 신경망 파라미터)는 **우도 인자들의 곱(product of likelihood factors)** 형태로 짜여 있습니다. 각 시점의 예측 확률들을 꼭 곱한 것이죠. 학습이란 이 곱(전체 우도)을 **최대화**하는 θ 를 찾는 과정입니다.

💡 **쉬운 설명** — "우도를 최대화한다"는 건, **실제로 관측된 과거 데이터가 나올 확률이 가장 높아지도록** 신경망을 조율한다는 뜻입니다. 여러 날의 확률을 모두 곱한 값이 커지도록 학습하면, 모델이 데이터의 실제 움직임을 가장 잘 설명하게 됩니다.

학습이 끝나면, 그 분포에서 **몬테카를로 샘플(Monte Carlo samples)**을 여러 개 뽑아 미래 수익률의 시나리오들을 만들고, 그 시나리오들의 분포에서 VaR 분위수를 읽어냅니다. 또한 DeepAR는 비슷한 여러 시계열을 **동시에** 학습하는데, 이를 **교차학습(cross-learning)**이라 합니다. 한 종목의 변화가 다른 종목의 예측 분포에도 영향을 주므로, 자산 간 연관성까지 자연스럽게 배웁니다.

논문이 사용한 실제 하이퍼파라미터도 짚어둘게요. 신경망은 초기 가중치를 랜덤하게 시작하는데, 이 노이즈 때문에 결과가 시드(seed)에 따라 크게 흔들립니다. 그래서 서로 다른 시드로 학습한 **10개 모델을 배깅(Bagging, Bootstrap Aggregating) 앙상블**로 평균 내어 안정성을 확보합니다.

하이퍼파라미터	설정값	한 줄 설명
앙상블	10개 모델 배깅	서로 다른 시드로 학습 후 결합
훈련셋 크기	900개	학습에 쓰는 과거 관측 수
에폭(epoch)	5	전체 데이터를 5번 반복 학습
예측 길이	1일	하루 앞 VaR
컨텍스트 길이	15일	예측 시 참고하는 최근 구간
옵티마이저	ADAM	널리 쓰는 적응형 경사하강
학습률	0.0001	한 걸음의 크기
LSTM 구조	2층 · 각 50셀	순환신경망 깊이·폭
드롭아웃	10%	과적합 방지

 **예시** — GluonTS 라이브러리의 기본 셀 수(num_cells)는 40인데, 이 논문은 이를 **50으로 상향**했습니다. 셀 수가 많을수록 표현력이 커지지만 과적합 위험도 커지므로, 드롭아웃 10%로 균형을 맞춘 셈이에요.

백테스팅 수식(논문 정식)

만든 VaR가 실제로 잘 맞았는지 과거 데이터로 검증하는 것을 **백테스팅(backtesting)** 이라 합니다. 핵심은 "손실이 VaR를 넘어선 날(위반)이 얼마나, 어떻게 났는가"예요. 먼저 **히트 지표(hit indicator)**(식 7)를 정의합니다.

$$I_t = \begin{cases} 1, & \text{if } \text{VaR}_t > \text{PnL}_t \\ 0, & \text{otherwise} \end{cases}$$

- I_t : t 일의 위반 여부. 손실이 VaR를 초과하면 1, 아니면 0.
- 여기서 VaR와 PnL은 손실 쪽이 음수인 값입니다. $\text{VaR}_t > \text{PnL}_t$ 는 "실제 손익이 VaR 한계선보다 더 아래로 떨어졌다"는 뜻이에요.

이론적으로 기대되는 위반 횟수(식 8)는 다음과 같습니다.

$$E[v] = (1 - \alpha) N_{\text{days}}$$

 **예시** — 99% VaR($1 - \alpha = 0.01$)로 250일을 관측하면, 기대 위반은 $0.01 \times 250 = 2.5$ 회입니다. 모델이 제대로라면 250일 중 대략 2~3번 정도만 VaR를 넘어야 정상이에요.

실제 위반 수(식 9)와 위반율(식 10)은 이렇게 셉니다.

$$N_{\text{violations}} = \sum_{t=1}^T I_t$$

$$r_{\text{Violations}} = \frac{N_{\text{violations}}}{N_{\text{days}}}$$

- $N_{\text{violations}}$: 전체 기간의 히트($I_t = 1$)를 모두 더한 총 위반 횟수
- $r_{\text{Violations}}$: 그 위반 수를 전체 관측일수 N_{days} 로 나눈 비율

🎯 **핵심** — 좋은 모델은 위반율이 명목 임계값 이하, 즉 $r_{\text{Violations}} \leq 1 - \alpha$ 여야 합니다. 99% VaR라면 위반율이 0.01(1%)을 넘지 않아야 한다는 뜻이에요. 위반이 너무 많으면 위험을 과소평가한 것이고, 이게 실전에서 가장 위험한 실패입니다.

위반 횟수만으로는 "얼마나 크게 틀렸나"를 못 잹니다. 그래서 여러 **손실함수(loss function)** 로 오차의 크기까지 평가해요. 하나씩 봅시다.

이차 손실(quadratic loss)(식 11):

$$l_{QL} = \sum_{t=1}^N I_t (1 + (\text{PnL}_t - \text{VaR}_t)^2)$$

위반이 난 날($I_t = 1$)만 골라, 실제 손익과 VaR의 차이를 제곱해 벌점을 줍니다. 크게 빗나갈수록 제곱만큼 커지는 벌점이에요.

부드러운 손실(smooth loss)(식 12), $d = 25$, $m = \text{PnL}_t - \text{VaR}_t^\alpha$:

$$l_Q = \frac{1}{N} \sum_{t=1}^N (\alpha - (1 + e^{dm})^{-1}) m$$

- m : 실제 손익에서 VaR를 뺀 차이(초과분)
- $(1 + e^{dm})^{-1}$: **시그모이드(sigmoid)** 형태의 부드러운 스위치. 계단 함수인 히트 지표 I_t 를 미분 가능한 매끈한 곡선으로 근사한 것입니다. $d = 25$ 는 그 스위치가 얼마나 급격히 꺾이는지(가파를수록 계단에 가까움)를 정합니다.

💡 **쉬운 설명** — 히트 지표 I_t 는 0 아니면 1로 툭 끊기는 계단이라 미분이 안 돼서 최적화에 불편합니다. 부드러운 손실은 그 계단을 매끈한 S자 곡선으로 바꿔서, 경사하강 같은 최적화 기법이 잘 작동하도록 만든 버전이에요.

틱 손실(tick loss)(식 13):

$$l_T = \sum_{t=1}^N (\alpha - I_t) (\text{PnL}_t - \text{VaR}_t^\alpha)$$

분위수 회귀에서 쓰는 고전적 손실입니다. 위반이 났을 때와 안 났을 때 서로 다른 가중치(α 또는 $\alpha - 1$)를 주어, VaR가 목표 분위수에 정확히 놓이도록 유도합니다.

펌 손실(firm loss)(식 14):

$$l_F = \begin{cases} (\text{PnL}_t - \text{VaR}_t)^2, & \text{if } \text{PnL}_t < \text{VaR}_t \\ -a \text{VaR}_t, & \text{otherwise} \end{cases}$$

- 위반이 난 경우(윗줄): 초과분의 제곱만큼 벌점 → "위험을 놓친 대가"

- 위반이 없는 경우(아랫줄): $-a \text{VaR}_t$ 만큼 비용 $\rightarrow$ "너무 보수적으로 자본을 묶어둔 기회비용". 여기서 a 는 자본의 기회비용으로 $a = 1$ 로 둡니다.

💡 쉬운 설명 — 펌 손실은 회사(firm) 입장을 흉내 낸 손실입니다. VaR를 너무 작게 잡으면 손실을 놓쳐서 벌점, 너무 크게 잡으면 쓸데없이 자본을 많이 쌓아두는 기회비용으로 벌점. 양쪽을 다 벌하니 "딱 알맞은" VaR를 찾게 됩니다.

손실함수	식	무엇을 벌하나
이차 손실 l_{QL}	식 11	위반 시 초과분의 제곱
부드러운 손실 l_Q	식 12	미분 가능한 분위수 오차
틱 손실 l_T	식 13	분위수 정렬 오차(비대칭)
펌 손실 l_F	식 14	위반 손실 + 과보수 기회비용

마지막으로, 위반이 **통계적으로 타당한지**를 검정하는 두 가지 유효성 검정이 있습니다.

- Christoffersen 조건부 커버리지(conditional coverage) 검정**: 아래처럼 두 조각으로 나뉘며, 점근적으로 χ^2 분포를 따릅니다.

$$\text{LR}_{cc} = \text{LR}_{uc} + \text{LR}_{ind}$$

여기서 LR_{uc} 는 **무조건부 커버리지(unconditional coverage)**, 즉 "위반 횟수가 기대치와 맞는가"를 봅니다. LR_{ind} 는 **독립성(independence)**, 즉 "위반이 서로 뭉치지 않고 독립적으로 흩어져 있는가"를 봅니다. LR_{ind} 는 위반이 1차 마코프(Markov) 종속을 가진다는 대립가설 대비, 직렬 독립이라는 귀무가설의 우도비(likelihood ratio)예요.

- Engle-Manganelli 동적 분위수(Dynamic Quantile, DQ) 검정**: 히트 지표 I_t 가 계산 시점에서 알 수 있는 정보집합 Ω_{t-1} 의 그 어떤 변수와도 상관관계가 없는지(무상관인지)를 봅니다.

🎯 핵심 — 이 두 검정의 공통 관심사는 "**위반이 뭉쳐 있으면 안 된다**"입니다. 위반 횟수가 맞아도(무조건부 통과) 특정 시기에 몰려 터지면(독립성 실패) 그 모델은 위기를 못 잡는 것이므로 탈락입니다. DeepVaR의 강점이 바로 이 뭉침을 피하는 데 있어요.

포트폴리오 VaR(식 15)

지금까지는 개별 자산 VaR였습니다. 여러 자산을 섞은 **포트폴리오 VaR**(식 15)는 이렇게 합칩니다.

$$\text{VaR}_p^\alpha = \sqrt{V R V^\top}$$

- V : 각 자산의 (가중치 $\times$ VaR)를 늘어놓은 벡터입니다.

$$V = [w_1 \text{VaR}_1^\alpha, w_2 \text{VaR}_2^\alpha, w_3 \text{VaR}_3^\alpha, w_4 \text{VaR}_4^\alpha]$$

- w_i : 자산 i 의 포트폴리오 가중치
- VaR_i^α : 자산 i 의 개별 VaR
- R : 자산 간 **상관행렬(correlation matrix)**. 이 논문에서는 **직전 125일** 수익률로 계산한 4×4 행렬입니다.

방향에 따라 어떤 VaR를 쓰는지도 규칙이 있습니다.

- 롱(long, $w_i > 0$) 포지션이면 VaR_i^α (아래쪽 꼬리 위험)
- 숏(short, $w_i < 0$) 포지션이면 $\text{VaR}_i^{1-\alpha}$ (반대편 꼬리 위험)

 **쉬운 설명** — 상관행렬 R 을 끼워 넣는 이유는 **분산투자 효과** 때문입니다. 자산들이 서로 반대로 움직이면(상관이 낮으면) 전체 위험이 줄어들고, 같이 움직이면(상관이 높으면) 위험이 커집니다. $\sqrt{V^T R V}$ 는 이 상관관계를 반영해 개별 위험들을 합친 포트폴리오 총 위험이에요.

 **주의** — 논문은 상관행렬 R 을 **직전 125일 롤링(rolling)**으로 계산하라고 지시합니다. 그런데 공개된 GitHub 코드는 전체 기간을 통째로 쓰는 단일 상관 `returns.corr()`를 사용해서, 논문의 125일 롤링 지시와 **어긋납니다**. 재현 실험을 돌릴 때 이 불일치를 반드시 확인하고, 필요하면 125일 롤링으로 바로잡아야 결과가 논문과 맞습니다.

GARCH·RiskMetrics 벤치마크

DeepVaR가 얼마나 좋은지 비교하려면 기준선(benchmark)이 필요하죠. 대표적인 변동성 모델 두 가지를 짚어둡니다.

GARCH(1,1)(식 4~6)은 변동성이 시간에 따라 변한다고 보는 고전 모델입니다.

$$r_t = \mu + \epsilon_t$$

$$\epsilon_t = \sigma_t e_t, \quad e_t \sim N(0, 1)$$

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

기호를 풀면,

- r_t : t 일 수익률, μ : 평균 수익률, ϵ_t : 그날의 충격(잔차)
- σ_t : t 일의 변동성(표준편차), e_t : 표준정규분포를 따르는 순수 노이즈
- σ_t^2 : t 일의 분산(변동성 제곱)
- ω : 기본 상수(장기 변동성 수준), α : 최근 충격의 반영 강도, β : 과거 변동성의 지속성

 **쉬운 설명** — 세 번째 식이 핵심입니다. **오늘의 분산 = 상수 ω + 어제 받은 충격의 제곱($\alpha \epsilon_{t-1}^2$, 최근의 놀람) + 어제의 분산($\beta \sigma_{t-1}^2$, 지속성)**. 즉 어제 시장이 크게 출렁였으면 오늘도 출렁일 가능성이 높다는, 변동성의 "관성"을 수식으로 표현한 겁니다. $\alpha + \beta$ 가 1에 가까울수록 변동성이 오래 지속됩니다.

RiskMetrics(RM)는 GARCH를 아주 단순하게 만든 버전으로, 지수가중이동평균(EWMA)을 씁니다.

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_{t-1}^2$$

- λ : 감쇠계수(decay factor). 이 논문(및 RiskMetrics 표준)에서는 $\lambda = 0.94$.
- 과거 분산 σ_{t-1}^2 에 0.94, 어제 수익률 제곱 r_{t-1}^2 에 0.06의 가중치를 줍니다.

 **예시** — $\lambda = 0.94$ 이면 유효하게 반영되는 과거는 약 **74일**입니다. 실제로 $0.94^{74} \approx 0.01$ 이라, 74일 전 데이터의 가중치는 최근치의 약 1%로 쪼그라들거든요. 그래서 RM은 "최근 두 달 반 정도"만 보는 셈입니다.

실험과 결과

이제 실전 검증입니다. 데이터는 외환(FX) 4쌍 — **AUDUSD, GBPUSD, USDJPY, EURUSD** — 이고, 기간은 2007/01/01부터 2020/12/18까지입니다. 테스트는 2018~2020년 구간을 **1일 롤링**으로 진행했어요. 포트폴리오에는 **1000개를 무작위로** 생성했는데, 롱·숏을 섞되 가중치의 절댓값 합이 1이 되도록 맞췄습니다.

비교 대상은 총 5종입니다: **GARCH(1,1), RiskMetrics(RM), 역사적 시뮬레이션(HS), 몬테카를로(MC), BiGAN**. 각 모델의 성격을 정리하면 이래요.

모델	성격	위기 대응의 약점
HS / MC	큰 과거창을 참조	급변에 느리게 적응 → VaR 곡선이 긴 직선처럼 뻗뻗함
RiskMetrics	최근 74일만 참조	실제엔 가깝지만 위반이 가장 많음
BiGAN	생성모델	좁은 범위에서만 진동 → 급락을 못 잡음
DeepVaR	900일 학습 + 최근 15일 컨텍스트	(해당 트레이드오프를 극복)

핵심 결과를 정리하면 다음과 같습니다.

- **99% VaR에서 거의 모든 손실지표가 최저** — 이차·부드러운·편 손실 등 대부분의 지표에서 DeepVaR가 1등이었습니다.
- **위반율이 명목 임계값 아래인 유일한 모델** — 위반율이 $1 - \alpha = 0.01$ 밑으로 내려간 것은 DeepVaR뿐이었어요. 나머지는 위험을 과소평가했다는 뜻입니다.
- **뒤틀린 위반을 피하는 유일한 모델** — 비선형성 덕분에 서로 무상관인 VaR 추정치를 만들어, 위반이 한 시기에 몰리지 않았습다. 앞서 본 Christoffersen·DQ 검정의 관점에서 결정적 강점이에요.
- **유일한 약점은 틱 손실** — 딱 한 지표, 틱 손실에서만 다른 모델에 뒤졌습니다.

🎯 **핵심** — DeepVaR가 이길 수 있었던 비결은 "**과거 기억**"과 "**최근 적응**"의 트레이드오프를 동시에 잡은 것입니다. 900일치로 넉넉히 학습해 과거의 극단 사건을 기억하면서도, 예측할 때는 최근 15일(컨텍스트)에 집중해 요즘 시장에 빠르게 적응합니다. HS·MC는 과거에 갇혀 굼뜨고, RM은 최근에만 매달려 위반이 잦은데, DeepVaR는 그 사이의 균형점을 찾은 거예요.

결론적으로 DeepVaR는 위험 추정의 정확도를 높이는 동시에, 유동자산에 대해 **더 적은 자본을 쌓아도 되게** 해줍니다 (위험을 정확히 재니 과도한 자본 버퍼가 불필요). 저자들이 제시한 향후 과제는 세 가지예요: ① 고빈도 거래에서의 계산 효율화, ② 하이퍼파라미터 튜닝 고도화, ③ 뉴스·트윗 같은 텍스트를 활용한 **감성분석(sentiment analysis)** 통합으로 시장 심리까지 위험 모델에 반영하기.

논문 ② 완전 해설 — Multi-Transformer (Ramos-Pérez et al., Mathematics, 2021)

부제 — "S&P 변동성 예측을 위한 새로운 신경망 기반 아키텍처"

앞의 논문(DeepVaR)이 **외환(FX, Foreign Exchange) 수익률의 분포 자체**를 신경망으로 그려 놓고 그 분포의 왼쪽 꼬리에서 VaR를 읽어냈다면, 이번 논문은 겨냥하는 표적이 다르다. 이 논문은 먼저 **S&P 주가의 변동성(volatility) 그 자체**를 예측하고, 그 예측된 변동성을 "분포의 폭(너비)"으로 삼아 거기서 VaR를 뽑아낸다. 즉 "분포를 통째로 맞히는" 대신, "출렁임의 크기를 맞히고 → 그 크기로 분포를 세우고 → 위험을 잴다"는 2단계 전략이다.

💡 쉬운 설명 — 변동성은 "주가가 앞으로 얼마나 크게 출렁일까"를 나타내는 숫자다. 이 논문의 목표는 미래의 출렁임 크기 σ 를 먼저 맞히고, 그 크기를 분포의 폭으로 써서 손실 한도인 VaR를 계산하는 것이다. DeepVaR가 "결과(수익률 분포)"를 직접 그렸다면, 여기서는 "흔들림의 세기"를 먼저 그린 뒤 위험을 유도한다.

초록·서론

이야기는 위기에서 출발한다. **2007-2008 글로벌 금융위기**와 뒤이은 **코로나19 팬데믹**은 은행과 보험사에 막대한 손실을 안겼다. 이런 사건이 반복되면서 "주식 위험을 정확히 재는 모델"과 "그 위험을 상쇄하는 **헤징(hedging)**"의 중요성이 그 어느 때보다 커졌다. 그런데 문제가 하나 있다. 위험의 핵심 재료인 변동성은 **직접 관측되지 않는다**는 점이다.

💡 쉬운 설명 — 주가는 눈으로 볼 수 있지만("오늘 증가 4,000") 변동성은 그럴 수 없다. "오늘의 출렁임 정도"라는 건 시장이 움직인 흔적을 보고 통계적으로 추정해야만 얻어지는 숨은 값이다. 그래서 변동성 예측은 곧 "보이지 않는 값을 데이터로 역추적하는" 작업이 된다.

변동성을 모델링하는 방법은 크게 세 가족으로 나뉜다. 이 분류는 이 논문 전체의 지도이므로 표로 정리해 두자.

가족	핵심 아이디어	대표 모델
① GARCH 계열	변동성이 "몰려다니는" 성질을 통계식으로 잡음	GARCH, EGARCH, GJR-GARCH
② 확률적 변동성(stochastic volatility)	변동성 자체가 하나의 확률과정을 따른다고 봄	Heston 모델
③ 머신·딥러닝(ML/DL)	데이터에서 패턴을 직접 학습	ANN, 부스팅, 랜덤포레스트, SVM

첫째, **GARCH 계열**은 **변동성 군집(volatility clustering)**이라는 현상을 반영한다. 이는 "출렁임이 큰 날 뒤엔 큰 날이, 잔잔한 날 뒤엔 잔잔한 날이 몰려 온다"는 성질이다. 특히 **EGARCH**와 **GJR-GARCH**는 하락장과 상승장이 변동성에 미치는 영향이 다르다는 **비대칭(asymmetry)** 까지 반영한다.

 **예시** — 같은 크기라도 -3% 하락일이 +3% 상승일보다 이후의 변동성을 더 크게 키우는 경향이 있다(이른바 레버리지 효과). 대칭 모델은 이 둘을 똑같이 취급하지만, EGARCH·GJR-GARCH는 "떨어질 때 시장이 더 놀란다"를 식 안에 담아 낸다.

둘째, **확률적 변동성** 모델은 변동성이 그 자체로 별도의 확률과정을 따른다고 본다. 대표 주자가 옵션 가격결정에 널리 쓰이는 **Heston 모델**이다. 셋째, **머신·딥러닝** 접근은 인공지능망(ANN), 부스팅, 랜덤포레스트, SVM 등을 쓴다. 특히 **ANN을 GARCH와 결합한 하이브리드**가 흔하며, 잘 만들면 전통 GARCH를 능가한다.

이 논문이 새로 내놓는 것은 **트랜스포머(Transformer) 층과 멀티-트랜스포머(Multi-Transformer) 층을 GARCH 또는 LSTM과 결합한 하이브리드**다. 트랜스포머는 원래 자연어처리(NLP)의 최첨단 구조로, **BERT**와 **GPT-3** 같은 모델의 뼈대다. 핵심은 RNN·LSTM 같은 **순환구조 없이**, 오직 **멀티-헤드 어텐션(multi-head attention)** 과 **위치 임베딩(positional embedding)** 만으로 시계열을 처리한다는 점이다.

 **쉬운 설명** — 순환구조(RNN·LSTM)는 데이터를 한 시점씩 순서대로 훑어야 해서 느리다. 트랜스포머는 전체 시퀀스를 한꺼번에 펼쳐 놓고 "어느 시점이 어느 시점을 얼마나 참고할지"를 어텐션으로 병렬 계산한다. 순서 정보는 별도의 위치 신호로 따로 넣어 준다.

멀티-트랜스포머는 여기서 한 걸음 더 나간다. 학습 데이터의 서로 다른 부분집합을 무작위로 골라 여러 개의 멀티-헤드 어텐션을 만들고, 그 결과를 하나로 병합한다. 이것은 통계학의 **배깅(bagging)** 직관을 어텐션에 이식한 것이다(자세한 원리는 뒤에서 다룬다).

이렇게 만든 신경망에 결합하는 GARCH는 총 6종이다: **GARCH, EGARCH, GJR-GARCH, TrGARCH, FIGARCH, AVGARCH**. 논문의 세 가지 기여를 정리하면 다음과 같다.

기여	내용
①	트랜스포머 층을 변동성 예측용으로 개조 하고, 이를 확장한 멀티-트랜스포머를 제안
②	이 신경망을 GARCH와 결합하면 더 정확한 예측이 됨을 보임
③	저변동·고변동 국면 모두에서 적절한 위험측정 이 가능함을 검증

 **핵심** — 한 문장으로 요약하면 이렇다. "**NLP의 트랜스포머를 변동성 예측용으로 뜯어고치고(개조), 배깅으로 안정화한(멀티-트랜스포머) 뒤, 전통 통계모형 GARCH와 손을 잡게 해서(하이브리드) 위기·평시 모두에서 잘 맞는 위험측정을 만든다.**"

데이터·입력·출력

모델을 굴리는 방식은 **롤링 윈도우(rolling window)**다. 창(window)의 크기는 **650**, 예측 지평(forecast horizon)은 **1일**이다. 즉 과거 650개 관측치로 학습하고 바로 다음 1일을 예측한 뒤, 창을 하루씩 밀며 이 과정을 반복한다.

 **쉬운 설명** — 롤링 윈도우는 "최근 650일만 보는 이동하는 관찰창"이다. 하루가 지나면 가장 오래된 하루를 버리고 새 하루를 넣어 다시 학습한다. 덕분에 모델이 항상 "최신 시장 상황"에 맞춰 갱신된다.

입력(input) 부터 보자. 첫 번째 자료는 직전 5일 로그수익률의 표준편차다(식 1).

$$\sigma_{t-1} = \sqrt{\frac{\sum_{i=1}^n (r_{t-i} - E[r])^2}{n-1}}$$

기호를 하나씩 풀면 이렇다. σ_{t-1} 은 시점 $t-1$ 에서 계산한 변동성(표준편차), r_{t-i} 는 i 일 전의 로그수익률, $E[r]$ 은 그 구간 수익률의 평균, n 은 사용한 날 수(여기서는 5), 분모 $n-1$ 은 표본 표준편차를 만들 때 쓰는 자유도 보정이다. 루트 안은 "각 수익률이 평균에서 얼마나 벗어났는가"를 제곱해 더한 뒤 평균 낸 값, 즉 분산이고, 루트를 씌워 표준편차로 만든다.

 **예시** — 최근 5일 로그수익률이 0.01, -0.02, 0.015, -0.005, 0이라 하자. 평균 $E[r] = 0$ 이고, 편차 제곱의 합은 $0.0001 + 0.0004 + 0.000225 + 0.000025 + 0 = 0.00075$ 다. 이를 $n-1 = 4$ 로 나누면 0.0001875, 루트를 씌우면 $\sigma_{t-1} \approx 0.0137$, 즉 약 1.37%의 변동성이 된다.

트랜스포머와 LSTM은 시계열 모델이라 한 시점의 값 하나가 아니라 **직전 10개 시차(lag)**를 한꺼번에 받는다. 그래서 입력 벡터가 두 개다(식 2, 식 3).

$$X_1 = (\sigma_{t-1}, \sigma_{t-2}, \dots, \sigma_{t-10})$$

$$X_2 = (r_{t-1}, r_{t-2}, \dots, r_{t-10})$$

X_1 은 지난 10일 각각에 대해 계산한 5일 변동성들을 늘어놓은 것이고, X_2 는 지난 10일의 로그수익률을 늘어놓은 것이다. 즉 모델은 "최근 변동성의 흐름"과 "최근 수익률의 흐름"을 함께 본다.

이제 **출력(반응변수, response variable)**이다. 목표는 **실현 변동성(realized volatility)**이다(식 4). 여기서 $E[r_f] = \sum_{n=0}^{i-1} r_{t+n}/i$ 이고 $i = 5$ 다.

$$Y = \hat{\sigma}_{i,t} = \sqrt{\frac{\sum_{n=0}^{i-1} (r_{t+n} - E[r_f])^2}{i-1}}$$

핵심 차이를 놓치면 안 된다. 입력의 σ_{t-1} 은 **과거** 방향(r_{t-i})을 봤지만, 출력의 $\hat{\sigma}_{i,t}$ 는 **미래** 방향(r_{t+n} , 즉 $n = 0, 1, \dots, i-1$ 일 뒤)을 본다. $E[r_f]$ 는 앞으로 5일간 수익률의 평균이고, 분모 $i-1 = 4$ 는 역시 자유도 보정이다. 결국 예측 목표 Y 는 **"앞으로 5일간 로그수익률의 표준편차"**다.

 **핵심** — 입력은 "지금까지의 변동성·수익률", 출력은 "앞으로 5일의 변동성"이다. 모델은 과거 10시차를 보고 미래 5일의 출력임 크기를 맞히도록 학습된다. 이 미래 5일 표준편차가 뒤에서 VaR를 만드는 분포의 폭이 된다.

데이터는 **S&P 지수**, 기간은 **2008/01/01 ~ 2020/12/31**이다. 최적 설정(하이퍼파라미터)은 **2008-2015 구간의 RMSE를 최소화**하도록 고르고, 최종 성능은 **2016-2020 구간**에서 테스트한다.

 **주의** — 설정 선택과 최종 평가의 기간을 분리한 것이 중요하다. 만약 테스트 구간(2016-2020)을 보고 설정을 골랐다면 "미래를 훑쳐본" 셈이라 성능이 부풀려진다. 학습·검증 기간(2008-2015)에서만 튜닝하고 테스트 기간은 마지막에 딱 한 번 채점에 쓰는 것이 정직한 절차다.

모델 검증

검증은 두 축으로 이뤄진다. 하나는 "변동성 예측이 얼마나 정확한가", 다른 하나는 "거기서 나온 VaR가 실제 위험을 제대로 재는가"다.

먼저 예측 정확도는 **MAE(Mean Absolute Error, 평균절대오차)** 와 **RMSE(Root Mean Squared Error, 평균제곱근오차)** 로 잰다(식 5).

$$MAE = \frac{\sum_{t=1}^N |\sigma_{i,t} - \hat{\sigma}_{i,t}|}{N}, \quad RMSE = \sqrt{\frac{\sum_{t=1}^N (\sigma_{i,t} - \hat{\sigma}_{i,t})^2}{N}}$$

$\sigma_{i,t}$ 는 실제 실현 변동성, $\hat{\sigma}_{i,t}$ 는 모델의 예측 변동성, N 은 테스트 시점 개수다. MAE는 오차의 절댓값을 평균 내므로 "평균적으로 몇 %p 틀리는가"를 직관적으로 보여 준다. RMSE는 오차를 제곱해 평균 낸 뒤 루트를 씌우므로 **큰 오차에 더 크게 벌점**을 준다.

 **쉬운 설명** — MAE와 RMSE의 차이는 "벌점 방식"에 있다. MAE는 2만큼 틀리든 4만큼 틀리든 그대로 벌점을 주지만, RMSE는 제곱을 하므로 4만큼 틀린 큰 실수를 훨씬 아프게 벌한다. 그래서 "가끔 크게 빗나가는 것"을 특히 싫어할 때 RMSE를 본다.

위험측정은 **Solvency II** 규제에 맞춰 **99.5% VaR** 를 쓴다. Solvency II는 유럽 보험사의 지급여력 규제로, 1년에 200번 중 1번(0.5%) 꼴로만 넘어서는 손실 한도를 요구한다.

 **예시** — 99.5% VaR는 "정상적인 하루 손실이 이 선을 넘을 확률이 0.5%"라는 뜻이다. 뒤집으면 200일 중 약 1일만 이 선을 넘는 게 정상이다. 신뢰수준이 95%인 VaR보다 훨씬 바깥쪽(더 나쁜 시나리오)을 자르므로 값이 크다.

VaR가 잘 맞는지는 두 가지 검정으로 채점한다.

검정	무엇을 보는가
Kupiec 검정(Kupiec test)	초과(exceedance)가 총 몇 번 일어났는지가 신뢰수준과 맞는가
Christoffersen 검정(Christoffersen test)	초과가 연속으로 뭉치지 않는지(독립성) 까지 함께 봄

 **쉬운 설명** — Kupiec은 "초과 횟수의 총량"만 센다. 예컨대 99.5% VaR라면 1000일 중 초과가 대략 5번이어야 하는데, 15번이 나오면 모델이 위험을 과소평가한 것이다. Christoffersen은 여기에 더해 "초과가 특정 시기에 우르르 몰려 있지 않은가"를 본다. 초과가 며칠씩 붙어 나오면 모델이 위기 국면을 못 따라간다는 신호이기 때문이다.

벤치마크 GARCH 6종

비교 대상(벤치마크)이 되는 전통 모델 6종을 본다. 모든 GARCH의 공통 가정은 **혁신항(innovation) ϵ_t 가 스튜던트 t 분포(Student's t distribution)** 를 따른다는 것이다. 정규분포 대신 t 분포를 쓰는 이유는 금융 수익률의 **두꺼운 꼬리(fat tail)**, 즉 "극단적 사건이 정규분포 예측보다 자주 일어나는" 성질을 담기 위해서다.

💡 **쉬운 설명** — 정규분포는 극단적 급락·급등을 "거의 일어나지 않는 일"로 본다. 하지만 실제 시장에서는 그런 사건이 생각보다 자주 터진다. 스튜던트 t 분포는 종 모양의 양쪽 끝(꼬리)이 정규분포보다 두꺼워서, 이 "예상보다 잦은 극단"을 더 현실적으로 반영한다.

기본형인 **GARCH(p, q)** 부터 보자(식 6).

$$\hat{\sigma}_t^2 = \omega + \sum_{i=1}^q \alpha_i r_{t-i}^2 + \sum_{i=1}^p \beta_i \sigma_{t-i}^2$$

$\hat{\sigma}_t^2$ 은 예측 분산, ω 는 상수(기저 변동성), α_i 는 과거 충격 r_{t-i}^2 (제곱수익률)에 붙는 가중치, β_i 는 과거 분산 σ_{t-i}^2 에 붙는 가중치다. 두 합 기호는 각각 "과거 q 개의 충격"과 "과거 p 개의 변동성"을 끌어와 오늘 변동성을 만든다는 뜻이다. 이 구조가 바로 변동성 군집을 표현한다: 어제 충격이 컸으면(r_{t-1}^2 이 크면) 오늘 변동성도 커진다.

나머지 5종은 이 기본형을 조금씩 비틀어 만든다. 아래 표로 큰 그림을 먼저 잡자.

모델	기본형과의 차이	잡아내는 성질
GARCH	—	변동성 군집
AVGARCH	제곱 대신 절댓값	이상치에 덜 민감
EGARCH	로그 를 씌우고 비대칭항 추가	하락·상승 비대칭
GJR-GARCH	하락일에만 커지는 항 추가	하락 충격 강조
TrGARCH	절댓값 + 임계값(threshold) 항	하락 비대칭(절댓값판)
FIGARCH	분수차분으로 장기기억	오래 지속되는 충격

AVGARCH 는 제곱 대신 절댓값을 쓴다(식 7).

$$\hat{\sigma}_t = \omega + \sum_{i=1}^q \alpha_i |r_{t-i}| + \sum_{i=1}^p \beta_i \sigma_{t-i}$$

제곱은 큰 값을 더 크게 부풀리는데, 절댓값 $|r_{t-i}|$ 은 그런 증폭 없이 크기만 취한다. 그래서 극단적 하루 수익률(이상치)에 덜 휘둘린다.

EGARCH 는 로그를 씌우고 비대칭항을 넣는다(식 8). 여기서 $e_t = r_t / \sigma_t$ 는 표준화 잔차다.

$$\log \hat{\sigma}_t^2 = \omega + \sum_{i=1}^p \alpha_i \log \hat{\sigma}_{t-i}^2 + \sum_{i=1}^q (\beta_i e_{t-i} + \gamma_i (|e_{t-i}| - E|e_{t-i}|))$$

좌변이 $\log \hat{\sigma}_t^2$ 인 것이 핵심이다. 로그를 취하면 분산이 항상 양수가 되도록 강제할 필요가 없어져(로그를 벗기면 지수함수라 자동으로 양수) 파라미터에 제약을 덜 걸어도 된다. 괄호 안 $\beta_i e_{t-i}$ 항이 **부호(방향)**를 반영하는 비대칭 부분이다. e_{t-i} 가 음수(하락)면 이 항이 음/양으로 다르게 작동해 하락·상승의 효과가 달라진다. $\gamma_i (|e_{t-i}| - E|e_{t-i}|)$ 는 크기 효과를 담는다.

 **쉬운 설명** — EGARCH에서 e_{t-i} 의 부호가 있는 항 덕분에 "떨어질 때"와 "오를 때"가 변동성에 다른 영향을 준다. 로그를 쓴 덕분에 계산 도중 분산이 음수가 될 걱정도 사라진다. 두 마리 토끼(비대칭 + 양수 보장)를 한 번에 잡은 셈이다.

GJR-GARCH 는 하락일에만 켜지는 스위치를 단다(식 9). $I_{[r_{t-1}<0]}$ 은 지시함수(indicator)로, 하락일이면 1, 아니면 0이다.

$$\hat{\sigma}_t^2 = \omega + \sum_{i=1}^q \alpha_i r_{t-i}^2 + \sum_{i=1}^o \gamma_i r_{t-i}^2 I_{[r_{t-i}<0]} + \sum_{i=1}^p \beta_i \sigma_{t-i}^2$$

기본 GARCH에 세 번째 항이 추가됐다. 이 항은 $I_{[r_{t-1}<0]} = 1$ 일 때, 즉 **하락일에만** $\gamma_i r_{t-i}^2$ 만큼 변동성을 더 얻는다. 그래서 하락 충격이 상승 충격보다 변동성을 더 키우는 비대칭이 자연스럽게 표현된다.

 **예시** — 어제 -2% 떨어졌다면 $I = 1$ 이 되어 $\alpha_i r_{t-1}^2 + \gamma_i r_{t-1}^2$ 만큼 반영되지만, 어제 $+2\%$ 올랐다면 $I = 0$ 이라 $\alpha_i r_{t-1}^2$ 만 반영된다. 같은 2% 라도 하락일 때 오늘 변동성이 γ 만큼 더 커지는 것이다.

TrGARCH(Threshold GARCH) 는 같은 아이디어를 절댓값 버전으로 구현한다(식 10).

$$\hat{\sigma}_t = \omega + \sum_{i=1}^q \alpha_i |r_{t-i}| + \sum_{i=1}^o \gamma_i |r_{t-i}| I_{[r_{t-i}<0]} + \sum_{i=1}^p \beta_i \sigma_{t-i}$$

구조는 GJR-GARCH와 쌍둥이지만, 제곱 r_{t-i}^2 대신 절댓값 $|r_{t-i}|$ 을 쓴다는 점이 다르다. 세 번째 항의 지시함수 $I_{[r_{t-i}<0]}$ 이 하락일에만 켜져 비대칭을 만든다.

FIGARCH(Fractionally Integrated GARCH) 는 **장기 기억(long memory)**을 담는다(식 11). L 은 시차연산자(lag operator), d 는 분수차분(fractional differencing) 차수다.

$$\hat{\sigma}_t = \omega + [1 - \beta L - \phi L(1 - L)^d] \epsilon_t^2 + \sigma h_{t-1}$$

시차연산자 L 은 "한 시점 뒤로 미는" 연산자로, $L x_t = x_{t-1}$ 이다. 보통의 차분은 차수가 정수($d = 0, 1, 2, \dots$)지만, 여기서는 d 가 0과 1 사이의 **분수**일 수 있다. 이 분수차분 $(1 - L)^d$ 덕분에 과거 충격이 아주 천천히, 오랫동안 감쇠하며 영향을 남긴다.

 **쉬운 설명** — 보통 GARCH에서 옛 충격의 영향은 빠르게(지수적으로) 사라진다. FIGARCH의 분수차분은 그 감쇠를 느리게(멱함수적으로) 만들어서, 몇 달 전 충격이 오늘까지 희미하게 살아 있게 한다. "시장은 큰 사건을 오래 기억한다"를 수식으로 옮긴 것이다.

마지막으로 신경망 하이브리드 벤치마크도 둘 있다. **ANN-GARCH** 는 피드포워드 신경망(구조 $16 \rightarrow 8 \rightarrow 1$)을 GARCH와 결합한 것이고, **LSTM-GARCH** 는 32유닛 LSTM 뒤에 피드포워드($8 \rightarrow 1$)를 붙여 GARCH와 결합한 것이다. 이 논문이 제안하는 트랜스포머 계열 모델은 결국 이들을 넘어서는 것이 목표다.

트랜스포머 개조 — 위치 인코더

트랜스포머의 아킬레스건은 **순서 정보의 부재**다. 어텐션은 모든 시점을 동시에 보므로, 그대로 두면 "3일 전"과 "10일 전"을 구분하지 못한다. 그래서 각 위치에 고유한 "위치 도장"을 찍어 주는 **위치 인코딩(positional encoding)**이

필요하다.

원래 NLP용(Vaswani et al.) 위치인코딩은 사인·코사인을 번갈아 쓴다(식 14, 식 15). 여기서 dim 은 설명변수(특징)의 개수다.

$$PE_{(pos, 2i)} = \sin(pos/1000^{2i/dim})$$

$$PE_{(pos, 2i+1)} = \cos(pos/1000^{2i/dim})$$

pos 는 시퀀스에서의 위치(몇 번째인지), i 는 특징 벡터 안에서의 차원 인덱스다. 짝수 차원($2i$)에는 사인을, 홀수 차원($2i + 1$)에는 코사인을 쓰고, 차원마다 주파수($1000^{2i/dim}$ 로 조절)를 달리해 위치마다 고유한 패턴을 만든다.

문제는 변동성 데이터엔 "단어"가 없다는 점이다. NLP식 인코딩은 차원(단어 임베딩) 개수 dim 에 의존하는데, 변동성 예측에서는 그렇게 여러 차원이 필요 없다. 그래서 저자들은 **변수 개수와 무관하고 오직 시차(위치)에만 의존하는** 위치인코더로 수정한다(식 16). 여기서 $pos = 0, 1, \dots, N_{pos} - 1$ 이고 N_{pos} 는 최대 시차다.

$$PE_{pos} = \cos\left(\frac{\pi pos}{N_{pos} - 1}\right) = \sin\left(\frac{\pi}{2} + \frac{\pi pos}{N_{pos} - 1}\right)$$

pos 가 0에서 $N_{pos} - 1$ 까지 갈 때, 코사인의 각도는 0에서 π 까지 움직인다. 코사인은 이 구간에서 **단조감소**하므로, 위치값이 $+1$ 에서 -1 까지 부드럽게 줄어든다. 즉 각 시차가 서로 다른 고유한 위치값을 하나씩 받는다. 우변은 삼각함수 항등식 $\cos \theta = \sin(\frac{\pi}{2} + \theta)$ 로 같은 값을 사인으로 다시 쓴 것뿐이다.

 **예시** — 최대 시차 $N_{pos} = 10$ 이라 하자. $pos = 0$ 이면 $\cos(0) = 1$, $pos = 9$ 이면 $\cos(\pi) = -1$ 이 되어 위치값이 $+1 \rightarrow -1$ 로 매끄럽게 내려간다. 가장 가까운 시차와 가장 먼 시차가 정확히 양 끝값을 받고, 중간 시차들은 그 사이를 채운다. 변수 개수가 몇 개든 이 값은 바뀌지 않는다.

 **핵심** — 개조의 요점은 "단어 임베딩 차원에 매달리던 NLP식 위치인코딩을, 시차에만 의존하는 단순하고 매끄러운 코사인 곡선으로 바꾼 것"이다. 변동성처럼 특징이 몇 개뿐인 시계열에 딱 맞게 트랜스포머를 재단한 셈이다.

어텐션·멀티-헤드

이제 트랜스포머의 심장인 어텐션이다. 기본 단위는 **스케일드 닷-프로덕트 어텐션(scaled dot-product attention)** 이다(식 17).

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

세 재료 Q (Query, 질의), K (Key, 키), V (Value, 값)는 모두 입력을 각각 다른 가중치로 변환해 만든 행렬이다. QK^T 은 질의와 키의 내적으로 "어느 시점이 어느 시점과 얼마나 관련 있는가"의 점수를 만든다. 이를 $\sqrt{d_k}$ (키 벡터의 차원 d_k 의 제곱근)로 나눠 크기를 안정화한 뒤, softmax로 확률처럼 정규화(합이 1)한다. 마지막으로 이 가중치를 V 에 곱해 "관련 높은 시점의 값을 더 많이 가져오는" 가중합을 만든다.

 **쉬운 설명** — 어텐션은 도서관에서 자료를 찾는 것과 비슷하다. 내 질문(Q)을 각 책의 색인(K)과 대조해 관련도 점수를 매기고, 그 점수대로 책 내용(V)을 섞어 답을 만든다. $\sqrt{d_k}$ 로 나누는 이유는 점수가 너무 커지면 softmax가 한쪽에만 쏠려 학습이 불안정해지기 때문이다. 예컨대 $d_k = 64$ 면 $\sqrt{64} = 8$ 로 나눠 점수를 적당한 크기로 눌러 준다.

한 종류의 어텐션만 쓰면 관점이 하나뿐이다. **멀티-헤드 어텐션**은 여러 개의 어텐션(헤드)을 병렬로 돌려 다양한 관점을 동시에 본다(식 18, 식 19).

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

h 개의 헤드를 만드는데, 각 헤드 head_i 는 Q, K, V 를 각자의 가중치 행렬 W_i^Q, W_i^K, W_i^V 로 변환한 뒤 어텐션을 수행한다. 그렇게 나온 h 개의 결과를 Concat (옆으로 이어 붙이기)한 다음, 출력 가중치 W^O 로 한 번 더 섞어 최종 출력을 만든다. 모든 가중치 행렬(W_i^Q, W_i^K, W_i^V, W^O)은 **선형활성(linear activation) 피드포워드 층**으로 학습된다.

 **예시** — 헤드가 여러 개면 한 헤드는 "직전 하루"에 집중하고, 다른 헤드는 "일주일 주기"에 집중하는 식으로 서로 다른 시간 패턴을 나눠 볼 수 있다. 여러 전문가의 의견을 모아 종합하는 것과 같다.

트랜스포머 층 전체의 조립 순서는 다음과 같다.

순서	구성요소
1	멀티-헤드 어텐션
2	정규화(normalization)
3	ReLU 피드포워드
4	정규화

여기에 더해 **2개의 잔차연결(residual connection)**이 들어간다. 잔차연결은 어떤 층의 입력을 그 층의 출력에 그대로 더해 주는 지름길이다.

 **쉬운 설명** — 잔차연결은 "이 층을 통째로 건너뛰어도 되는 우회로"를 만들어 준다. 모델이 학습 중에 "이 층은 별 도움이 안 되네" 싶으면 우회로로 정보를 그대로 흘려보내고, 도움이 되면 층을 거친 결과를 쓴다. 즉 모델이 **어느 층을 학습에서 건너뛸지 스스로 결정**한다. 덕분에 층을 깊이 쌓아도 학습이 잘 된다.

멀티-트랜스포머 — 어텐션에 배경

멀티-트랜스포머의 아이디어는 통계학의 **배깅(bootstrap aggregating)**을 어텐션에 이식하는 것이다. 방법은 이렇다. 입력에서 무작위 표본을 T 개 만든다(각 표본은 전체 관측치의 90%, $T = 5$). 그리고 T 개의 멀티-헤드 어텐션을 만든 뒤 그 결과를 평균 낸다. 이것을 **평균 멀티-헤드(Average Multi-Head, AMH)**라고 부른다(식 20, 식 21).

$$\text{AMH}(Q, K, V) = \frac{\sum_{t=1}^T \text{Concat}(\text{head}_{1,t}, \dots, \text{head}_{h,t}) W_t^O}{T}$$

$$\text{head}_{i,t} = \text{Attention}(Q_t W_{i,t}^Q, K_t W_{i,t}^K, V_t W_{i,t}^V)$$

식을 읽어 보자. 아래 첨자 t 는 이제 시점이 아니라 **표본 번호**($t = 1, \dots, T$)를 뜻한다. $\text{head}_{i,t}$ 는 t 번째 표본(Q_t, K_t, V_t)으로 만든 i 번째 헤드다. 각 표본마다 멀티-헤드 어텐션 한 번($\text{Concat}(\cdot) W_t^O$)을 통째로 만든 뒤, 그것들을 모두 더해 T 로 나눈다. 즉 **T 개의 멀티-헤드 어텐션을 평균**한 것이 AMH다.

 **예시** — 관측치가 100개면 매번 90개를 무작위로 뽑아 어텐션을 하나 만들고, 이를 $T = 5$ 번 반복해 5개의 어텐션 결과를 얻은 뒤 평균 낸다. 5명의 서로 다른 표본으로 학습한 전문가에게 물어보고 답을 평균 내는 것과 같다.

핵심은 **배깅을 어텐션 메커니즘에만** 적용한다는 점이다. 신경망 전체를 여러 번 복제하는 게 아니라 어텐션 부분만 여러 표본으로 앙상블하므로, 안정성과 정확도를 끌어올리면서도 계산부담을 일부 상쇄한다.

 **핵심** — 배깅은 "여러 표본으로 만든 예측을 평균 내 오차의 분산을 줄이는" 기법이다. 평균을 내면 각 표본 특유의 요동(분산)이 상쇄돼 예측이 안정된다. 멀티-트랜스포머는 이 원리를 어텐션에만 적용해 **과적합(overfitting) 위험을 낮추고** 정확도를 높인다.

다만 공짜 점심은 없다. 장단점을 정리하면 이렇다.

구분	내용
장점	오차의 분산이 크게 감소, 과적합 위험 감소
단점	편향(bias)은 그대로 보존 → 과소적합(underfitting) 이 생길 수 있음

 **주의** — 배깅은 분산은 줄여도 편향은 줄이지 못한다. 원래 모델이 구조적으로 틀린 방향으로 치우쳐 있다면(편향이 크다면), 아무리 여러 표본을 평균 내도 그 치우침은 그대로 남는다. 그래서 모델이 지나치게 단순하면 과소적합, 즉 "데이터의 패턴을 충분히 못 잡는" 상태가 될 수 있다.

제안 모델 4종·학습 설정

이제 재료가 다 모였다. 논문이 제안하는 신경망은 4종이며, 트랜스포머냐 멀티-트랜스포머냐, LSTM을 붙이냐 마느냐의 조합이다. 모두 마지막에 6종 GARCH와 결합한다.

모델	구성	특징
T-GARCH	트랜스포머 + 피드포워드 + 6 GARCH	기본 트랜스포머 하이브리드
TL-GARCH	T-GARCH + LSTM(32유닛)	LSTM이 시간구조를 잡아 위치인코더 불필요
MT-GARCH	멀티-트랜스포머 + 6 GARCH	어텐션에 배깅 적용
MTL-GARCH	멀티-트랜스포머 + LSTM	배깅 + LSTM (최강 조합 후보)

 **쉬운 설명** — LSTM을 붙인 모델(TL, MTL)은 위치인코더가 필요 없다. LSTM 자체가 시점을 순서대로 훑으며 시간 구조를 파악하기 때문에, 굳이 위치 도장을 따로 찍어 줄 이유가 사라지는 것이다. 순환구조가 순서를 자연히 알고 있는 셈이다.

학습 설정은 네 모델이 공통으로 쓴다.

설정	값
최적화기(optimizer)	ADAM
초기 학습률	$\delta = 0.01$
피드포워드	8뉴런 → 출력 1뉴런
손실함수	MSE(Mean Squared Error)
배치 크기	64
에폭(epoch)	5000

여기서 $\delta = 0.01$ 은 학습 초기의 보폭(한 번에 파라미터를 얼마나 크게 조정할지)이고, 손실함수 MSE는 예측 변동성과 실제 변동성의 제곱오차를 최소화하도록 학습을 이끈다. 배치 64는 한 번에 64개 표본을 묶어 갱신한다는 뜻, 에폭 5000은 전체 데이터를 5000번 반복 학습한다는 뜻이다.

위험측정(VaR 산출) 단계에서는 예측된 변동성을 **비표준 스튜던트 t 분포(non-standardized Student's t distribution)**의 척도로 쓴다. 즉 이 t 분포의 **표준편차를 모델이 예측한 변동성 $\hat{\sigma}$ 로 설정**하고, 그 분포의 0.5% 분위수를 읽어 99.5% VaR를 얻는다.

 **핵심** — 흐름을 한 줄로 잇자. "신경망이 미래 변동성 $\hat{\sigma}$ 를 예측 → 그 $\hat{\sigma}$ 를 폭으로 하는 스튜던트 t 분포를 세움 → 그 분포의 꼬리에서 99.5% VaR를 읽음." 변동성 예측이 정확할수록 분포의 폭이 맞고, 그래야 VaR도 정확해진다.

결과·결론

실험은 S&P 데이터 2008–2020에서, 훈련 2008–2015, 테스트 2016–2020으로 이뤄졌다. 논문의 결론은 세 가지다.

결론 ①: 하이브리드가 전통 모델을 능가한다. LSTM·트랜스포머·멀티-트랜스포머를 GARCH와 결합한 하이브리드가 전통 GARCH와 ANN-GARCH를 모두 앞섰다. 특히 코로나로 변동성이 폭등한 **2020년에 성능 격차가 가장 크게 벌어졌다.**

 **예시** — 평소(잔잔한 시장)에는 전통 GARCH도 그럭저럭 따라가지만, 2020년 3월처럼 변동성이 순식간에 치솟는 국면에서는 격차가 확 벌어졌다. 위기일수록 신경망 하이브리드의 강점이 두드러진 것이다. 위험관리에서 가장 중요한 순간이 바로 이런 위기 국면이라는 점에서 의미가 크다.

결론 ②: 멀티-트랜스포머 > 트랜스포머. 어텐션에 배경을 적용한 멀티-트랜스포머가 일반 트랜스포머보다 나왔다(배경의 분산 감소 효과). 또한 **TL-GARCH는 가중치(파라미터) 수가 더 적으면서도 LSTM-GARCH보다 정확했다.** 이는 "더 간단하면서도 더 정확한 모델"이 가능함을 보여 준다. 그리고 전체에서 **최고 성능은 MTL-GARCH(멀티-트랜스포머 + LSTM)**였다.

 **핵심** — 두 가지를 기억하자. 첫째, **배경(멀티-트랜스포머)은 실제로 예측을 개선한다.** 둘째, 더 적은 파라미터로 더 높은 정확도를 낸 TL-GARCH처럼, "복잡함 = 성능"이 아니라 **잘 설계된 단순함이 이길 수 있다.** 종합 우승은 배경과 LSTM을 모두 품은 MTL-GARCH다.

결론 ③: 위험측정의 적절성. Kupiec·Christoffersen 검정 결과, **MTL-GARCH·TL-GARCH·MT-GARCH만이 2016-2020 전체 기간에서 적절한 위험측정**(두 검정 모두 $p > 0.05$)을 보였다. 나머지 모델은 최소 하나의 검정에서 실패했다.

💡 쉬운 설명 — 두 검정의 $p > 0.05$ 는 "초과 횟수도, 초과가 물리는 정도도 통계적으로 문제없다"는 합격 도장이다. 세 모델(MTL·TL·MT-GARCH)만 이 두 도장을 모두 받았다는 것은, 이들이 평시든 위기든 VaR를 믿고 쓸 만하다는 뜻이다.

마지막으로 실무적 교훈 하나. **드롭아웃(dropout) 정규화가 필수**였다. 드롭아웃 비율 θ 를 $\theta = 0$ (정규화 없음)으로 두면 오차가 최대가 됐다. 즉 학습 중 일부 뉴런을 무작위로 꺼서 과적합을 막는 정규화가 성능에 결정적이었다.

⚠ 주의 — $\theta = 0$ 이면 신경망이 훈련 데이터에 과적합해 테스트 오차가 커진다. 드롭아웃은 "매번 신경망의 일부만 써서 학습"함으로써 특정 뉴런에 대한 의존을 줄이고 일반화를 돕는다. 화려한 구조를 설계해도 정규화를 빠뜨리면 실전에서 무너질 수 있다는 경고다.

끝으로 저자들은 변동성이 **위험관리(VaR)뿐 아니라 파생상품 평가(옵션 가격결정 등)에도 핵심 입력**임을 강조한다. 즉 이 논문의 정확한 변동성 예측 기법은 리스크 관리를 넘어 금융공학 전반으로 확장될 여지를 남긴다.

🎯 핵심 — 논문 ②를 한 문단으로 압축하면 이렇다. "**NLP의 트랜스포머를 변동성용으로 개조하고(시차 기반 위치인코더), 어텐션에 배깅을 넣어 안정화한(멀티-트랜스포머) 뒤, GARCH-LSTM과 결합했더니, 위기 국면에서 특히 강하고 두 백테스트를 모두 통과하는 위험측정을 얻었다. 그 정점이 MTL-GARCH다.**"

코드 완전 해설 ① — Bayer의 backtest.py

이번 장에서는 Bayer가 공개한 `backtest.py` 를 한 줄 한 줄 뜯어봅니다. 이 파일은 놀랍게도 **백테스팅(backtesting)** 전용 클래스 하나, 즉 `Backtest` 클래스만으로 이루어져 있어요. 여기서 백테스팅이란 "우리가 어제 계산해 둔 위험 예측치가 오늘 실제 손익과 견주어 봤을 때 잘 맞았는가?"를 사후에 채점하는 작업입니다.

우리가 채점하려는 대상은 **VaR(Value at Risk, 밸류앳리스크)** 예측치예요. VaR는 "앞으로 하루 동안 최악의 경우 이 정도까지는 잃을 수 있다"는 손실의 경계선(분위수)입니다. 예컨대 1% VaR라면 "100일 중 1일 꼴로만 이 선을 넘는 손실이 나야 정상"이라는 뜻이죠. `Backtest` 클래스는 이 "1일 꼴"이 실제로 지켜졌는지를 세 가지 방식으로 검증합니다. 즉 (1) 초과가 몇 번 났는지 세고, (2) 여러 종류의 **손실함수(loss function)** 로 예측의 품질에 점수를 매기고, (3) Christoffersen과 Engle-Manganelli의 통계 검정으로 "초과가 무작위로 잘 흩어져 있는가"를 판정합니다. 그럼 생성자부터 순서대로 살펴볼게요.

생성자와 기본 메서드

```
class Backtest:
    def __init__(self, actual, forecast, alpha):
        self.index = actual.index      # 시계열 날짜 인덱스 보관
        self.actual = actual.values    # 실제 수익률 (numpy 배열)
        self.forecast = forecast.values # VaR 예측값 (numpy 배열)
        self.alpha = alpha             # 유의수준 (예: 0.01 = 99% VaR)
```

생성자 `__init__` 는 객체를 만들 때 딱 한 번 실행되어, 앞으로 계속 쓸 재료 네 가지를 인스턴스 안에 챙겨 둡니다.

- `self.index` 에는 `actual.index` , 즉 날짜(시계열 인덱스)만 따로 보관합니다. 나중에 그래프의 가로축으로 쓰기 위해서예요.
- `self.actual` 에는 `actual.values` 를 담습니다. `.values` 를 붙이면 pandas 시리즈에서 알맹이 숫자만 뽑아 **넘파이 배열(numpy array)** 로 바꿔 줍니다. 이렇게 하면 뒤에서 벡터 연산을 빠르게 할 수 있어요. 이건 실제로 관측된 수익률입니다.
- `self.forecast` 에는 마찬가지로 VaR 예측값 배열을 담습니다.
- `self.alpha` 에는 **유의수준(significance level)** α 를 담습니다.

여기서 규약을 꼭 짚고 넘어가야 합니다. 논문에서 흔히 "99% 신뢰수준의 VaR"라고 부르는 것을, 이 코드는 `alpha=0.01` 로 표현해요. 즉 신뢰수준 99%와 유의수준 $\alpha = 0.01$ 은 $1 - \alpha$ 관계로 서로 반대편에서 같은 것을 가리킵니다. α 는 "우리가 이 정도 비율만큼은 초과가 나기를 목표로 삼은 확률"입니다.

💡 **쉬운 설명** — α 를 "허용된 실패율"이라고 생각하면 편해요. $\alpha = 0.01$ 이면 "100번 중 1번은 VaR 선을 넘어도 괜찮다"는 약속입니다. 신뢰수준 99%가 곧 $1 - \alpha = 0.99$ 이니, 둘은 같은 이야기의 앞뒷면일 뿐이에요.

⚠️ **주의** — 부호 규약을 헷갈리면 코드 전체가 뒤집힙니다. 이 파일에서 α 는 신뢰수준이 아니라 그 여집합인 목표 초과 확률이에요. 그래서 "99% VaR"를 넣을 때 `alpha` 에는 0.99가 아니라 0.01을 넣어야 합니다.

```
def hit_series(self):
    return (self.actual < self.forecast) * 1
```

`hit_series` 는 이 클래스의 심장입니다. 매일매일 "초과가 났는가, 아닌가"를 0과 1로 기록한 **히트 시리즈(hit series)** 를 만들어 돌려줘요.

`self.actual < self.forecast` 는 넘파이의 벡터 비교라서, 모든 날짜에 대해 한꺼번에 참/거짓(불리언) 배열을 만듭니다. 여기서 실제 수익률이 VaR 예측보다 **아래(actual < forecast)** 라는 것은 곧 "예상했던 최악의 선보다 더 나쁜 손실이 났다", 즉 **초과(exceedance)** 가 발생했다는 뜻이에요. 마지막에 `* 1` 을 곱하면 `True` 는 1로, `False` 는 0으로 바뀝니다. 논문의 식 (7)에 해당하는 지표변수죠. 수식으로 쓰면 다음과 같습니다.

$$I_t = \begin{cases} 1 & \text{만약 } \text{actual}_t < \text{forecast}_t \\ 0 & \text{그 외} \end{cases}$$

여기서 I_t 는 시점 t 의 히트(초과 여부)이고, actual_t 는 그날 실제 수익률, forecast_t 는 그날의 VaR 예측치입니다. VaR는 보통 음수(손실 쪽)이기 때문에, 실제 수익률이 그보다 더 내려가면 $I_t = 1$ 이 됩니다.

 **예시** — 어느 날 VaR 예측이 $\text{forecast}_t = -0.03$ (즉 -3%)이었다고 합시다. 실제 수익률이 $\text{actual}_t = -0.05$ 였다면 $-0.05 < -0.03$ 이 참이므로 $I_t = 1$, 초과입니다. 반대로 $\text{actual}_t = -0.01$ 이었다면 $-0.01 < -0.03$ 은 거짓이라 $I_t = 0$, 안전하게 넘긴 날이에요.

```
def number_of_hits(self):
    return self.hit_series().sum()      # 총 초과 횟수 (식 9)
def hit_rate(self):
    return self.hit_series().mean()     # 초과 비율 = 위반율 (식 10)
def expected_hits(self):
    return self.actual.size * self.alpha # 기대 초과 수 = N * alpha (식 8)
```

이 세 메서드는 히트 시리즈를 요약하는 기본 통계량입니다.

- `number_of_hits` 는 `hit_series().sum()` , 즉 1의 개수를 모두 더해 **총 초과 횟수**를 셉니다. 식 (9)의 $\sum_{t=1}^N I_t$ 예요.
- `hit_rate` 는 `.mean()` 으로 평균을 냅니다. 0과 1로만 이루어진 배열의 평균은 곧 "1이 차지하는 비율"과 같으므로, 이 값이 바로 실제 관측된 초과 비율, 즉 **위반율(violation rate)** 입니다. 식 (10)의 $\hat{\pi} = \frac{1}{N} \sum_{t=1}^N I_t$ 죠.
- `expected_hits` 는 `self.actual.size * self.alpha` 로, 표본 크기 N (전체 날짜 수)에 목표 확률 α 를 곱합니다. "이론적으로라면 초과가 몇 번쯤 나와야 하는가"를 뜻하는 **기대 초과 수** $N \times \alpha$ 이고, 식 (8)에 해당합니다.

 **핵심** — 백테스팅의 첫 관문은 이 두 숫자를 비교하는 것입니다. 실제 초과 수 $\sum_t I_t$ 가 기대치 $N\alpha$ 와 비슷해야 VaR 모형이 "옳은 개수"를 맞힌 것이에요. 예를 들어 $N = 1000$ 이고 $\alpha = 0.01$ 이면 기대 초과 수는 $N\alpha = 10$ 번이고, 실제 위반율 $\hat{\pi}$ 는 0.01 언저리여야 합니다.

```
def duration_series(self):
    hit_series = self.hit_series()
    hit_series[0] = 1
    hit_series[-1] = 1
    return np.diff(np.where(hit_series == 1))[0]
```

`duration_series` 는 초과와 초과 사이가 며칠씩 벌어져 있는지, 그 **간격(duration)** 을 계산합니다. 초과가 여기저기 골고루 흩어져 있으면 간격들이 들쭉날쭉 다양하겠지만, 만약 초과가 특정 시기에 우르르 몰려 있다면 간격이 짧은 값들이 뭉쳐서 나올 거예요. 그러니 이 배열은 "초과들이 서로 독립적으로 흩어져 있는지"를 눈으로 확인하는 진단 도구입니다.

한 줄씩 볼게요. 먼저 `hit_series()` 로 히트 배열을 새로 받아옵니다. 그다음 `hit_series[0] = 1` 과 `hit_series[-1] = 1` 로 배열의 맨 앞과 맨 뒤를 강제로 1로 만들어요. 이걸 "관측 시작·끝을 초과가 난 것처럼 취급"해서 첫 간격과 마지막 간격도 계산에 넣기 위한 처리(경계 보정)입니다. 이어서 `np.where(hit_series == 1)` 은 값이 1인 위치(인덱스)들을 찾아 주고, `np.diff(...)` 는 그 인접한 위치들의 차이를 구합니다. 즉 초과가 난 날들의 간격이죠. 마지막 `[0]` 은 `np.where` 가 튜플 형태로 결과를 돌려주기 때문에 그 안의 실제 배열을 꺼내는 것입니다.

⚠ 주의 — `hit_series[0]` 과 `hit_series[-1]` 에 값을 직접 대입하는 것은 배열을 그 자리에서 바꿔 버리는 동작이에요. 다행히 `self.hit_series()` 가 호출될 때마다 새 배열을 만들어 주므로 원본 데이터가 오염되지는 않습니다. 하지만 이런 식의 자리 바꾸기는 습관적으로 조심해야 합니다.

손실함수 4종

지금까지는 "초과가 났느냐 안 났느냐"만 썼지만, 손실함수는 한 걸음 더 나아가 "얼마나 잘, 혹은 얼마나 아슬아슬하게 맞혔는가"에 연속적인 점수를 매깁니다. 여기 나오는 네 함수는 모두 기본값 `return_mean=True` 를 가지는데, 참이면 날짜별 손실을 평균 내어 하나의 대표 점수로 돌려주고, 거짓이면 날짜별 손실 배열을 통째로 돌려줍니다. 앞으로 편의상 그날의 예측 오차(실제와 예측의 차이)를 $m_t = \text{actual}_t - \text{forecast}_t$ 라 부를게요.

```
def tick_loss(self, return_mean=True):
    loss = (self.alpha - self.hit_series()) * (self.actual - self.forecast)
```

틱 손실(tick loss) 은 분위수 회귀에서 쓰는 그 유명한 핀볼(pinball) 손실입니다. 식 (13)에 해당하고, 수식은 다음과 같아요.

$$L_t^{\text{tick}} = (\alpha - I_t) (\text{actual}_t - \text{forecast}_t)$$

α 는 목표 확률, I_t 는 히트, 뒤의 괄호는 예측 오차 m_t 입니다. 이 손실이 왜 영리한지 볼게요. 초과가 안 난 날($I_t = 0$)에는 계수가 α (작은 양수)이고 보통 $m_t > 0$ (실제가 VaR 위)이라 작은 벌점이 붙습니다. 반대로 초과가 난 날($I_t = 1$)에는 계수가 $\alpha - 1$ (음수)인데 그날은 $m_t < 0$ 이라, 음수 곱하기 음수로 다시 양수가 되고 가중치는 $1 - \alpha$ (큰 값)로 커집니다. 즉 초과를 훨씬 무겁게 벌하죠. 이 손실의 기댓값을 최소로 만드는 예측치가 바로 참된 α -분위수라서, VaR 예측의 정확도를 재는 데 이론적으로 딱 맞습니다.

💡 쉬운 설명 — 틱 손실은 "안전할 때는 살짝, 뚫렸을 때는 크게" 벌점을 주는 저울이에요. α 가 작을수록 초과 한 번의 벌점 가중치 $1 - \alpha$ 가 커져서, 드물어야 할 사건을 놓쳤을 때 더 크게 혼내는 구조입니다.

```
def smooth_loss(self, delta=25, return_mean=True):
    loss = ((self.alpha - (1 + np.exp(delta*(self.actual - self.forecast))))**-1)
           * (self.actual - self.forecast))
```

부드러운 손실(smooth loss)은 방금 본 틱 손실의 매끄러운 버전입니다. 식 (12)에 해당해요. 문제는 히트 I_t 가 m_t 의 부호에 따라 0에서 1로 똑 끊기는 계단함수라 미분이 안 된다는 점입니다. 그래서 이 계단을 **시그모이드(sigmoid)** 함수로 부드럽게 이어 줍니다.

$$L_t^{\text{smooth}} = \left(\alpha - \frac{1}{1 + e^{\delta m_t}} \right) m_t, \quad m_t = \text{actual}_t - \text{forecast}_t$$

코드의 `(1 + np.exp(delta*(...)))**-1`이 바로 $\frac{1}{1 + e^{\delta m_t}}$ 입니다. `** -1`은 역수(즉 $\dots^{-1}$)를 뜻해요. 이 시그모이드는 m_t 가 크게 음수면(강한 초과) 1에 가까워지고, 크게 양수면 0에 가까워져서 히트 I_t 의 역할을 매끄럽게 흉내 냅니다. δ 는 계단의 가파르기를 조절하는 손잡이로, 기본값 `delta=25`는 꽤 날카롭게 세팅한 값이에요.

🔗 핵심 — δ 를 무한대로 키우면 시그모이드가 진짜 계단함수가 되어 부드러운 손실은 틱 손실과 완전히 같아집니다. 즉 부드러운 손실은 "미분 가능해서 최적화·비교에 편리한 틱 손실"이라고 이해하면 됩니다. $\delta = 25$ 는 그 근사를 실용적으로 충분히 가파르게 만든 타협점이에요.

```
def quadratic_loss(self, return_mean=True):
    loss = (self.hit_series() * (1 + (self.actual - self.forecast)**2))
```

이차 손실(quadratic loss)은 식 (11)로, 초과가 난 날에만 집중해서 그 심각도를 재는 손실입니다.

$$L_t^{\text{quad}} = I_t (1 + (\text{actual}_t - \text{forecast}_t)^2) = I_t (1 + m_t^2)$$

앞에 I_t 가 곱해져 있어서 초과가 안 난 날($I_t = 0$)은 손실이 0입니다. 초과가 난 날에는 1에 오차의 제곱 m_t^2 을 더한 값이 벌점이 되죠. 제곱이라 초과 폭이 두 배가 되면 벌점은 대략 네 배로 커집니다. 그래서 이차 손실은 "얼마나 자주 뚫렸나"보다 "뚫렸을 때 얼마나 크게 뚫렸나"를 중시하는 손실이에요.

📌 예시 — 같은 초과라도 오차 $m_t = -0.01$ 일 때는 벌점이 $1 + 0.0001 = 1.0001$ 이지만, $m_t = -0.10$ 일 때는 $1 + 0.01 = 1.01$ 입니다. 큰 붕괴가 났을 때 제공함이 훨씬 크게 반응하는 게 이 손실의 성격이에요.

```
def firm_loss(self, c=1, return_mean=True):
    loss = (self.hit_series() * (1 + (self.actual - self.forecast)**2)
           - c*(1-self.hit_series()) * self.forecast)
```

펌 손실(firm loss)은 식 (14)로, 금융회사(firm)의 현실적 고민을 담은 손실입니다. 앞부분은 방금 본 이차 손실 그대로이고, 여기에 항 하나가 더 붙어요.

$$L_t^{\text{firm}} = I_t (1 + m_t^2) - c(1 - I_t) \text{forecast}_t$$

새로 붙은 $-c(1 - I_t) \text{forecast}_t$ 항을 뜯어보면, $1 - I_t$ 때문에 이 항은 초과가 안 난 날($I_t = 0$)에만 살아납니다. 그런데 VaR 예측치 forecast_t 는 보통 음수이므로 $-\text{forecast}_t$ 는 양수가 되고, 결국 "초과가 안 난 날마다 붙는 양의 비용"이

됩니다. 이걸 VaR를 너무 보수적으로(너무 크게) 잡아 두면 그만큼 자본을 묶어 두어야 하는 **기회비용(opportunity cost)** 을 뜻해요. 계수 c (논문에서는 a)는 이 자본 비용의 무게이고 기본값은 $c=1$ 입니다.

💡 쉬운 설명 — 펴 손실은 회사 입장의 두 가지 고민을 한 저울에 올린 것입니다. 하나는 "뚝리면 위험하다"(이차 손실 부분), 다른 하나는 "너무 안전하게만 잡으면 돈이 놀아서 손해다"(자본 비용 부분). c 를 키우면 자본을 아끼라는 압력이 세지고, 줄이면 안전을 더 중시하게 됩니다.

lr_bt — Christoffersen 우도비 검정

이제 이 파일의 백미인 `lr_bt` 입니다. Christoffersen이 제안한 **우도비 검정(likelihood ratio test)** 으로, 좋은 VaR라면 초과가 (1) 올바른 개수만큼 나와 하고 **동시에** (2) 서로 뭉치지 않고 독립적으로 흩어져 있어야 한다는 두 조건을 함께 검사합니다. 조각별로 따라가 볼게요.

```
def lr_bt(self):
    hits = self.hit_series()
    tr = hits[1:] - hits[:-1]      # 인접한 날의 차이로 '전이(transition)' 탐지
```

먼저 히트 배열을 받아 `hits` 에 담고, `hits[1:] - hits[:-1]` 로 하루씩 어긋나게 뺀 차이 배열 `tr` 을 만듭니다. `hits[1:]` 은 둘째 날부터 끝까지, `hits[:-1]` 은 첫째 날부터 끝에서 둘째까지라서, 둘을 빼면 "오늘 상태 빼기 어제 상태"가 되는 거예요. 이게 상태 사이의 **전이(transition)** 를 잡아냅니다. `tr` 이 +1이면 어제 0(비초과)에서 오늘 1(초과)로 바뀐 것($0 \rightarrow 1$), -1이면 $1 \rightarrow 0$, 0이면 어제와 오늘 상태가 같은 것(유지)입니다.

```
n01, n10 = (tr == 1).sum(), (tr == -1).sum()
n11, n00 = (hits[1:][tr == 0] == 1).sum(), (hits[1:][tr == 0] == 0).sum()
```

이제 네 종류의 전이 횟수를 셉니다. `n01` 은 `tr == 1` 의 개수, 즉 비초과에서 초과로 넘어간 횟수($0 \rightarrow 1$)입니다. `n10` 은 `tr == -1` 의 개수, 초과에서 비초과로 돌아온 횟수($1 \rightarrow 0$)예요. 나머지 둘은 상태가 유지된 날들(`tr == 0`) 중에서 나눕니다. `hits[1:][tr == 0]` 는 "어제와 상태가 같은 날들의 오늘 값"인데, 그중 값이 1이면 `n11` (초과 다음날도 초과, $1 \rightarrow 1$), 0이면 `n00` (비초과 다음날도 비초과, $0 \rightarrow 0$)입니다. 특히 `n11` 은 초과가 연달아 붙는 "뭉침 정도"를 나타내는 핵심 숫자예요.

```
n0, n1 = n01 + n00, n10 + n11    # 상태 0·1에 머문 총 횟수
n = n0 + n1
p01, p11 = n01 / (n00 + n01), n11 / (n11 + n10)  # 조건부 전이확률
p = n1 / n                                     # 전체 초과 확률(추정)
```

여기서 카운트를 확률로 바꿉니다. `n0 = n01 + n00` 은 다음날이 비초과(0)로 끝난 날의 총수, `n1 = n10 + n11` 은 다음날이 초과(1)로 끝난 날의 총수이고, `n = n0 + n1` 은 전이 관측의 총 개수입니다. 그다음 세 확률을 추정해요.

- `p01 = n01 / (n00 + n01)` 은 π_{01} , 즉 "어제 비초과였는데 오늘 초과할 조건부 확률"입니다.
- `p11 = n11 / (n11 + n10)` 은 π_{11} , "어제 초과였는데 오늘도 초과할 조건부 확률"입니다.
- `p = n1 / n` 은 $\hat{p}$, 어제 상태를 따지지 않은 전체 초과율입니다.

만약 초과가 정말로 서로 **독립(independence)** 이라면, 오늘 초과할 확률이 어제 상태에 좌우되지 않아야 합니다. 즉 다음이 성립해야 해요.

$$\pi_{01} \approx \pi_{11} \approx \hat{p}$$

🔴 **핵심** — 독립성 검정의 아이디어가 바로 이 한 줄에 담겨 있습니다. "어제 뚫렸다는 사실"이 "오늘 뚫릴 확률"을 바꾸면 안 된다는 거예요. 만약 π_{11} 이 $\hat{p}$ 보다 눈에 띄게 크다면, 초과가 한번 나면 연달아 나는 뚫침(군집)이 있다는 위험 신호입니다.

```
if n1 > 0:
    uc_h0 = n0*np.log(1-self.alpha) + n1*np.log(self.alpha) # 귀무: 초과율=alpha
    uc_h1 = n0*np.log(1-p) + n1*np.log(p) # 대립: 초과율=관측 p
    uc = -2 * (uc_h0 - uc_h1)
```

`if n1 > 0` 은 초과가 한 번이라도 있어야(로그 계산에서 문제가 안 생기게) 검정을 진행한다는 안전장치입니다. 안쪽에서 먼저 **무조건부 커버리지(unconditional coverage)**, 즉 "초과 개수가 맞느냐"를 검정합니다.

- `uc_h0` 은 귀무가설(초과율이 목표값 α 라는 가정) 아래의 로그우도입니다. 비초과 n_0 번은 확률 $1 - \alpha$ 로, 초과 n_1 번은 확률 α 로 일어났다고 보고 $n_0 \ln(1 - \alpha) + n_1 \ln \alpha$ 을 계산해요.
- `uc_h1` 은 대립가설(초과율이 실제 관측값 $\hat{p}$ 라는 가정) 아래의 로그우도로, $n_0 \ln(1 - \hat{p}) + n_1 \ln \hat{p}$ 입니다.
- `uc` 는 두 로그우도의 차에 -2 를 곱한 우도비 통계량입니다.

$$LR_{uc} = -2 [(n_0 \ln(1 - \alpha) + n_1 \ln \alpha) - (n_0 \ln(1 - \hat{p}) + n_1 \ln \hat{p})]$$

이 값이 크다는 것은 관측된 초과율 $\hat{p}$ 가 목표 α 와 많이 다르다는 뜻이라, VaR가 초과를 너무 많이 또는 너무 적게 냈다는 신호입니다.

```
ind_h0 = (n00+n01)*np.log(1-p) + (n01+n11)*np.log(p) # 귀무: 전이가 p로 동일
ind_h1 = n00*np.log(1-p01) + n01*np.log(p01) + n10*np.log(1-p11)
if p11 > 0:
    ind_h1 += n11 * np.log(p11) # 대립: 전이확률이 상태별로 다
    림
ind = -2 * (ind_h0 - ind_h1)
```

다음은 **독립성(independence)** 검정입니다.

- `ind_h0` 은 귀무가설(어제 상태와 무관하게 전이확률이 모두 같은 $\hat{p}$ 라는 가정) 아래의 로그우도로, 전체를 하나의 확률 $\hat{p}$ 로 설명한 값입니다.
- `ind_h1` 은 대립가설(전이확률이 상태별로 달라 $\pi_{01} \neq \pi_{11}$ 이라는 가정) 아래의 로그우도입니다. $n_{00} \ln(1 - \pi_{01}) + n_{01} \ln \pi_{01} + n_{10} \ln(1 - \pi_{11})$ 을 먼저 계산하고, `if p11 > 0` 일 때만 $n_{11} \ln \pi_{11}$ 을 더합니다. $\pi_{11} = 0$ 이면 $\ln 0$ 이 $-\infty$ 가 되어 버리니 이를 피하는 방어 코드예요.
- `ind` 는 역시 두 로그우도 차에 -2 를 곱한 우도비입니다.

$$LR_{ind} = -2 (\text{ind_h0} - \text{ind_h1})$$

이 값이 크면 상태별 전이확률이 서로 다르다는 것, 즉 초과가 직렬로 종속되어(뭉쳐서) 나타난다는 증거입니다.

```
cc = uc + ind # 조건부 커버리지 = UC + IND (논문 LR_cc = LR_uc + LR_ind)
```

조건부 커버리지(conditional coverage) 통계량 `cc` 는 앞의 둘을 그냥 더한 것입니다. Christoffersen의 아름다운 결과가 바로 이거예요. 논문 식으로 $LR_{cc} = LR_{uc} + LR_{ind}$ 가 성립합니다.

💡 쉬운 설명 — 좋은 VaR가 되려면 두 시험을 모두 통과해야 합니다. 하나는 "초과 개수가 맞는가"(LR_{uc}), 다른 하나는 "초과가 뭉치지 않고 흩어졌는가"(LR_{ind}). 조건부 커버리지 LR_{cc} 는 이 둘을 합쳐 한 번에 채점하는 종합 시험이예요.

```
df = pd.concat([pd.Series([uc, ind, cc]),
                pd.Series([1 - stats.chi2.cdf(uc, 1),
                           1 - stats.chi2.cdf(ind, 1),
                           1 - stats.chi2.cdf(cc, 2)])], axis=1)
```

마지막으로 세 통계량을 **카이제곱 분포(chi-squared distribution)** 를 이용해 p값으로 바꿉니다. 우도비 통계량은 표본이 클 때 근사적으로 카이제곱 분포를 따른다는 점을 쓰는 거예요. `stats.chi2.cdf(통계량, 자유도)` 는 누적분포함수 값이고, 1에서 그걸 빼면 "통계량이 이보다 클 확률", 즉 p값이 됩니다. 자유도는 LR_{uc} 와 LR_{ind} 가 각각 1, LR_{cc} 는 2예요(제약이 두 개라서). `pd.concat(..., axis=1)` 은 통계량 열과 p값 열을 나란히 붙여 표로 만듭니다.

$$p\text{-value} = 1 - F_{\chi_k^2}(\text{통계량}), \quad k \in \{1, 1, 2\}$$

여기서 $F_{\chi_k^2}$ 는 자유도 k 인 카이제곱 분포의 누적분포함수입니다.

🎯 핵심 — 판정 규칙은 간단합니다. **p값이 0.05보다 크면 통과**예요. p값이 크다는 것은 "관측된 결과가 귀무가설 (VaR가 잘 맞는다)과 모순되지 않는다"는 뜻이니깐요. 반대로 p값이 0.05보다 작으면 귀무가설을 기각하고, VaR 모형에 문제가 있다고 봅니다.

dq_bt — Engle-Manganelli 동적 분위수 검정

`dq_bt` 는 Engle과 Manganelli의 **동적 분위수 검정(dynamic quantile test)** 입니다. 아이디어는 이래요. VaR가 완벽하다면 히트 I_t 는 과거의 그 무엇으로도 예측되지 않는 순수한 잡음(평균 α 인 독립 사건)이어야 합니다. 그래서 "히트를 과거 정보로 설명하려는 회귀"를 돌려 보고, 설명력이 정말 0인지를 검정하는 거죠.

```
def dq_bt(self, hit_lags=4, forecast_lags=1):
    hits = self.hit_series()
    p, q, n = hit_lags, forecast_lags, hits.size
    pq = max(p, q - 1)
    y = hits[pq:] - self.alpha # 종속변수: 히트 - alpha (평균0이 이상적)
    x = np.zeros((n - pq, 1 + p + q))
    x[:, 0] = 1 # 상수항(절편)
    for i in range(p): # 과거 히트들을 설명변수로
        x[:, 1 + i] = hits[pq-(i+1):- (i+1)]
    for j in range(q): # 현재+과거 VaR 예측을 설명변수로
        if j > 0:
            x[:, 1 + p + j] = self.forecast[pq-j:-j]
        else:
            x[:, 1 + p + j] = self.forecast[pq:]
```

세팅 부분을 볼게요. `hit_lags=4` 는 과거 히트를 며칠치 넣을지, `forecast_lags=1` 은 VaR 예측을 몇 개 넣을지 정합니다. `pq = max(p, q-1)` 은 과거 값을 참조하느라 앞쪽에서 잘라내야 하는 날짜 수예요.

종속변수 `y` 는 `hits[pq:] - self.alpha` , 즉 히트에서 α 를 뺀 값입니다. 수식으로 $y_t = I_t - \alpha$ 인데, 히트가 평균적으로 α 만큼 나야 정상이므로 잘 맞는 VaR라면 이 y_t 의 평균은 0에 가깝습니다.

설명변수 행렬 `x` 는 `np.zeros` 로 $(n - pq)$ 행, $(1 + p + q)$ 열 크기의 빈 판을 만든 뒤 채웁니다. 첫 열 `x[:, 0] = 1` 은 절편(상수항)이고, 그다음 `p` 개 열에는 하루씩 밀린 과거 히트들 $I_{t-1}, \dots, I_{t-p}$ 을, 마지막 `q` 개 열에는 현재와 과거의 VaR 예측치를 넣습니다. 결국 이런 회귀식을 세우는 거예요.

$$y_t = \beta_0 + \sum_{i=1}^p \beta_i I_{t-i} + \sum_{j=0}^{q-1} \gamma_j \text{forecast}_{t-j} + \varepsilon_t$$

β_0 는 절편, β_i 는 과거 히트의 계수, γ_j 는 VaR 예측의 계수, ε_t 는 오차입니다. 히트가 독립이고 편향 없이 α 를 지킨다면 이 모든 계수가 0이어야 합니다. 즉 과거 히트도, VaR 값 자체도 오늘의 초과를 예측하는 데 아무 쓸모가 없어야 해요.

 **예시** — 만약 과거 히트의 계수 β_1 이 유의하게 양수로 나온다면, "어제 뚫렸으면 오늘도 뚫릴 가능성이 높다"는 뜻이라 초과가 뭉치고 있다는 증거입니다. 마찬가지로 γ_0 가 0이 아니면 "VaR를 크게 잡은 날일수록 초과가 더/덜 난다"는 체계적 편향이 있는 거예요.

```
beta = np.dot(np.linalg.inv(np.dot(x.T, x)), np.dot(x.T, y)) # (X^T X)^-1 X^T y
lr_dq = np.dot(beta, np.dot(np.dot(x.T, x), beta)) / (self.alpha*(1-self.alpha))
p_dq = 1 - stats.chi2.cdf(lr_dq, 1+p+q)
```

이제 계수를 실제로 풀고 검정합니다. 첫 줄은 **선형회귀(linear regression)의 정규방정식(normal equation)** 그대로예요. `np.linalg.inv(x.T @ x)` 가 $(X^T X)^{-1}$, 여기에 $X^T y$ 를 곱해 계수 추정치를 얻습니다.

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

X 는 설명변수 행렬, X^T 는 그 전치(transpose), y 는 종속변수 벡터입니다. 그다음 DQ 통계량을 계산해요.

$$DQ = \frac{\hat{\beta}^T (X^T X) \hat{\beta}}{\alpha(1 - \alpha)}$$

분자 $\hat{\beta}^T (X^T X) \hat{\beta}$ 는 추정된 계수들이 전체적으로 0에서 얼마나 멀리 떨어졌는지를 재는 이차형식이고, 분모 $\alpha(1 - \alpha)$ 는 히트가 성공확률 α 인 베르누이 사건일 때의 분산이라 통계량을 표준화해 줍니다. 계수가 모두 0에 가까우면 분자가 작아져 DQ도 작아지죠. 마지막으로 이 DQ를 자유도 $1 + p + q$ 인 카이제곱 분포로 검정해 p값 `p_dq`를 얻습니다.

$$\text{p-value} = 1 - F_{\chi^2_{1+p+q}}(DQ)$$

 **핵심** — 여기서도 판정은 같습니다. **p값이 0.05보다 크면 독립(통과)** 이예요. 계수 전부가 0과 다를 바 없다는 뜻이니, 과거 정보로 초과를 예측할 수 없다는 좋은 결론입니다. 자유도가 $1 + p + q$ 인 이유는 검정하는 계수가 절편 1개, 히트 시차 p 개, 예측 시차 q 개로 모두 $1 + p + q$ 개이기 때문이에요.

⚠ **주의** — $X^T X$ 가 역행렬을 갖지 못하는 **특이행렬(singular matrix)** 인 경우 `np.linalg.inv` 가 실패합니다. 그래서 실제 코드는 이 계산을 `try/except` 로 감싸, 역행렬을 못 구하면 통계량 대신 NaN(결측)을 돌려주도록 방어해 둡니다.

실행 예시

```
if __name__ == "__main__":
    Y = pd.read_csv("input/returns.txt", ...) # IBM 일별 로그수익률
    X = pd.read_csv("input/quantile_predicitions.txt", ...) # 여러 모델의 1% VaR 예측
    bt = Backtest(actual=Y, forecast=X.loc[:, "eGARCH-fhs"], alpha=0.01)
    bt.plot()
```

마지막은 이 파일을 직접 실행했을 때 돌아가는 예시입니다. `if __name__ == "__main__":` 는 "이 파일을 (다른 데서 불러오는 게 아니라) 직접 실행할 때만" 아래 블록을 돌리라는 파이썬의 관용구예요.

- `Y` 에는 `returns.txt` 에서 IBM 주식의 일별 로그수익률을 읽어 옵니다.
- `X` 에는 `quantile_predicitions.txt` 에서 여러 모델이 각각 내놓은 1% VaR 예측치들을 읽어 옵니다.
- `Backtest(...)` 로 백테스트 객체를 만드는데, `actual` 에는 실제 수익률 `Y` 를, `forecast` 에는 그 여러 모델 중 `eGARCH-fhs` 열 하나를 골라 넣고, `alpha=0.01` 로 1% VaR임을 지정합니다.
- `bt.plot()` 으로 결과를 그림으로 그립니다.

정리하면, 이 예시는 IBM 주식 수익률에 대해 **eGARCH-fhs** 모형이 내놓은 1% VaR 예측을 백테스트하는 장면입니다. 지금까지 뜯어본 히트 시리즈, 손실함수 네 종, Christoffersen 검정, Engle-Manganelli 검정이 모두 이 한 객체 안에서 함께 작동하게 되는 거예요.

💡 **쉬운 설명** — `eGARCH-fhs` 는 변동성을 다루는 eGARCH 모형에 과거 오차 분포를 재활용하는 여과 과거 시뮬레이션(filtered historical simulation, FHS) 기법을 엮은 조합입니다. 이 예시의 핵심은 "어떤 VaR 모형이든 이 `Backtest` 클래스에 넣기만 하면 똑같은 잣대로 채점된다"는 점이에요. 모형 이름만 바꿔 끼우면 여러 모형을 공정하게 비교할 수 있습니다.

코드 완전 해설 ② — DeepVaR 노트북

앞에서 배운 DeepVaR의 이론을, 이제 실제 코드로 한 줄씩 뜯어볼 차례예요. 이 노트북의 큰 흐름은 딱 하나로 요약됩니다. GluonTS라는 라이브러리로 **확률예측(probabilistic forecasting)**을 하고, 그 예측 분포에서 **분위수(quantile)**를 뽑아 **VaR(Value at Risk)**로 바꾸는 파이프라인이에요. 즉 "미래 수익률이 이런 분포로 나올 것 같다"를 신경망이 먼저 그려 주면, 우리는 그 분포의 꼬리를 잘라 위험을 읽어 냅니다.

💡 **쉬운 설명** — 보통의 예측 모델은 "내일 수익률은 -0.3%"처럼 숫자 하나만 내놓아요. 하지만 DeepVaR는 "내일 수익률의 분포는 이렇게 생겼다"를 통째로 내놓습니다. 분포가 있으면 그 아래쪽 1% 지점을 잘라 "99% 확신하는 최악의 손실"을 바로 읽을 수 있죠. 그게 VaR입니다.

데이터 준비(셀 0~7)

먼저 필요한 도구들을 불러옵니다.

```
from gluonts.dataset.common import ListDataset
from gluonts.model import deepar
from gluonts.mx.trainer import Trainer
from gluonts.evaluation import Evaluator
```

한 줄씩 볼게요. `ListDataset` 은 GluonTS가 요구하는 **전용 데이터 형식**이에요. 우리가 가진 판다스 표를 이 그릇에 담아야 모델이 먹을 수 있습니다. `deepar` 는 이 노트북의 심장인 **DeepAR** 모델 모듈, `Trainer` 는 학습을 돌리는 엔진(에폭 수·학습률 같은 걸 쥐고 있음), `Evaluator` 는 예측이 얼마나 맞았는지 채점하는 평가기예요.

```
paths = ["data/AUDUSD1440.csv", "data/GBPUSD1440.csv",
         "data/USDJPY1440.csv", "data/EURUSD1440.csv"]
```

논문이 다룬 **4개 통화쌍(currency pair)**의 데이터 경로예요. 호주달러/미국달러(AUDUSD), 영국파운드/미국달러(GBPUSD), 미국달러/일본엔(USDJPY), 유로/미국달러(EURUSD)입니다. 파일명 뒤의 **1440** 은 분 단위 캔들 주기(하루 = 1440분), 즉 **일봉 데이터**라는 뜻이에요.

```
def read_asset_data(path):
    asset = pd.read_csv(path, delimiter='\t', usecols=[0,4], names=['datetime', 'price'])
    asset.datetime = pd.to_datetime(asset.datetime) # 문자열→날짜
    asset.set_index('datetime', inplace=True) # 날짜를 인덱스로
    asset_name = "".join(re.findall("[a-zA-Z]+", path))[-9:-3] # 파일명에서 통화쌍 이름
    asset.rename(columns={"price": asset_name}, inplace=True)
    return asset[[asset_name]]
```

이 함수는 CSV 하나를 깔끔한 표로 읽어 주는 도우미예요. 한 줄씩 따라가 볼게요.

- `pd.read_csv(..., delimiter='\t', usecols=[0,4], names=['datetime', 'price'])` — 탭(`\t`)으로 나뉜 파일에서 **0번 열(시간)**과 **4번 열(종가)**만 골라 읽고, 각각 `datetime` · `price` 라는 이름을 붙입니다. 나머지 시가·고가·저가 열은 버려요.

- `pd.to_datetime(asset.datetime)` — 문자열로 된 날짜("2015.01.02" 같은 것)를 파이썬이 진짜 **날짜 자료형**으로 이해하도록 변환합니다.
- `set_index('datetime', inplace=True)` — 그 날짜를 표의 **인덱스(행 이름)**로 삼아요. 이러면 시계열로 다루기 편해집니다.
- `re.findall("[a-zA-Z]+", path)` — 경로 문자열에서 **알파벳만** 골라냅니다. 여기에 `[-9:-3]` 슬라이싱을 걸어 "AUDUSD" 같은 여섯 글자 통화쌍 이름만 뽑아내죠.
- `rename(...)` 후 `return asset[[asset_name]]` — `price` 라는 멋진 열 이름을 실제 통화쌍 이름으로 바꿔서, 이 열 하나만 돌려줍니다.

 **예시** — "data/EURUSD1440.csv" 가 들어오면, 알파벳만 모아 "dataEURUSDcsv" 가 되고 여기서 `[-9:-3]` 을 자르면 정확히 "EURUSD" 가 남아요. 그래서 반환되는 표의 열 이름이 `EURUSD` 가 됩니다.

이렇게 4개 파일을 읽어 옆으로 붙이면 `prices` 라는, 날짜가 인덱스이고 4개 통화쌍이 열인 가격표가 완성돼요. 다음은 이 가격을 수익률로 바꾸는 단계입니다.

```
returns = prices.pct_change().dropna()          # 일별 수익률
returns = returns.asfreq(freq='1D', fill_value=0.0) # DeepAR는 '일정 빈도' 요구
returns = returns[returns.index >= '2015-01-01']  # 최근 6년만
```

- `prices.pct_change()` — 어제 대비 오늘의 **변화율**, 즉 일별 수익률을 계산합니다. 수식으로는 $r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$ 이예요. 여기서 P_t 는 t 일의 종가, P_{t-1} 은 그 전날 종가입니다. 맨 첫날은 비교할 전날이 없어 `NaN` 이 되므로 `dropna()` 로 지웁니다.
- `asfreq(freq='1D', fill_value=0.0)` — 여기가 핵심이에요. **DeepAR**는 시계열이 **일정한 시간 간격 (regular frequency)**으로 놓여 있길 요구합니다. 그런데 외환 데이터는 주말·공휴일에 구멍이 나 있죠. 그래서 `'1D'` (하루 간격)로 격자를 강제하고, 거래가 없어 빈 날은 수익률 0으로 메꿉니다.
- `returns.index >= '2015-01-01'` — 논문 설정에 맞춰 **2015년 이후 최근 약 6년치**만 남깁니다.

 **주의** — 빈 날을 0으로 채우는 건 "그날은 가격이 그대로였다"는 가정이에요. 편의를 위한 처리지만, 휴장일이 많으면 변동성을 실제보다 살짝 낮게 보이게 할 수 있습니다. DeepAR의 "일정 간격" 요구를 맞추려는 타협이라는 점을 기억해 두세요.

DeepARModel 클래스(셀 8)

이제 DeepAR 모델을 쓰기 편하게 감싼 **래퍼 클래스(wrapper class)**를 봅시다. 데이터 변환·학습·예측을 한 묶음으로 정리해 둔 상자예요.

```
class DeepARModel:
    def __init__(self, freq='1D', context_length=15, prediction_length=10,
                  epochs=5, learning_rate=1e-4, n_layers=2., dropout=0.1):
        # 논문 설정과 일치: 15일 컨텍스트, lr=0.0001, 2층, 드롭아웃 10%
```

생성자 `__init__` 은 모델의 **하이퍼파라미터(hyperparameter)**를 정합니다. 하나씩 볼게요.

- `context_length=15` — 예측할 때 **과거 며칠을 참고**할지. 여기서 15일치를 봅니다.
- `prediction_length=10` — 한 번에 **며칠 앞을 예측**할지. 기본은 10일이에요.
- `epochs=5` — 전체 데이터를 몇 번 반복 학습할지.
- `learning_rate=1e-4` — 학습 보폭, 즉 0.0001. 한 번에 얼마나 크게 가중치를 고칠지를 정합니다.
- `n_layers=2`. — 순환신경망을 **2층** 쌓습니다.
- `dropout=0.1` — 학습 중 뉴런의 10%를 무작위로 끄는 **드롭아웃(dropout)** 비율. 과적합을 막는 장치예요.

주석에도 적혀 있듯 이 값들은 모두 **논문 설정과 일치**합니다.

```
def list_dataset(self, ts, train=True):
    custom_dataset = self.df_to_np(ts) # (시간×자산)→(자산×시간) 전치
    start = pd.Timestamp(ts.index[0], freq=self.freq)
    if train == True:
        ds = ListDataset([{'target': x, 'start': start}
                           for x in custom_dataset[:, :-self.prediction_length]],
                           freq=self.freq)
    else:
        ds = ListDataset([{'target': x, 'start': start}
                           for x in custom_dataset], freq=self.freq)
    return ds
```

이 메서드는 판다스 표를 GluonTS의 `ListDataset` 으로 번역합니다. 차근차근 볼게요.

- `self.df_to_np(ts)` — 표를 넘파이 배열로 바꾸면서 **(시간 × 자산)** 모양을 **(자산 × 시간)**으로 **전치(transpose)**합니다. GluonTS는 "한 행 = 하나의 시계열"을 원하기 때문에, 각 통화쌍이 하나의 행이 되도록 놓히는 거예요.
- `start = pd.Timestamp(ts.index[0], ...)` — 시계열의 **시작 날짜**를 잡아 둡니다. GluonTS는 각 시계열에 "언제부터 시작인지"를 함께 알려 줘야 하거든요.
- `if train == True:` 분기가 중요합니다. **학습용**이면 `custom_dataset[:, :-self.prediction_length]` 로 **마지막 prediction_length 일을 잘라냅니다**. 잘라낸 그 뒷부분이 나중에 채점할 **정답(미래)**으로 남죠. 반대로 **테스트용**(`else`)이면 자르지 않고 **전체**를 넣습니다.
- 각 원소 `{'target': x, 'start': start}` — `x` 가 한 자산의 수익률 시계열(`target`), `start` 가 시작일이에요. 즉 **자산 하나 = 시계열 하나**로 취급합니다.

 **쉬운 설명** — 학습용에서 뒷부분을 잘라 두는 건 시험 공부에 비유할 수 있어요. 뒤 10일을 가려 놓고 앞부분만 보여 주며 "가려진 뒤를 맞춰 봐"라고 시키는 거죠. 그래야 모델이 진짜 미래를 예측하는 연습을 하게 됩니다.

```
def fit(self, ts):
    estimator = deepar.DeepAREstimator(
        prediction_length=self.prediction_length,
        context_length=self.context_length,
        freq=self.freq,
        trainer=Trainer(epochs=self.epochs, ctx="cpu",
                        learning_rate=self.learning_rate, num_batches_per_epoch=50),
        num_layers=self.n_layers,
        dropout_rate=self.dropout,
        cell_type='lstm',          # LSTM 셀
        num_cells=50               # 논문대로 층당 50셀(기본 40에서 상향)
    )
    list_ds = self.list_dataset(ts, train=True)
    predictor = estimator.train(list_ds)  # 학습 실행(내부에서 로그우도 최대화, Adam)
    return predictor
```

`fit` 은 실제로 모델을 **학습**시키는 메서드예요.

- `deepar.DeepAREstimator(...)` — DeepAR 모델의 설계도를 만듭니다. 앞서 정한 예측 길이·컨텍스트·빈도가 그대로 들어가고, `Trainer(...)` 로 학습 엔진을 붙여요. `ctx="cpu"` 는 CPU로 돌리라는 뜻, `num_batches_per_epoch=50` 은 한 에폭에 미니배치 50개를 처리하라는 지시입니다.
- `cell_type='lstm'` — 순환 셀로 **LSTM**을 씁니다. 앞 레슨에서 배운 그 장·단기 기억 셀이예요.
- `num_cells=50` — 각 층에 뉴런(셀)을 50개 둡니다. GluonTS 기본값 40에서 **논문대로 50으로 올린** 부분이에요.
- `estimator.train(list_ds)` — 학습을 실행합니다. 이 한 줄 안에서 DeepAR는 내부적으로 **로그우도(log-likelihood)**를 **최대화**하는 방향으로, **Adam** 최적화기를 써서 가중치를 조정해요.

🎯 **핵심** — `cell_type='lstm'` 과 `num_cells=50` , 이 두 설정이 논문 재현의 핵심 포인트예요. DeepAR는 단순히 "다음 값"이 아니라 **다음 값의 확률분포(스튜던트 t 분포)의 파라미터**를 예측하도록 학습됩니다. 그래서 로그우도를 최대화한다는 말은 "실제 관측이 나올 확률을 가장 높게 만드는 분포를 찾는다"는 뜻이에요.

```
def predict(self, ts):
    test_ds = self.list_dataset(ts, train=False)
    return self.estimator.predict(test_ds, num_samples=1000)  # 몬테카를로 샘플 1000개
```

`predict` 는 학습된 모델로 미래를 예측합니다. 여기서 `num_samples=1000` 이 결정적이에요. DeepAR는 예측을 **숫자 하나가 아니라 1000개의 표본**으로 뱉습니다. 이 1000개의 **몬테카를로(Monte Carlo) 샘플**을 히스토그램으로 그리면 그게 바로 미래 수익률의 **예측 분포**예요. 우리는 이 분포에서 분위수를 취해 VaR을 만들 겁니다.

📝 **예시** — 내일 EURUSD 수익률을 1000번 시뮬레이션했더니 대부분 -1%에서 +1% 사이에 몰렸다고 해 봐요. 이 1000개를 작은 순서로 줄 세워 아래에서 10번째(1% 지점) 값을 읽으면, 그게 99% 신뢰수준 VaR의 재료가 됩니다.

var_p — 포트폴리오 VaR(셀 11)

이제 종목별 예측 분포를 **포트폴리오 전체의 VaR** 하나로 합치는 함수입니다.

```
def var_p(predictions, returns, weights, days_ahead=0, alpha=95):
    V = np.zeros(len(weights))
    for i in range(len(weights)):
        if weights[i] < 0 : # 숏 → 높은 수익률(alpha 퍼센타일)이 손실
            V[i] = weights[i] * np.percentile(predictions[i].samples[:,days_ahead], alpha)
        else: # 롱 → 낮은 수익률(100-alpha 퍼센타일)이 손실
            V[i] = weights[i] * np.percentile(predictions[i].samples[:,days_ahead], 100-alpha)
    R = returns.corr() # 전체 기간 상관행렬(논문 125일 롤링과 어긋남)
    return -np.sqrt(V @ R @ V.T) # 논문 식 15:  $-\sqrt{VRV^T}$ 
```

핵심은 두 단계예요. 먼저 종목별 위험 벡터 V 를 만들고, 그다음 상관행렬로 하나로 묶습니다.

반복문 안에서 각 종목 i 의 위험 V_i 를 계산하는데, 포지션 방향에 따라 손실이 나는 꼬리가 반대라는 점이 포인트입니다.

- `if weights[i] < 0` (숏 포지션) — 가격이 오르면 손해죠. 그래서 수익률 분포의 위쪽 꼬리, 즉 `np.percentile(..., alpha)` (예: 95 퍼센타일)의 높은 수익률이 손실 시나리오가 됩니다.
- `else` (롱 포지션) — 가격이 내리면 손해예요. 그래서 아래쪽 꼬리 `np.percentile(..., 100-alpha)` (예: 5 퍼센타일)의 낮은 수익률이 손실이 됩니다.
- 어느 쪽이든 그 분위수에 비중 `weights[i]`를 곱해 V_i 에 저장해요. `predictions[i].samples[:, days_ahead]`가 바로 앞서 만든 1000개 몬테카를로 샘플에서 `days_ahead` 일째 값들을 꺼낸 것입니다.

그리고 마지막 두 줄에서 이들을 결합합니다. 수식으로는 논문 식 15예요.

$$\text{VaR}_p = -\sqrt{VRV^T}$$

기호를 풀어 볼게요.

- V — 방금 만든 종목별 가중 VaR 벡터. 각 성분이 V_i 입니다.
- V^T — 그 벡터의 전치(transpose).
- R — 종목 간 상관행렬(correlation matrix), 코드의 `returns.corr()` 예요.
- $-\sqrt{\cdot}$ — 상관을 반영해 결합한 위험의 제곱근을 씌우고, 손실을 음수 부호로 표현합니다.

⚠ 주의 — 여기 숨은 한계가 하나 있어요. `returns.corr()`는 넘겨받은 전체 기간의 단일 상관행렬을 씁니다. 그런데 논문은 원래 125일 롤링(rolling) 상관을 쓰도록 되어 있어요. 상관관계는 시장 국면에 따라 계속 변하는데, 전체 기간 하나의 값으로 고정하면 위기 때 급변하는 상관을 놓칠 수 있습니다.

🎯 핵심 — 종목별 VaR을 그냥 더하면 안 되는 이유가 바로 이 식에 담겨 있어요. 종목들이 완벽히 같이 움직이지 않으므로(R 의 비대각 성분이 1보다 작음), 상관행렬을 끼운 $-\sqrt{VRV^T}$ 가 단순 합보다 작은(=현실적인) 위험을 줍니다. 이게 분산 효과(diversification effect)예요.

벤치마크 VaRCalculation(셀 13)

DeepVaR가 잘하는지 비교하려면 전통적 VaR 방법들과 겨뤄 봐야겠죠. 이 클래스는 교과서에 나오는 세 가지 고전 기법을 담고 있어요.

```
class VaRCalculation:
    def mc(self, x, alpha, n_sims=5000, seed=42):      # 몬테카를로 VaR
        np.random.seed(seed)
        sim_returns = np.random.normal(x.mean(), x.std(), n_sims) # 정규분포 시뮬
        return np.sqrt(self.time*self.freq) * np.percentile(sim_returns, alpha)
    def vc(self, x, alpha):                            # 분산-공분산 VaR
        c = alpha / 100
        return np.sqrt(self.time*self.freq) * (x.std() * stats.norm.ppf(c)) #  $\sigma \times z$ 
    def hs(self, x, alpha):                            # 역사적 시뮬레이션 VaR
        return np.sqrt(self.time*self.freq) * np.percentile(x, 100 - alpha) # 경험 분위수
```

세 메서드를 하나씩 볼게요.

- **mc** — 몬테카를로(Monte Carlo) VaR. 실제 데이터의 평균 `x.mean()` 과 표준편차 `x.std()` 를 가진 정규분포에서 5000개를 뽑아(시뮬레이션) 그중 분위수를 취합니다. `np.random.seed(seed)` 는 매번 같은 난수가 나오게 고정해 재현성을 보장해요.
- **vc** — 분산-공분산(variance-covariance) VaR. 시뮬레이션 없이 공식 하나로 끝냅니다. `c = alpha/100` 로 신뢰수준을 확률로 바꾸고, 표준편차에 정규분포 분위수 `stats.norm.ppf(c)` 를 곱해요. 이게 논문 식 10자, 딱 $\sigma \times z$ 꼴입니다.
- **hs** — 역사적 시뮬레이션(historical simulation) VaR. 분포를 가정하지 않고 실제 수익률 데이터를 그대로 줄 세워 `np.percentile(x, 100-alpha)` , 즉 경험 분위수(empirical quantile)를 읽습니다. 가장 비모수적(nonparametric)인 방법이에요.

vc 의 공식을 수식으로 쓰면 이렇습니다.

$$\text{VaR}_{vc} = \sqrt{t \cdot f} \cdot \sigma \cdot \Phi^{-1}(c)$$

기호를 풀면, σ 는 수익률의 표준편차(변동성), $\Phi^{-1}(c)$ 는 신뢰수준 c 에 해당하는 표준정규분포의 분위수(코드의 `stats.norm.ppf(c)` , 흔히 z -값), $t \cdot f$ 는 시간 관련 배수예요.

💡 쉬운 설명 — 세 방법의 차이를 한 줄로 정리하면 이래요. **MC**는 "정규분포라 치고 무수히 굴러 본다", **VC**는 "정규분포라 치고 공식으로 바로 계산한다", **HS**는 "가정 없이 실제 과거 데이터에서 그냥 꺼내 쓴다"입니다. 앞 둘은 모수적(정규 가정), 마지막은 비모수적이에요.

세 방법 모두 앞에 `np.sqrt(self.time*self.freq)` 가 붙어 있죠. 이건 시간지평 스케일링(time-horizon scaling), 이른바 $\sqrt{t}$ 법칙입니다.

$$\text{VaR}(t\text{일}) = \sqrt{t} \cdot \text{VaR}(1\text{일})$$

즉 하루짜리 VaR을 t 일짜리로 늘릴 때 t 배가 아니라 $\sqrt{t}$ 배를 곱해요. 수익률이 서로 독립이면 분산은 시간에 비례하고, 표준편차(변동성)는 그 제곱근에 비례하기 때문입니다.

📝 예시 — 하루 VaR이 -2%인데 10일 지평으로 늘리고 싶다면, 10배(-20%)가 아니라 $\sqrt{10} \approx 3.16$ 배를 곱해 약 -6.3%가 됩니다. 손실이 시간에 정비례해 쌓이지는 않는다는 뜻이에요.

롤링 윈도우 + 연속 학습(셀 14)

마지막으로 이 모든 조각을 엮어 **하루씩 앞으로 굴리며** DeepVaR와 전통 기법을 비교하는 백테스트 루프예요.

```
for i in range(10):
    estimator = DeepARModel(prediction_length=1, context_length=15,
epochs=5).fit(returns.iloc[:-10])
    predictions = list(estimator.predict(DeepARModel().list_dataset(returns.iloc[:-10+i],
train=False), 1000))
    deepvar95.append(var_p(predictions, returns.iloc[-260+i:-10+i], w, 0, alpha=95))
    var99 = VaRCalculation(time=1, alpha=99).predict(returns.iloc[-260+i:-10+i], w)
```

10일에 걸쳐 하루씩 반복하는 루프입니다. 반복 변수 `i` 가 커질수록 관찰 창이 하루씩 미끄러져요.

- `DeepARModel(prediction_length=1, ...).fit(returns.iloc[:-10])` — 매 반복마다 **모델을 새로 학습**합니다. `prediction_length=1` 이니 이번엔 **딱 하루 앞만** 예측하도록 맞췄어요. 이렇게 매번 최신 데이터로 다시 배우는 방식을 **연속 학습(continuous learning)**이라 부릅니다.
- `estimator.predict(... returns.iloc[:-10+i] ..., 1000)` — 학습된 모델로 1000개 몬테카를로 샘플을 뽑아 예측 분포를 만듭니다. `iloc[:-10+i]` 라 `i` 가 커질수록 끝점이 하루씩 뒤로 밀려요.
- `var_p(predictions, returns.iloc[-260+i:-10+i], w, 0, alpha=95)` — 앞서 본 `var_p` 로 95% DeepVaR을 계산해 리스트에 쌓습니다. 여기서 상관행렬 재료로 `iloc[-260+i:-10+i]`, 즉 최근 약 250일 창을 잘라 씁니다. 창이 `i` 와 함께 이동하니, 이 부분만큼은 **롤링 상관**의 정신에 훨씬 가까워요.
- `VaRCalculation(time=1, alpha=99).predict(...)` — 같은 데이터 창으로 전통 기법(VC·HS)의 99% VaR도 계산합니다. 이렇게 **DeepVaR vs 전통 기법**을 같은 날짜에서 나란히 비교하는 거예요.

🎯 **핵심** — 이 루프의 두 얼굴을 기억하세요. 하나는 **연속 학습**(매 스텝 재학습으로 최신 시장에 적응), 다른 하나는 **롤링 윈도우**(상관 계산 창 `iloc[-260+i:-10+i]` 을 하루씩 이동)예요. 다만 앞서 본 `var_p` 내부의 `returns.corr()` 는 여전히 넘겨받은 창 **전체**의 단일 상관을 쓴다는 점에서, 완전한 125일 롤링과는 미묘하게 어긋난다는 걸 함께 알아 두면 좋습니다.

코드 완전 해설 ③ — Multi-Transformer 노트북 (3_MT_GARCH.ipynb)

이제 이 프로젝트에서 **가장 흥미로운 코드**로 넘어갑니다. 앞 레슨에서 개념으로만 배운 **멀티-트랜스포머(Multi-Transformer)**, 그 핵심인 **AMH(Attention averaged Multi-Head, 논문 식 20-21)**가 실제 파이썬으로 어떻게 구현되는지 두 눈으로 확인할 거예요. 결론부터 말하면, "여러 어텐션을 평균 낸다"는 그 웅장한 아이디어가 **for** 반복문 몇 줄로 깔끔하게 표현됩니다. 그 대비가 이 노트북의 백미예요.

MultiHeadSelfAttention 클래스 — 표준 어텐션 한 단위

먼저 멀티-트랜스포머의 **부품인**, 표준 트랜스포머의 어텐션 한 단위부터 봅시다.

```
class MultiHeadSelfAttention(tf.keras.layers.Layer):
    def __init__(self, embed_dim, num_heads=8):
        super().__init__()
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        if embed_dim % num_heads != 0:
            raise ValueError("임베딩 차원은 헤드 수로 나누어떨어져야 함")
        self.projection_dim = embed_dim // num_heads # 헤드당 차원
        self.query_dense = tf.keras.layers.Dense(embed_dim) # Q 사영
        self.key_dense = tf.keras.layers.Dense(embed_dim) # K 사영
        self.value_dense = tf.keras.layers.Dense(embed_dim) # V 사영
        self.combine_heads = tf.keras.layers.Dense(embed_dim) # W^O
```

생성자에서 부품들을 준비합니다.

- `embed_dim` — 각 시점을 표현하는 **임베딩 차원(embedding dimension)**, 즉 벡터 하나의 길이예요.
- `num_heads=8` — **헤드(head)** 개수, 즉 어텐션을 몇 갈래로 병렬 계산할지.
- `if embed_dim % num_heads != 0:` — 임베딩 차원이 헤드 수로 **나누어떨어지지 않으면** 에러를 냅니다. 차원을 헤드별로 정확히 쪼개야 하니까요.
- `self.projection_dim = embed_dim // num_heads` — 그래서 **헤드 하나가 맡는 차원**은 전체를 헤드 수로 나눈 값입니다.
- `query_dense`, `key_dense`, `value_dense` — 입력을 각각 **쿼리 Q·키 K·값 V**로 사영하는 **밀집층(dense layer)** 세 개예요.
- `combine_heads` — 여러 헤드 결과를 합친 뒤 한 번 더 사영하는 층으로, 수식의 출력 가중치 W^O 에 해당합니다.

```
def attention(self, query, key, value):
    score = tf.matmul(query, key, transpose_b=True) #  $QK^T$  (내적=유사도)
    dim_key = tf.cast(tf.shape(key)[-1], tf.float32) #  $d_k$ 
    scaled_score = score / tf.math.sqrt(dim_key) #  $\div \sqrt{d_k}$ 
    weights = tf.nn.softmax(scaled_score, axis=-1) # softmax → 확률
    output = tf.matmul(weights, value) # 확률 · V
    return output, weights
```

이 메서드가 바로 앞 레슨에서 배운 **스케일드 닷-프로덕트 어텐션** 공식의 코드 구현이에요.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

각 줄을 수식과 하나하나 맞춰 볼게요.

- `score = tf.matmul(query, key, transpose_b=True)` → QK^T . 쿼리와 키의 **내적(dot product)**으로 둘이 얼마나 닮았는지, 즉 **유사도**를 잹니다.
- `dim_key = ... tf.shape(key)[-1]` → d_k . 키의 마지막 차원, 곧 **키 벡터의 차원**입니다.
- `scaled_score = score / tf.math.sqrt(dim_key)` → $\div \sqrt{d_k}$. 차원이 크면 내적이 과도하게 커져 softmax가 한쪽으로 쏠리므로, 제곱근으로 나눠 **스케일링**합니다.
- `weights = tf.nn.softmax(scaled_score, axis=-1)` → $\text{softmax}(\cdot)$. 점수들을 합이 1인 **확률**로 바꿉니다.
- `output = tf.matmul(weights, value)` → $(\cdot)V$. 그 확률로 값 V 의 **가중합**을 냅니다. 이게 어텐션의 최종 출력이에요.

💡 **쉬운 설명** — 어렵게 느껴지면 딱 네 동작으로 외우세요. 닮은 정도 재기(QK^T) → 크기 다듬기($\div \sqrt{d_k}$) → 확률로 바꾸기(softmax) → 그 확률로 값 섞기($\times V$). 이 네 줄이 트랜스포머의 심장입니다.

```
def separate_heads(self, x, batch_size):
    x = tf.reshape(x, (batch_size, -1, self.num_heads, self.projection_dim))
    return tf.transpose(x, perm=[0, 2, 1, 3])
```

`separate_heads` 는 하나로 뭉친 큰 벡터를 **헤드 수만큼 쪼개는** 도우미예요. `tf.reshape` 로 전체 차원을 `(batch, seq, num_heads, projection_dim)` 으로 나눈 뒤, `tf.transpose` 로 축 순서를 바꿔 `(batch, num_heads, seq, projection_dim)` 으로 만듭니다. 이렇게 헤드 축을 앞으로 빼 두면, 각 헤드가 **서로 독립적으로 병렬 어텐션**을 계산할 수 있어요.

```
def call(self, inputs):
    batch_size = tf.shape(inputs)[0]
    query = self.query_dense(inputs); key = self.key_dense(inputs); value = self.value_dense(inputs)
    query = self.separate_heads(query, batch_size)
    key = self.separate_heads(key, batch_size)
    value = self.separate_heads(value, batch_size)
    attention, weights = self.attention(query, key, value)
    attention = tf.transpose(attention, perm=[0, 2, 1, 3])
    concat_attention = tf.reshape(attention, (batch_size, -1, self.embed_dim))
    output = self.combine_heads(concat_attention)
    return output
```

`call` 은 이 부품들을 순서대로 실행하는 **전체 흐름**이에요.

- `batch_size = tf.shape(inputs)[0]` — 입력의 첫 차원에서 **배치 크기**를 읽습니다.
- `query/key/value = ...dense(inputs)` — 입력을 $Q \cdot K \cdot V$ 세 갈래로 **선형변환**합니다.
- `separate_heads(...)` 세 번 — 각각을 헤드별로 **분리**합니다.

4. `self.attention(query, key, value)` — 헤드별로 앞의 어텐션 공식을 계산합니다.
5. `tf.transpose(...)` 후 `tf.reshape(...)` — 헤드별 결과를 다시 원래 축 순서로 돌리고 **이어붙여 (concatenate)** 하나의 큰 벡터로 합칩니다.
6. `output = self.combine_heads(concat_attention)` — 마지막으로 $W^O (= \text{combine_heads})$ 로 한 번 더 사영해 마무리합니다.

🎯 **핵심** — 여기까지가 **표준 트랜스포머의 어텐션 한 개**, 즉 논문의 **멀티-헤드(식 18-19)**입니다. $Q \cdot K \cdot V$ 선형변환 → 헤드 분리 → 헤드별 어텐션 → 다시 합쳐 W^O 사영. 다음에 볼 `TransformerBlock` 이 바로 이 부품을 **여러 개** 거느리면서 멀티-트랜스포머가 탄생해요.

TransformerBlock 클래스 — ★멀티-트랜스포머 핵심(AMH/배깅)★

드디어 이 노트북의 하이라이트입니다. 방금 만든 어텐션 부품을 **여러 개 묶어 평균 내는** 곳이에요.

```
class TransformerBlock(layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super().__init__()
        self.Bagging = 5
        nb_dict = {}
        for i in range(self.Bagging):
            nb_dict["att" + str(i)] = MultiHeadSelfAttention(embed_dim, num_heads)
        self.nb_dict = nb_dict
        self.ffn = keras.Sequential([
            layers.Dense(ff_dim, activation="relu"),
            layers.Dense(embed_dim),
        ])
        self.layernorm1 = layers.LayerNormalization(epsilon=1e-6)
        self.layernorm2 = layers.LayerNormalization(epsilon=1e-6)
        self.dropout1 = layers.Dropout(rate)
        self.dropout2 = layers.Dropout(rate)
```

생성자에서 가장 중요한 줄은 단연 `self.Bagging = 5` 예요.

- `self.Bagging = 5` — 어텐션 유닛을 **몇 개** 돌지 정합니다. 여기선 5개예요.
- `for i in range(self.Bagging): nb_dict["att"+str(i)] = MultiHeadSelfAttention(...)` — 그래서 앞에서 만든 `MultiHeadSelfAttention` 을 `att0` 부터 `att4` 까지 **다섯 개** 만들어 딕셔너리에 담습니다. 이게 논문이 말한 "여러 어텐션의 평균"의 재료예요.
- `self.ffn` — 어텐션 뒤에 붙는 **완전연결 순방향 신경망(FFN, feed-forward network)**입니다. `Dense(ff_dim, relu)` 로 늘렸다가 `Dense(embed_dim)` 으로 다시 줄여요.
- `layernorm1` , `layernorm2` — 두 개의 **층 정규화(layer normalization)**. 잔차 연결 뒤 값을 안정적으로 다듬습니다.
- `dropout1` , `dropout2` — 두 개의 **드롭아웃**. 과적합을 막는 동시에, 곧 볼 배깅에서 결정적 역할을 해요.

📝 **예시** — 표준 트랜스포머라면 이 자리에 어텐션이 딱 1개만 있어요(`4_T_GARCH` 노트북이 그렇습니다). 그런데 이 멀티-트랜스포머는 `att0 ~ att4` , 무려 5개를 준비합니다. 이 개수 하나의 차이가 T와 MT를 가르는 전부예요.

```
def call(self, inputs, training):
    attn_output = 0
    for i in range(self.Bagging):
        aux = self.dropout1(inputs, training=training)
        attn_output = attn_output + self.nb_dict["att" + str(i)](aux) / self.Bagging
    attn_output = self.dropout1(attn_output, training=training)
    out1 = self.layernorm1(inputs + attn_output)
    ffn_output = self.ffn(out1)
    ffn_output = self.dropout2(ffn_output, training=training)
    return self.layernorm2(out1 + ffn_output)
```

이 `call` 안의 `for` 반복문이 바로 논문 식 20-21, 즉 AMH의 정체입니다. 수식으로 쓰면 이래요.

$$\text{AMH}(X) = \frac{1}{n} \sum_{i=1}^n \text{MultiHead}_i(X), \quad n = 5$$

기호를 풀면, X 는 입력, $\text{MultiHead}_i(X)$ 는 i 번째 어텐션 유닛(`att i`)의 출력, $n = 5$ 는 어텐션 개수(`self.Bagging`), $\frac{1}{n} \sum$ 은 그 5개를 평균낸다는 뜻이에요. 반복문을 한 줄씩 대응시켜 볼게요.

- `attn_output = 0` — 누적할 그릇을 0으로 시작합니다($\sum$ 의 초기값).
- `aux = self.dropout1(inputs, training=training)` — 매 반복마다 입력에 드롭아웃을 새로 겁니다. 그래서 `att0` 이 보는 입력과 `att1` 이 보는 입력이 서로 조금씩 달라져요. 이게 곧 배깅(bagging, bootstrap aggregating) 효과입니다 — 서로 다른(교란된) 데이터로 여러 학습기를 만드는 것.
- `attn_output = attn_output + self.nb_dict["att"+str(i)](aux) / self.Bagging` — i 번째 어텐션의 출력을 미리 5로 나눠서 누적합니다. 5개를 다 더하면 결국 $\frac{1}{5} \sum$, 즉 평균이 되죠.

이렇게 얻은 평균 어텐션 뒤로는 표준 트랜스포머와 똑같은 뒷단이 이어집니다.

- `out1 = self.layernorm1(inputs + attn_output)` — 입력을 그대로 더하는 잔차 연결(residual connection) 뒤 층 정규화. 수식으로 $\text{out}_1 = \text{LayerNorm}(X + \text{AMH}(X))$.
- `ffn_output = self.ffn(out1)` 후 `return self.layernorm2(out1 + ffn_output)` — FFN을 통과시키고 다시 잔차 연결 + 정규화. 수식으로 $\text{output} = \text{LayerNorm}(\text{out}_1 + \text{FFN}(\text{out}_1))$.

전체 블록을 두 줄의 수식으로 정리하면 이렇습니다.

$$\begin{aligned} \text{out}_1 &= \text{LayerNorm}(X + \text{AMH}(X)) \\ \text{output} &= \text{LayerNorm}(\text{out}_1 + \text{FFN}(\text{out}_1)) \end{aligned}$$

🔗 핵심 — 표준 트랜스포머는 이 자리에 단일 어텐션 하나를 놓고, 멀티-트랜스포머는 5개를 평균 낸 AMH를 놓습니다. 딱 이 한 가지가 T와 MT를 가르는 전부예요. 나머지 잔차 연결·FFN·정규화는 완전히 똑같습니다.

💡 쉬운 설명 — 그럼 왜 굳이 평균을 낼까요? 트랜스포머 하나는 무작위 초기화와 드롭아웃 탓에 예측의 분산(variance)이 큼니다. 들쭉날쭉하다는 뜻이죠. 그런데 서로 조금씩 다른 여러 개를 평균내면, 튀는 값들이 서로 상쇄되어 분산이 줄고 과적합에도 강해집니다. 이게 앙상블 학습(ensemble learning)의 원리이고, 논문이 위기 국면에서 MT가 더 안정적인 이유로 꼽는 대목이에요.

GARCH 보조 함수 요약

이 노트북에는 신경망 말고도, 비교 대상인 전통 **GARCH** 계열과 데이터 준비를 위한 도우미 함수들이 함께 들어 있어요. 코드가 길어 핵심만 정리합니다.

- **ReturnCalculation** — 가격을 **로그수익률(log return)**로 바꿉니다. 수식으로 $r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$ 이예요. 여기서 P_t 는 t 시점 가격, $\ln$ 은 자연로그입니다. 로그수익률은 시간에 걸쳐 더하기 쉬워 금융에서 즐겨 씁니다.
- **SDCalculation** / **TrueSDCalculation** — 신경망이 맞혀야 할 **정답 라벨인 실현 변동성(realized volatility)**을 계산합니다. 일정 구간 수익률의 표준편차로 "그 기간 실제 변동성"을 만들어 학습 타깃으로 삼아요.
- **DatabaseGeneration** — **슬라이딩 윈도우(sliding window)**로 데이터를 훑으며, 과거 며칠(입력)과 그 다음 변동성(타깃) 쌍을 잔뜩 만들고 **학습/검증/테스트로 분할**합니다.
- **6개 GARCH 적합 함수** — **GARCH_Model_Student**, **GJR_GARCH_Model_Student**, **TARCH_Model_Student**, **EGARCH_Model_Student**, **AVGARCH_Model_Student**, **FIGARCH_Model_Student**. 모두 오차가 **스튜던트 t 분포**를 따른다고 가정하는 전통 변동성 모델들로, 신경망의 성능을 견줄 **벤치마크**예요.
- **Transformer_Model** — 앞서 만든 **TransformerBlock**을 케라스 모델로 **조립**합니다. 흐름은 **입력 → 위치 인코딩(또는 LSTM) → TransformerBlock → 변동성 출력**이예요.

⚠ **주의** — 여섯 GARCH 함수 이름 끝에 붙은 **_Student**를 놓치지 마세요. 모두 정규분포가 아니라 **스튜던트 t 분포**를 가정한다는 표시예요. t 분포는 꼬리가 두꺼워, 금융 수익률에 흔한 **극단적 급등락**을 정규분포보다 잘 담아냅니다.

6개 노트북 ↔ 논문 모델 매핑

이 프로젝트는 총 여섯 개의 노트북으로 서로 다른 모델을 비교해요. 헷갈리기 쉬우니 표로 한눈에 정리합니다.

노트북	모델	특징
1_ANN_ARCH	ANN	기본 신경망 벤치마크
2_LSTM_GARCH	LSTM	순환신경망 벤치마크
4_T_GARCH	T (단일 트랜스포머)	Bagging = 1 (평균 없음)
3_MT_GARCH	MT (멀티-트랜스포머)	Bagging = 5 (AMH 평균)
6_TL_GARCH	TL	T + LSTM 층
5_MTL_GARCH	MTL	MT + LSTM 층

이 여섯 모델은 **두 개의 축**으로 이해하면 깔끔해요.

- **축 ①: T vs MT** — 어텐션이 **1개냐**(**Bagging=1**), **5개 평균이냐**(**Bagging=5** , AMH). 방금 본 그 **self.Bagging** 숫자 하나의 차이예요.
- **축 ②: + L(LSTM) 유무** — 뒤에 **LSTM 층**을 붙이느냐. LSTM을 붙이면 **위치 인코딩(positional encoding)**이 **필요 없어집니다**. 순서 정보를 LSTM이 알아서 담당하기 때문이에요.

🎯 **핵심** — 논문의 결론은 이렇습니다. **MT·MTL**이 **T·TL**과 전통 **GARCH**를 대부분의 오차지표에서 능가했고, 특히 **위기(고변동) 국면에서 더 안정적**이었어요. 그 원동력이 바로 우리가 코드로 확인한 **Bagging=5** **평균, 곧 AMH**입니다. "여러 번 시도해 평균 낸다"는 한 가지 철학이 헤드 단위(멀티-헤드)에서 모델 단위(멀티-트랜스포머)까지 그대로 반복된 셈이죠.

종합 마무리 — M3 전체를 한 줄로 꿰기

여기까지 오느라 정말 고생 많았습니다. 마지막으로 큰 그림을 다시 한 번 이어 붙여 봅시다. M3의 전체 이야기는 결국 이 한 문장입니다 — "딥러닝으로 꼬리 위험(tail loss)과 변동성을 더 잘 추정하기."

흐름을 되짚어 보면 이렇습니다.

- **레슨 1~2 (DeepVaR)** — LSTM 기반 **DeepAR**로 수익률의 **전체 분포**를 예측하고, 그 분포의 **분위수(quantile)**에서 VaR를 뽑았습니다. 그리고 그 VaR가 믿을 만한지 **Kupiec·Christoffersen 백테스트**로 검증했습니다.
- **레슨 3~4 (트랜스포머)** — 순환구조 대신 **셀프 어텐션(self-attention)**으로 시계열의 장거리 의존성을 포착하는 법을 배우고, 어텐션을 여러 개 평균낸 **멀티-트랜스포머(AMH)**로 변동성 예측의 분산을 줄였습니다.
- **논문 2편**이 이 두 접근의 학술적 근거를, **코드 3개**(`backtest.py`, DeepVaR 노트북, Multi-Transformer 노트북)가 실제 구현을 보여주었습니다.

🎯 **핵심** — 금융 시계열은 노이즈가 크고 꼬리가 두꺼워 단일 모델은 불안정합니다. DeepVaR는 **분포 전체를 예측**해서, 멀티-트랜스포머는 **여러 모델을 평균**내서 이 불안정성에 대응합니다. 두 아이디어 모두 "점 하나가 아니라 불확실성 자체를 다룬다"는 리스크 관리의 근본 철학을 딥러닝으로 구현한 것입니다.

두 논문을 관통하는 다섯 가지 공통 교훈

두 논문은 출발점(FX 수익률 분포 vs S&P 변동성)이 다르지만, 놀랍도록 같은 방향을 가리킵니다. 시험 직전에 이 다섯 줄만 기억해도 절반은 됩니다.

#	교훈	한 줄 요약
1	전통 계량모형의 한계	GARCH·HS·RM은 위기 때 무너지고, 특히 몽친 위반(clustered violations) 과 두꺼운 꼬리(fat tails) 를 못 잡는다
2	딥러닝의 개선	LSTM·DeepAR·트랜스포머가 이를 크게 개선하며, 코로나 같은 극단기에서 격차가 가장 크다
3	배깅(앙상블)이 핵심 무기	DeepVaR는 10개 모델 을 배깅, 멀티-트랜스포머는 어텐션 자체 를 배깅한다
4	스튜던트 t 분포	꼬리를 잡으려면 정규분포보다 스튜던트 t 분포 가 낫다 (두 논문 모두 채택)
5	검증은 필수	모델이 좋다고 끝이 아니라, 반드시 Kupiec·Christoffersen·DQ 검정 으로 위험추정의 타당성을 확인해야 한다

💡 **쉬운 설명** — 3번 "배깅"이 두 논문을 잇는 가장 예쁜 다리입니다. "여러 번 시도해서 평균내면 더 안정적이다"라는 단순한 통계 원리가, 한쪽에서는 **모델 10개를 평균**으로, 다른 쪽에서는 ***어텐션 5개를 평균(AMH)***으로 각각 나타난 것뿐입니다. 규모와 위치만 다를 뿐 정신은 똑같습니다.

 **예시** — "코로나(2020)에서 격차가 가장 크다"는 교훈을 숫자 감각으로 붙잡아 두세요. 평상시에는 전통 GARCH와 딥러닝 하이브리드의 오차 차이가 크지 않다가, 변동성이 폭등한 2020년에 전통 모델의 오차가 급증하며 벌어집니다. 위험모델의 가치는 바로 **이런 위기 때** 판가를 난다는 뜻입니다.

다음 단계 예고 — M4: 극단값 이론(EVT)

DeepVaR 논문의 관련 연구에서 살짝 등장했던 **극단값 이론(Extreme Value Theory, EVT)**이 다음 모듈 M4의 주제입니다. EVT는 분포 '전체'가 아니라 **극단(꼬리) 그 자체의 극한 분포**에만 집중하는 접근으로, 두 가지 대표 방법이 있습니다 — **임계값 초과 모델(Peaks-Over-Threshold, POT)**과 **블록 최대값 모델(Block Maxima, BM)**. 문헌은 일반 신뢰수준에선 전통 방법과 비슷하더라도, **고신뢰수준의 극단손실 예측**에서는 **EVT가 가장 정확**할 수 있다고 봅니다.

 **핵심** — M3가 "분포 전체를 잘 예측하자(DeepVaR)"와 "여러 모델을 평균내자(Multi-Transformer)"였다면, M4는 **"꼬리만 따로, 극단 이론으로 정밀하게"**입니다. 세 관점은 경쟁이 아니라 상호보완입니다.

수고하셨습니다. 이 노트를 시험 전에 한 번, 코드 실습 전에 한 번 더 훑어보면, 수식과 코드가 서로 어떻게 연결되는지 자연스럽게 손에 잡힐 것입니다. 🎓